

Model order reduction for a transient problem using the Laplace transform

Keyvan Attarian^{1,3} and Amine Ammar^{2,3}

¹Ecole polytechnique, Institut Polytechnique de Paris

²LAMPA Laboratory & ESI Group Chair, Arts et Metiers Institute of Technology

³DesCartes Program, WP8, CNRS@CREATE

Printed July 22, 2026

Abstract

Deep-learning reduced-order models (DL-ROMs) predict a full spatiotemporal field from a parameter vector, but they all retain the full temporal resolution of the trajectory: one latent code per time step, so the surrogate’s output size and cost scale with the number of time steps N_t . We propose to compress the time axis itself with the Laplace transform, predicting $K \ll N_t$ complex spectral coefficients in a single shot and recovering the trajectory by a differentiable regularized inversion. Two instantiations are studied and compared, depending on where the transform is applied: in the physical domain (SLAE, K frequency frames to decode) or in the latent space (LLAE). We further derive a bias–amplification bound showing that everything the regularized inversion does to the reconstruction is carried by two scalars depending on the triplet (s_k, α_t, λ) — the poles and the regularisation weights together — and use it to select that triplet offline, without training any network, rather than fixing it by convention.

On a parametric transient combustion problem ($N_t = 150$), both instantiations improve substantially on a linear POD surrogate in the Laplace domain (8.91% median relative L^2 error): LLAE reaches 4.13% and SLAE 5.04% with an optimised contour. A Laplace-free DL-ROM ablation reaches 3.80%, so temporal compression is not what buys accuracy — the non-linear autoencoder is — and it costs 0.4 to 3.7 points. What it buys is cost and flexibility at a fixed output: at identical 150-step output and measured in a single benchmarking run, SLAE runs $2.0\times$ faster than the DL-ROM on $3.9\times$ less memory with a $6.8\times$ smaller surrogate, and a single trained Laplace model answers at any output resolution, where the DL-ROM requires one training per temporal grid. Optimising the inversion is the cheapest accuracy in the study: it improves the deployed error at every K , by up to 1.85 pp, at no inference cost.

Contents

1	Introduction	3
2	Methodology	5
3	Results	18

4	Comparison and Discussion	43
5	Conclusion	53
	Appendices	55

1 Introduction

High-fidelity simulations of parametric transient partial differential equations (PDEs), such as those governing combustion, reactive transport, and atmospheric dispersion, rely on computationally expensive numerical solvers. In many-query contexts like optimization, uncertainty quantification, real-time control, and digital twins, the complete spatiotemporal field $U(x, t; \mu)$ must be evaluated for thousands of parameter values μ [8, 23]. The prohibitive cost of these repeated evaluations motivates the development of surrogate models capable of reproducing the full dynamics in milliseconds. Applications such as air quality and pollution forecasting concretely illustrate this need: global physical models are accurate but unusable in real-time without approximation [12, 5].

Classical model order reduction (MOR) methods [14], such as Reduced Basis (RB) and Proper Orthogonal Decomposition (POD) combined with interpolation, construct a low-dimensional linear subspace by projecting high-fidelity solutions onto spatial modes. These approaches are theoretically well-founded and highly effective for smooth dynamics. However, they suffer from a **reconstruction floor** linked to the linearity of the basis: for problems with sharp transients, localized structures, or strongly non-linear dynamics (such as convection-diffusion-reaction equations), the number of required modes explodes. The transition toward **non-linear manifolds** becomes necessary as linear techniques struggle [6].

Beyond the spatial compression bottleneck, classical MOR methods also struggle with the temporal dimension. The treatment of time is either sequential (time-stepping on the projected ROM) or space-time (variational), both of which impose a computational cost that grows with the temporal horizon [9, 18, 27]. This dual bottleneck — spatial and temporal — motivates the approach developed in this paper. The **parameter-to-field** setting — predicting a full spatiotemporal field from a low-dimensional parameter vector $\mu \in \mathbb{R}^p$ — is addressed by **DL-ROMs** (Deep Learning Reduced-Order Models) [19, 6, 7]: a convolutional autoencoder compresses the spatial field into a compact latent code, a surrogate network learns $\mu \mapsto z$, and the decoder reconstructs the full field at inference, yielding significant gains over linear ROMs. This is a different problem from **field-to-field** operator learning (FNO [20], DeepONet [21, 22]), which takes an input function rather than a scalar parameter vector; the two settings are not directly comparable. Parameter-conditioned U-Nets [16] represent a lighter DL-ROM variant for 2D problems, operating directly in the time domain. However, all existing DL-ROM approaches share a common temporal bottleneck: they retain the full temporal resolution of the trajectory, one latent vector per time step. Whether the surrogate takes time as an input coordinate and is queried once per step, $g(\mu, t) \mapsto z(t)$ [6], or propagates the latent sequentially in time, often autoregressively [26] at the cost of error accumulation, the temporal work — the number of network evaluations, or the output size — scales with N_t . For long transients, this makes the learning problem harder and inference heavier, regardless of how compact the spatial latent is.

A natural path to address this bottleneck is to work in the **frequency domain**: compressing the time axis into $K \ll N_t$ spectral coefficients reduces the surrogate output dimension dramatically. The spectral intuition is well-established in the neural operator literature — FNO [20] parameterizes its kernel in Fourier space — but the Fourier transform is poorly suited for transient signals: it assumes periodicity and exhibits spectral leakage for non-periodic dynamics. The **Laplace transform** $s = \gamma + i\omega$ generalizes Fourier by adding a real decay axis γ that naturally encodes transients, exponential stability, and non-periodic regimes. Each Laplace frequency field $\hat{U}(s_k)$ is a static spatial

field, structurally simpler to compress than a full temporal sequence. The Laplace Neural Operator (LNO) [2] confirms this advantage in the field-to-field setting. On the linear reduction side, Reyes [29] and Goeckner et al. [10] show that Laplace-domain POD bases are exponentially accurate for parametric evolution problems — yet these approaches remain fundamentally linear. The remaining gap is to combine the Laplace spectral compression with the non-linear expressivity of autoencoders within the DL-ROM framework, enabling **end-to-end** differentiable training thanks to the exact invertibility of the transform [11].

This paper fills that gap. We situate our work in the DL-ROM family and introduce the Laplace transform as a **temporal** compression stage on top of the standard AE-surrogate pipeline: instead of predicting N_t latent frames, the surrogate predicts $K \ll N_t$ spectral coefficients, from which the full trajectory is recovered by inversion. Here $U(t) \in \mathbb{R}^{N_x}$ is the discretized spatial field at time step t , N_x the number of spatial nodes, N_t the number of time steps, and K the number of Laplace frequencies. Two instantiations are studied depending on where the transform is applied — in the physical domain (SLAE) or in the latent space (LLAE) — and compared against two baselines that each remove one ingredient: a linear POD baseline in the Laplace domain, and a Laplace-free DL-ROM keeping the full temporal grid. Our contributions are as follows:

- **Temporal compression via the Laplace transform (main contribution):** we propose and evaluate using the Laplace transform to compress the temporal dimension of the surrogate target from N_t frames to $K \ll N_t$ spectral coefficients, making the surrogate’s size and output dimension independent of the temporal horizon. Whether the **complete** inference cost inherits that independence turns out to depend on where the transform is applied, and we quantify this in Section 4.2: it does for SLAE, whose decoder sees only the K frequency frames, and does not for LLAE, whose decoder still emits N_t physical frames. Independently of that, because the trajectory is materialised by a regularized linear solve assembled on whatever time nodes are requested, the predicted coefficients refer to no time grid at all: a single trained model answers at **any** output resolution — any Δt at fixed horizon T — whereas a model that predicts one latent per time step is tied to the grid it was trained on, or at best pays one network evaluation per requested step.
- **SLAE vs. LLAE:** we systematically compare two instantiations of the Laplace compression — in the physical domain (SLAE) and in the latent space (LLAE) — and characterize the accuracy/latency trade-off they offer.
- **Designing the regularized inversion:** we treat the inversion as an object to be optimised rather than fixed by convention, and by that we mean the triplet (s_k, α_t, λ) — the poles together with the two regularisation weights — not the contour alone. For LLAE the triplet is learned end-to-end alongside the surrogate; for SLAE we derive a bias–amplification decomposition of the reconstruction error and optimise it offline, before any training, by minimizing a principled proxy loss that trades regularization bias against the amplification of the surrogate approximation error.

The remainder of this paper is organized as follows. Section 2 describes the methodology, detailing the Laplace transform, the two autoencoder families, the surrogate architectures and the two baselines. Section 3 presents the numerical results. Section 4 compares the pipelines on accuracy against cost, discusses the limitations of the study and outlines perspectives, and Section 5 concludes. The appendices detail the governing equations, the architectural hyperparameters and the derivations behind the contour optimisation.

2 Methodology

2.1 Laplace Transform and Inversion

Throughout this section, $u(t) \in \mathbb{R}$ denotes the field value at a single fixed spatial location (lowercase = one pixel); for the full spatial field $U(t) \in \mathbb{R}^{N_x}$ (uppercase = full grid of N_x nodes), all operations below are applied independently at each grid point. The forward Laplace transform is evaluated at discrete frequencies $s = \gamma + i\omega$ using a temporal quadrature rule (e.g., trapezoidal rule with weights w_n):

$$(1) \quad \hat{u}(s) = \int_0^T u(t) e^{-st} dt \approx \Delta t \sum_{n=0}^{N_t-1} w_n u(t_n) e^{-st_n}$$

Since $u(t)$ is strictly real-valued, its Laplace transform satisfies the conjugate symmetry property $\hat{u}(\bar{s}) = \overline{\hat{u}(s)}$. Consequently, we restrict our numerical evaluation to frequencies with non-negative imaginary parts $\omega_k \geq 0$, which halves the forward evaluation cost while providing the full spectral information required for inversion.

Regularized Inversion (Tikhonov). The inversion is posed not on the K stored frequencies but on their **conjugate-symmetric completion**

$$(2) \quad \mathcal{S} = \{s_k\}_{k=1}^K \cup \{\bar{s}_k : \omega_k > 0\}, \quad K_{\text{full}} = |\mathcal{S}| = 2K - \#\{k : \omega_k = 0\},$$

whose additional members are known at no cost from the stored half-spectrum through $\hat{u}(\bar{s}_k) = \overline{\hat{u}(s_k)}$. This is a free doubling of the sampling points: the conjugate coefficients require no additional forward evaluation, no additional surrogate output and no additional storage — they are obtained by changing a sign — yet they enter the least-squares problem below as genuine extra constraints. The inversion is thus posed on $K_{\text{full}} \simeq 2K$ points while the network only ever predicts K of them, which is the same economy that motivated restricting the forward evaluation to $\omega_k \geq 0$ in the first place. Let $\mathbf{F} \in \mathbb{C}^{K_{\text{full}} \times N_t}$ be the forward evaluation matrix assembled on $\mathcal{S}$, with $\mathbf{F}_{k,n} = \Delta t w_n e^{-s_k t_n}$, and let $\hat{\mathbf{u}} \in \mathbb{C}^{K_{\text{full}}}$, $\mathbf{u} \in \mathbb{R}^{N_t}$ be the corresponding vectors of Laplace coefficients and temporal values. Throughout this paper $\mathbf{F}$ and $\hat{\mathbf{u}}$ always denote these conjugate-extended objects. Taking advantage of the real-valued constraint on $\mathbf{u}$, the reconstruction is formulated as a regularized complex least-squares problem:

$$(3) \quad \mathbf{u}^* = \arg \min_{\mathbf{u} \in \mathbb{R}^{N_t}} \|\mathbf{F}\mathbf{u} - \hat{\mathbf{u}}\|_{\mathbb{C}^{K_{\text{full}}}}^2 + \alpha_t \|\mathbf{D}_t \mathbf{u}\|_2^2 + \lambda \|\mathbf{u}\|_2^2$$

where $\mathbf{D}_t \in \mathbb{R}^{(N_t-1) \times N_t}$ is the first-difference operator penalizing temporal roughness, and $\lambda > 0$ is a Tikhonov ridge regularization parameter. The corresponding optimality condition (normal equations) yields a real symmetric linear system:

$$(4) \quad \left(\Re(\mathbf{F}^H \mathbf{F}) + \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I} \right) \mathbf{u}^* = \Re(\mathbf{F}^H \hat{\mathbf{u}})$$

Two properties of $\mathcal{S}$ are worth making explicit, because the half-spectrum $\{s_k\}_{k=1}^K$ alone would give a different problem. First, a real $\mathbf{u}$ produces conjugate residuals at s_k and $\bar{s}_k$, so restoring the conjugates amounts to weighting every frequency with $\omega_k > 0$ twice in (3) relative to a purely real pole — the discrete counterpart of integrating along the whole Bromwich contour rather than over $\omega \geq 0$ only. Second, on $\mathcal{S}$ both $\mathbf{F}^H \mathbf{F}$ and $\mathbf{F}^H \hat{\mathbf{u}}$ are real, their imaginary parts cancelling pairwise between conjugate members, so the

operator $\Re(\cdot)$ in (4) records the real-valuedness of $\mathbf{u}$ without discarding any information. The linear system (4) is assembled and factorised in double precision — at $K = 16$ the columns of $\mathbf{F}$ become nearly collinear and $\kappa(\mathbf{A})$ exceeds the reach of single precision — by a Cholesky factorisation when $\mathbf{A}$ is reused across a batch of nodes, and by a dense LU solve otherwise; it is $N_t \times N_t$, so this costs nothing. The operator is fully differentiable in (s_k, α_t, λ) and compatible with gradient-based training, a property Section 2.4 exploits. Crucially, this formulation holds for any set of frequencies (s_k) , with no constraint on their distribution in the complex plane.

Choice of contour: truncated Bromwich. A key design choice is the set of evaluation frequencies $(s_1, \dots, s_K)$. We adopt throughout the **truncated Bromwich contour**: a vertical line in the complex plane at $\text{Re}(s) = \gamma$, with imaginary parts equal to the first K discrete Fourier frequencies,

$$(5) \quad s_k = \gamma + i \frac{2\pi k}{N_t \Delta t}, \quad k = 0, \dots, K - 1,$$

where $\gamma \geq 0$ is the **damping parameter** and $K \leq \lfloor N_t/2 \rfloor + 1$ controls the spectral truncation. We keep $\gamma \geq 0$ rather than $\gamma > 0$ because $\gamma = 0$ — a purely imaginary contour, i.e. a truncated DFT — is one of the settings we report, and because the contour optimisation of Section 2.4 is allowed to return slightly negative abscissae; over a finite horizon the quadrature (1) is a finite sum and remains well defined for any $s \in \mathbb{C}$. With this choice, the forward sum (1) coincides with a truncated DFT of the exponentially damped sequence:

$$(6) \quad \hat{u}(s_k) = \text{DFT}\{u(t_n) \Delta t w_n e^{-\gamma t_n}\}_k, \quad k = 0, \dots, K - 1,$$

and could therefore be evaluated by an FFT; since the transform is applied either to N_x pixels (SLAE) or to d_z latent channels (LLAE) with $K \leq 32$ and $N_t = 150$, our implementation simply forms the dense $K \times N_t$ matrix product, whose $O(KN_t)$ cost per node is negligible against the convolutional stages. Since $K < \lfloor N_t/2 \rfloor + 1$, the spectrum is incomplete and we rely on the Tikhonov inversion (3)–(4) in all cases. Restricting to the Bromwich structure, however, imposes that all poles share the same real part γ and lie on a regular frequency grid — a strong constraint that may be suboptimal. Since the Tikhonov inversion handles any set of frequencies, one can in principle place the poles freely in the complex plane to better capture the signal’s dynamics. This is the motivation for Section 2.4, which discusses how to optimize the contour beyond the Bromwich family.

2.2 Autoencoders in the Laplace Domain

We investigate two autoencoder architectures that differ solely in **where** the Laplace transform is applied, as illustrated in Figure 1. In the **Spatial Laplace AE (SLAE)**, the transform is applied first in the physical domain, and a frequency-conditioned autoencoder then compresses each of the K resulting complex frequency frames independently. In the **Latent Laplace AE (LLAE)**, the order is reversed: a time-conditioned autoencoder first encodes the full temporal sequence into a compact latent trajectory, and the Laplace transform is subsequently applied within that low-dimensional space.

In both cases, an L_2 ridge penalty on the latent representations enforces a sufficiently regular structure, which makes the latent space easier for the surrogate to navigate (Section 2.3).

Both architectures build on the same convolutional encoder–decoder backbone, applied directly to the spatial field with no POD pre-reduction; this is the standard deep

convolutional-autoencoder compression of the DL-ROM lineage [19, 6], as opposed to the POD-then-autoencoder pre-reduction of POD-DL-ROM [7]. On top of this shared backbone, both architectures use the same conditioning mechanism: a sinusoidal coordinate encoding combined with Feature-wise Linear Modulation (FiLM) [28]. The coordinate $\tau \in [0, 1]$ represents either the frequency ratio $f_k = k/(K - 1)$ for SLAE, or the normalised time step t/T for LLAE.

Sinusoidal Coordinate Encoding. The coordinate $\tau \in [0, 1]$ is embedded via sinusoidal basis functions inspired by NeRF positional encoding [24]:

$$(7) \quad e(\tau) = \text{MLP}\left(\left[\sin(2^i \pi \tau), \cos(2^i \pi \tau)\right]_{i=0}^{L-1}\right), \quad L = 8.$$

A plain scalar τ would conflate nearby coordinates and fail to capture sharp transitions in either the frequency or time domains. The multi-scale sinusoidal basis explicitly represents τ at L different resolutions, providing the network with a rich, non-saturating signal that allows fine discrimination across the entire range.

FiLM Conditioning. To share encoder/decoder weights across all coordinates (frequencies or time steps) while adapting to each specific configuration, the FiLM mechanism is employed. At each convolutional layer ℓ , the coordinate embedding $e(\tau)$ is projected to a channel-wise gain $\alpha_\ell \in \mathbb{R}^{C_\ell}$ and bias $\delta_\ell \in \mathbb{R}^{C_\ell}$, modulating the activations element-wise:

$$(8) \quad \mathbf{x}_\ell \leftarrow (\mathbf{x}_\ell \odot (1 + \alpha_\ell(e(\tau)))) + \delta_\ell(e(\tau)).$$

The residual formulation $(1 + \alpha_\ell)$ initializes the modulation close to the identity, which stabilizes early training. All coordinates share the same convolutional weights, and the FiLM parameters act as a lightweight per-coordinate adapter at each layer, enabling a single autoencoder to process the full spectrum or temporal sequence without training separate model copies.

2.2.1 Spatial Laplace AE (SLAE)

SLAE first applies the Laplace transform at the pixel level, converting the N_t -frame temporal sequence $U(t)$ into K complex-valued spatial fields $\hat{U}(s_k)$. Each frequency frame, represented as a two-channel (real/imaginary) image, is independently compressed by a FiLM-conditioned convolutional encoder into a real latent vector $z(s_k) \in \mathbb{R}^{d_z}$:

$$(9) \quad U(t) \xrightarrow{\mathcal{L}_{\text{pixel-wise}}} \hat{U}(s_k) \xrightarrow{\text{Encoder}(f_k)} z(s_k) \xrightarrow{\text{Decoder}(f_k)} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

The decoder reconstructs $\hat{U}(s_k)$ from $z(s_k)$ via separate output heads for the real and imaginary components. Training minimizes the Laplace-domain reconstruction error together with an L_2 regularization term:

$$(10) \quad \mathcal{L}_{\text{SLAE}} = \|\tilde{U}(s_k) - \hat{U}(s_k)\|_{\mathbb{C}^{N_x}}^2 + \beta \|z(s_k)\|_2^2$$

where $\|\cdot\|_{\mathbb{C}^{N_x}}^2$ is the squared L_2 norm over the N_x spatial nodes (real and imaginary parts combined), and β is a regularization hyperparameter.

2.2.2 Latent Laplace AE (LLAE)

LLAE encodes the physical frames $U(t)$ into a latent trajectory $z(t) \in \mathbb{R}^{d_z}$ using a time-conditioned convolutional encoder (conditioning on the normalised ratio t/T), then applies the Laplace transform directly within this compact latent space:

$$(11) \quad U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\mathcal{L}} \hat{z}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t)$$

Compressing in the latent space rather than the physical domain offers two advantages: the transform operates on d_z -dimensional vectors instead of N_x -dimensional fields, and the encoder produces a smoother temporal representation than the raw complex frequency frames, easing the spatial compression task. The main counterpart is computational: at each training step and at inference, the encoder and decoder must process all N_t physical frames, whereas SLAE processes only the K frequency frames. Since $N_t \gg K$, LLAE is significantly heavier, which limits the network capacity that can be deployed under a fixed compute budget. The loss combines three terms:

$$(12) \quad \begin{aligned} \mathcal{L}_{\text{LLAE}} = & \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|U_{\text{rec}}(t_n) - U(t_n)\|_2^2 \\ & + \beta_{\text{latent}} \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|\tilde{z}(t_n) - z(t_n)\|_2^2 \\ & + \beta \frac{1}{K} \sum_{k=0}^{K-1} \|\hat{z}(s_k)\|_{\mathbb{C}^{d_z}}^2 \end{aligned}$$

which combines the physical-time reconstruction error, a latent consistency term penalizing the round-trip error $\tilde{z}(t_n) - z(t_n)$, and an L_2 regularization penalty on the Laplace-domain latent codes $\hat{z}(s_k)$. The latent consistency term is unique to LLAE: it explicitly enforces the strict invertibility of the continuous Laplace transform within the learned discrete latent space, a structural constraint not required by SLAE since it encodes each frequency frame independently.

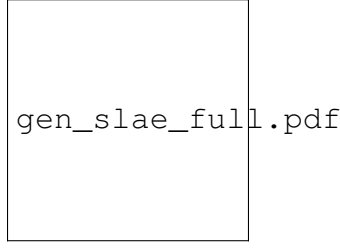
2.3 Surrogate Training

Following the DL-ROM paradigm of [6, 7], the surrogate MLP g_θ (shared trunk + per-frequency heads; architecture detailed in Figure 2 and Appendix F) replaces the encoding steps by learning the direct mapping $\mu \mapsto w_k$, where w_k denotes the Laplace-domain latent at frequency s_k that feeds the decoder: $w_k = z(s_k) \in \mathbb{R}^{d_z}$ for SLAE, and $w_k = \hat{z}(s_k) \in \mathbb{C}^{d_z}$ for LLAE. Two strategies for constructing the surrogate target w_k are described below.

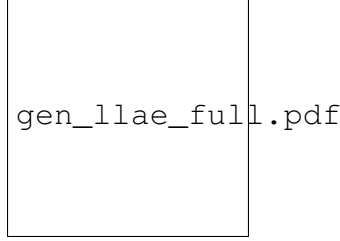
2.3.1 Direct Embedding Prediction

In the direct approach, the surrogate predicts the full latent vector at each index without any preliminary compression. For SLAE, the encoder produces frequency-domain latents $z(s_k) \in \mathbb{R}^{d_z}$ directly, and the pipeline is:

$$(13) \quad \mu \xrightarrow{g_\theta} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$



(a) SLAE: pixel-wise Laplace transform, then per-frequency convolutional AE ($d_z = 64, K = 16$).



(b) LLAE: convolutional AE first, then Laplace transform in latent space ($d_z = 64, K = 16$).

Figure 1: SLAE (a) and LLAE (b) autoencoder pipelines.

For LLAE, the time-domain latent sequence $z(t) \in \mathbb{R}^{d_z}$ is first transformed to the Laplace domain, and the surrogate predicts those Laplace-domain codes directly:

$$(14) \quad \mu \xrightarrow{g_\theta} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

2.3.2 Latent Sequence SVD Compression

To reduce the effective output dimension of the surrogate and simplify its learning task, one can apply a preliminary offline SVD compression to the Laplace-domain latents before training the surrogate, an approach inspired by [25]. For both SLAE and LLAE, the SVD is applied in the frequency domain: given the K Laplace-domain latent codes $\{\hat{z}(s_k)\}_{k=1}^K$ collected over the training set, a truncated SVD yields a projection matrix $V \in \mathbb{R}^{d_z \times k_{\text{svd}}}$ (with $k_{\text{svd}} \ll d_z$) such that $\hat{z}(s_k) \approx (\hat{z}(s_k)V)V^\top$. The reduced codes $\hat{G}(s_k) = \hat{z}(s_k)V \in \mathbb{R}^{k_{\text{svd}}}$ become the surrogate targets, reducing the prediction problem from dimension d_z to k_{svd} .

In the **SLAE-SVD** variant (latents indexed by frequency k): the projection yields $G(s_k) = z(s_k)V$, and the surrogate predicts in the compressed frequency-domain space:

$$(15) \quad \mu \xrightarrow{g_\theta} G_{\text{pred}}(s_k) \xrightarrow{V^\top} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

In the **LLAE-SVD** variant, the SVD is likewise applied in the Laplace domain: the projection yields $\hat{G}(s_k) = \hat{z}(s_k)V$, and the surrogate predicts the compressed frequency-domain latents before back-projection and inversion:

$$(16) \quad \mu \xrightarrow{g_\theta} \hat{G}_{\text{pred}}(s_k) \xrightarrow{V^\top} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

Because V is real and acts on the latent channel index while $\mathcal{L}^{-1}$ acts on the frequency index, the two commute: back-projecting before or after the inversion is equivalent, and the implementation applies whichever order is cheaper.

In both strategies, the entire pipeline is trained end-to-end to minimise the total physical-time loss:

$$(17) \quad \mathcal{L}_{\text{total}} = \alpha_{\text{lat}} \|w_k^{\text{pred}} - w_k^{\text{true}}\|^2 + \alpha_{\text{spat}} \|U_{\text{rec}} - U\|^2$$

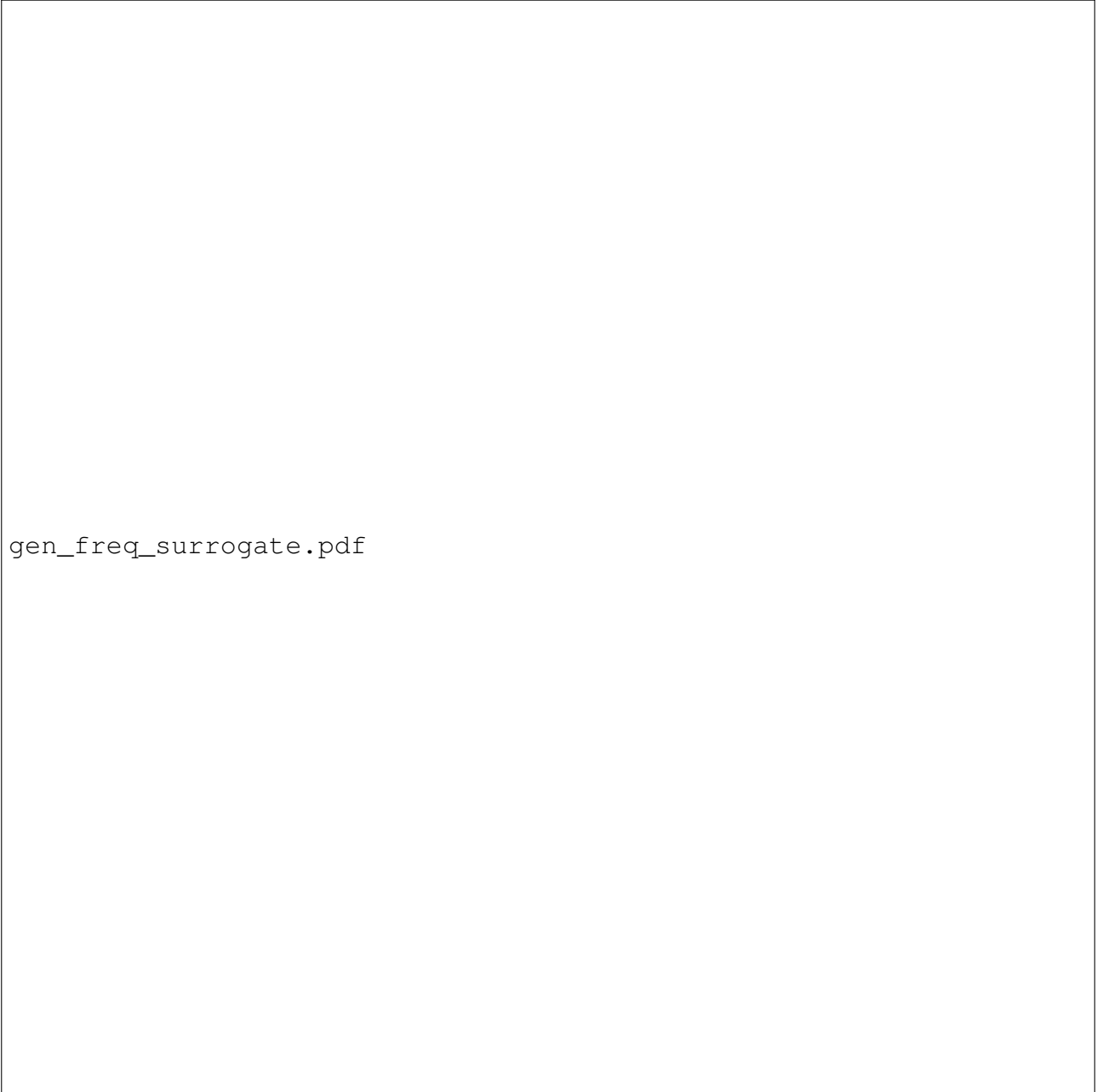


Figure 2: **Surrogate MLP architecture.** A shared trunk maps physical parameters μ to a latent representation, which is broadcast to K independent heads. Each head is conditioned on a frequency index via sinusoidal encoding and predicts the target latent w_k for that frequency.

where w_k^{pred} and w_k^{true} denote the predicted and true Laplace-domain latents (either $z(s_k)$ or $\hat{G}(s_k)$ depending on the chosen strategy). End-to-end backpropagation allows gradients from the spatiotemporal MSE to flow through the decoder and the inverse transform, jointly updating the surrogate weights, the AE decoder, and, in the SVD variant, the projection basis V .

2.3.3 Tucker Decomposition in Latent Space

Just like the SVD approach described in the previous section, the goal here is to further compress the latent representations of both LLAE and SLAE before training the surrogate MLP, which leads to two distinct variants: **LLAE-Tucker** and **SLAE-Tucker**. While the SVD approach reduces the latent sequence along a single mode at a time (for LLAE-SVD it collapses the latent dimension d_z ; for SLAE-SVD it collapses d_z independently at each frequency, ignoring cross-frequency correlations), Tucker decomposition [17] generalises this by simultaneously factorising two modes of the latent tensor.

Crucially, for both variants, the Tucker decomposition is applied in the latent space **after** the Laplace transform. We assemble the frequency-domain latents of the training set into a three-way tensor $\hat{\mathcal{Z}} \in \mathbb{C}^{N_{\text{train}} \times K \times d_z}$ (real-valued for SLAE, complex-valued for LLAE). A Tucker- (r_μ, r_s, r_z) decomposition approximates each scalar element of the tensor as a triple sum:

$$(18) \quad \hat{\mathcal{Z}}_{i,j,m} \approx \sum_{p=1}^{r_\mu} \sum_{l=1}^{r_s} \sum_{q=1}^{r_z} \hat{\mathcal{C}}_{p,l,q} U_{i,p}^{(\mu)} U_{j,l}^{(s)} U_{m,q}^{(z)},$$

where $\hat{\mathcal{C}} \in \mathbb{C}^{r_\mu \times r_s \times r_z}$ is the global core tensor, and $U^{(\mu)}, U^{(s)}, U^{(z)}$ are the orthogonal factor matrices for the parameter (sample), frequency, and latent modes, respectively. To enable predictions on unseen test samples, we absorb the parameter-dependent terms into a simulation-specific core matrix $\hat{\mathcal{G}}_i \in \mathbb{C}^{r_s \times r_z}$, whose elements are $(\hat{\mathcal{G}}_i)_{l,q} = \sum_{p=1}^{r_\mu} \hat{\mathcal{C}}_{p,l,q} U_{i,p}^{(\mu)}$. This allows us to rewrite the approximation for the full frequency-domain latent matrix $\hat{\mathcal{Z}}_i \in \mathbb{C}^{K \times d_z}$ of the i -th simulation in a compact matrix form:

$$(19) \quad \hat{\mathcal{Z}}_i \approx U^{(s)} \hat{\mathcal{G}}_i U^{(z)\top}, \quad U^{(s)} \in \mathbb{C}^{K \times r_s}, \quad U^{(z)} \in \mathbb{C}^{d_z \times r_z}.$$

The shared orthogonal factors $U^{(s)}$ and $U^{(z)}$ are computed offline by the Higher-Order Orthogonal Iteration (HOOI) algorithm [3] and kept frozen. The surrogate then learns the mapping $\mu \mapsto \hat{\mathcal{G}}(\mu) \in \mathbb{C}^{r_s \times r_z}$, whose dimensionality $r_s \cdot r_z$ is substantially smaller than $K \cdot d_z$ because it jointly exploits cross-frequency and latent correlations.

For LLAE-Tucker, the surrogate predicts the complex-valued core matrix, from which the full Laplace-domain latents are reconstructed and inverted back to the time domain before decoding:

$$(20) \quad \mu \xrightarrow{g_\theta} \hat{\mathcal{G}}_{\text{pred}} \xrightarrow{U^{(s)}, U^{(z)\top}} \hat{\mathcal{Z}}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t)$$

For SLAE-Tucker, the surrogate predicts the real-valued core matrix, which is expanded to the real-valued frequency latents, decoded to spatial Laplace fields, and finally inverted to the physical time domain:

$$(21) \quad \mu \xrightarrow{g_\theta} \mathcal{G}_{\text{pred}} \xrightarrow{U^{(s)}, U^{(z)\top}} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t)$$

Unlike the real basis V of the SVD variant, the frequency factor $U^{(s)}$ mixes the Laplace frequencies and $U^{(z)}$ is complex-valued; neither commutes with $\mathcal{L}^{-1}$, so the back-projection necessarily precedes the inversion. Both variants share the same conceptual advantage: instead of predicting K independent vectors of size d_z , the surrogate predicts a highly compact core matrix that captures the essential spatiotemporal dynamics.

2.4 Designing the Regularized Inversion

The question of how to choose the integration contour $(s_1, \dots, s_K)$ arises for both SLAE and LLAE, but the two architectures admit fundamentally different strategies.

Before describing them, one point deserves stating once and plainly, because it governs how every result of Section 3.5 should be read. The object being designed here is not the contour alone but the **triplet** (s_k, α_t, λ) : the K poles together with the two regularization weights of the inversion (3). Both strategies below move all three — the learnable variant treats α_t and λ as parameters alongside the poles, the offline variant optimises them as coordinates of the same proxy — and the reference they are compared against fixes all three by convention: poles on the truncated Bromwich contour, and (α_t, λ) set a priori and never tuned. The improvements we report are therefore attributable to the triplet as a whole. We have not isolated the share due to displacing the poles from the share due to fitting the regularization, and an ablation freezing (α_t, λ) at their prescribed values while optimising the poles alone — cheap, since no network is involved — would settle it. Until then the honest reading is also the wider one: it is the **regularized inversion**, poles and regularization together, that we treat as a design object of first order, rather than the placement of the sampling points on its own.

2.4.1 Learnable Contour for LLAE

In the LLAE setting, the Laplace transform is applied in the low-dimensional latent space: the surrogate directly predicts latent Laplace coefficients, and the complex frequencies s_k can be made learnable parameters of the model, optimized end-to-end with the rest of the network. This is possible because both the forward quadrature (1) — a finite sum of complex exponentials $e^{-s_k t_n}$, smooth in s_k — and the Tikhonov inversion (4) — a linear solve whose coefficient matrix A depends smoothly on s_k , α_t , and λ — are fully differentiable with respect to the contour and regularization parameters.

This is precisely what is done in our LLAE implementation: both the real parts $\sigma_k = \text{Re}(s_k)$ and imaginary parts $\omega_k = \text{Im}(s_k)$ of each of the K poles are learned independently, along with the regularization hyperparameters α_t and λ (via their log-parameterisation). The poles are initialised on the truncated Bromwich contour (ω_k at the K first FFT frequencies, $\sigma_k = \gamma_{\text{init}}$ uniform).

2.4.2 Offline Contour Optimisation for SLAE

In contrast, making the SLAE contour learnable would require recomputing all N_t spatial forward passes per gradient step on the frequencies — negating its primary computational advantage over LLAE. Instead, we seek an **offline** procedure to select a good fixed contour before training SLAE.

We present a theoretical framework for this purpose. The goal is to decompose the total reconstruction error $\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})$ into a **bias** term (introduced by regularization and the finite contour) and a **variance** term (the surrogate error propagated through

the inversion), and to jointly optimize the contour and regularization parameters by minimising an upper bound on the expected reconstruction error.

Formal Setup. Using the notation of Section 2.1, we fix a spatial location and work with scalar (per-pixel) quantities. All Laplace-domain objects are those of (2): coefficient vectors carry the K_{full} entries of the conjugate-symmetric set $\mathcal{S}$, the K predicted ones being extended by conjugation exactly as the exact ones are, and $\mathbf{F} \in \mathbb{C}^{K_{\text{full}} \times N_t}$ is assembled on $\mathcal{S}$. Let $\mathbf{u}(\boldsymbol{\mu}) \in \mathbb{R}^{N_t}$ be the true temporal signal $(u(t_n))_n$ at that location, $\hat{\mathbf{u}}(\boldsymbol{\mu}) \in \mathbb{C}^{K_{\text{full}}}$ its exact Laplace coefficients $(\hat{u}(s_k))_k$, and $\tilde{\mathbf{u}}(\boldsymbol{\mu}) \in \mathbb{C}^{K_{\text{full}}}$ the surrogate-predicted coefficients (the per-pixel counterpart of the full-field $\tilde{U}(s_k)$ of Section 2.2.1). The regularised reconstructions from the true and predicted coefficients are, respectively,

$$(22) \quad \mathbf{u}^*(\boldsymbol{\mu}) = \mathbf{A}^{-1} \Re(\mathbf{F}^H \hat{\mathbf{u}}(\boldsymbol{\mu})), \quad \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) = \mathbf{A}^{-1} \Re(\mathbf{F}^H \tilde{\mathbf{u}}(\boldsymbol{\mu})),$$

where $\mathbf{A} = \Re(\mathbf{F}^H \mathbf{F}) + \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I}$ is the normal-equation matrix of (4) and $\mathbf{L}_{\text{reg}} = \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I}$. The unique existence of these solutions is guaranteed since $\lambda > 0$ and $\mathbf{D}_t^T \mathbf{D}_t$ is positive semi-definite, making $\mathbf{L}_{\text{reg}} \succeq \lambda \mathbf{I}$ and hence $\mathbf{A}$ symmetric positive definite (see Appendix G).

Bias-Amplification Decomposition. We decompose the total reconstruction error by introducing $\mathbf{u}^*(\boldsymbol{\mu})$ and adding and subtracting it:

$$(23) \quad \begin{aligned} \mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) &= (\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu})) + (\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})) \\ &= (\mathbf{u}(\boldsymbol{\mu}) - \mathbf{A}^{-1} \Re(\mathbf{F}^H \mathbf{F}) \mathbf{u}(\boldsymbol{\mu})) + \mathbf{A}^{-1} \Re(\mathbf{F}^H (\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu}))). \end{aligned}$$

Using $\mathbf{A} = \Re(\mathbf{F}^H \mathbf{F}) + \mathbf{L}_{\text{reg}}$, the bias term simplifies to:

$$(24) \quad \mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu}) = \mathbf{A}^{-1} (\mathbf{A} - \Re(\mathbf{F}^H \mathbf{F})) \mathbf{u}(\boldsymbol{\mu}) = \mathbf{A}^{-1} \mathbf{L}_{\text{reg}} \mathbf{u}(\boldsymbol{\mu}).$$

Taking norms in (23), applying the triangle inequality, and using $\|\Re(\mathbf{F}^H \cdot)\| \leq \|\mathbf{F}^H\| \|\cdot\|$ yields:

$$(25) \quad \|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\| \leq \underbrace{\|\mathbf{A}^{-1} \mathbf{L}_{\text{reg}} \mathbf{u}(\boldsymbol{\mu})\|}_{\text{regularization bias}} + \underbrace{\|\mathbf{A}^{-1} \mathbf{F}^H\|}_{\text{amplification factor}} \cdot \underbrace{\|\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu})\|_{\mathbb{C}^{K_{\text{full}}}}}_{\text{surrogate error}},$$

valid for any $\boldsymbol{\mu}$; the derivation is elementary and given in Appendix G. The bound is not deep — it is a triangle inequality followed by sub-multiplicativity — and we state it only because of the structure it exposes, namely that the two coefficients multiplying 1 and the coefficient error depend on (s_k, α_t, λ) and on nothing else. This is what makes the contour scorable without training a network, and it is the whole basis of the offline procedure below.

The regularization bias depends on the true field $\mathbf{u}(\boldsymbol{\mu})$ and $\mathbf{L}_{\text{reg}}$, and is mitigated by the well-posedness of $\mathbf{A}^{-1}$. Any surrogate prediction error $\hat{\mathbf{u}} - \tilde{\mathbf{u}}$ is magnified during reconstruction by the amplification factor $\|\mathbf{A}^{-1} \mathbf{F}^H\|$, which plays the role of a condition number for the inversion process. In (25) and in everything derived from it, $\|\cdot\|$ applied to a matrix is the operator 2-norm; the implementation evaluates instead the Frobenius norm $\|\mathbf{A}^{-1} \mathbf{F}^H\|_F \geq \|\mathbf{A}^{-1} \mathbf{F}^H\|_2$, which is cheaper to differentiate and preserves the inequality.

Optimization Problem Formulation. The matrices F and L_{reg} , and consequently A , depend explicitly on the contour $(s_1, \dots, s_K)$, on λ , and on α_t . We aim to jointly optimize these parameters to minimise the expected mean reconstruction error $\mathbb{E}_\mu[\|\mathbf{u}(\mu) - \mathbf{u}_{\text{rec}}(\mu)\|]$. Since evaluating this true error directly is challenging without ground truth for every candidate contour, we treat the hyperparameters as learnable parameters and optimize a proxy loss $\mathcal{L}^{\text{proxy}}$ derived directly from the bound (25):

$$(26) \quad \mathcal{L}^{\text{proxy}} = \mathcal{L}_{\text{bias}} + \mathcal{L}_{\text{var}}, \quad \mathcal{L}_{\text{bias}} = \mathbb{E}_\mu[\|\mathbf{u}(\mu) - \mathbf{u}^*(\mu)\|], \quad \mathcal{L}_{\text{var}} = \|A^{-1}F^H\|_F \cdot \epsilon_{\text{surr}},$$

where $\epsilon_{\text{surr}} = \mathbb{E}_\mu[\|\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu)\|_{\mathbb{C}^{K_{\text{full}}}}]$ is the expected surrogate error. By Jensen's inequality ($\mathbb{E}[\sqrt{X}] \leq \sqrt{\mathbb{E}[X]}$), this expected norm is bounded by:

$$(27) \quad \begin{aligned} \epsilon_{\text{surr}} &\leq \sqrt{\mathbb{E}_\mu[\|\hat{\mathbf{u}}(\mu) - \tilde{\mathbf{u}}(\mu)\|_{\mathbb{C}^{K_{\text{full}}}}^2]} \\ &= \sqrt{\sum_{s_k \in S} \mathbb{E}_\mu[|\hat{u}_k(\mu) - \tilde{u}_k(\mu)|^2]}, \end{aligned}$$

Approximating the Surrogate Error via SVD Residual. Training a full neural surrogate for every proposed contour is computationally intractable. We therefore replace the expected surrogate error by the residual of the best **linear** reduction of the same rank, which is available in closed form from a singular value decomposition. Here $\hat{U}(s_k; \mu) \in \mathbb{C}^{N_x}$ denotes the **full spatial** Laplace field at frequency s_k (the vector of per-pixel values $\hat{u}_k(\mu)$ across all N_x locations), as opposed to the scalar $\hat{u}_k(\mu)$ at a single location used above. Each instance is represented by the real vector obtained by stacking its K spatial Laplace fields and splitting them into real and imaginary parts,

$$(28) \quad \mathbf{a}(\mu) = [\Re \hat{U}(s_1; \mu); \Im \hat{U}(s_1; \mu); \dots; \Re \hat{U}(s_K; \mu); \Im \hat{U}(s_K; \mu)] \in \mathbb{R}^{2N_x K},$$

and the design is summarised by the matrix $U = [\mathbf{a}(\mu_1), \dots, \mathbf{a}(\mu_{N_{\text{opt}}})] \in \mathbb{R}^{2N_x K \times N_{\text{opt}}}$ assembled over a sample of N_{opt} training instances. The expected squared error of the optimal linear rank- d_z reduction is the mean squared column-wise error, whose minimum over all rank- d_z approximations is attained by the truncated SVD and equals the sum of the discarded squared singular values:

$$(29) \quad \mathcal{E}_{\text{SVD}} = \sqrt{\min_{\text{rank } \hat{U} \leq d_z} \frac{1}{N_{\text{opt}}} \|U - \hat{U}\|_F^2} = \sqrt{\frac{1}{N_{\text{opt}}} \sum_{j > d_z} \sigma_j^2},$$

where σ_j are the singular values of U . Two features of (29) are deliberate and should be read literally. First, the basis is **single and shared across the K frequencies**: the residual scores a rank- d_z reduction of the entire space–frequency field of a simulation, not K independent rank- d_z reductions, one per frequency. This is a more constrained — hence more pessimistic — reduction than the one SLAE actually performs, since SLAE encodes each frequency separately and disposes of $K d_z$ coefficients in total; we retain the joint form because it depends on the contour through a single spectrum and is what our implementation computes. Second, U is never formed: the $N_{\text{opt}} \times N_{\text{opt}}$ Gram matrix $U^\top U$ is accumulated over spatial chunks and (29) read off its eigenvalues, at a cost independent of N_x in memory; the truncation rank is therefore capped at $\min(d_z, N_{\text{opt}})$, and we use $N_{\text{opt}} = 100$ instances against $d_z = 64$.

Since a non-linear AE with bottleneck d_z generalizes the linear reduction, the **optimal** AE error — the infimum over the hypothesis class, under exact global minimisation —

satisfies $\epsilon_{\text{AE}}^* \leq \mathcal{E}_{\text{SVD}}$ for the same class of reductions (see Appendix G). This bounds an idealisation, not a trained network: the error actually attained by gradient descent under a particular loss and a finite budget can exceed $\mathcal{E}_{\text{SVD}}$, and for SLAE it does in four of the seven configurations of Table 4. We therefore use $\mathcal{E}_{\text{SVD}}$ strictly as a proxy, relying on the **relative** behavior of ϵ_{Surr} with respect to the contour rather than on its absolute value — which is also why the discrepancy in reduction class noted above is tolerable. The SVD residual is fast and deterministic to compute for any candidate contour, making it highly suitable for iterative optimisation. To account for the enhanced expressivity of the non-linear AE over the linear baseline, we introduce a scaling hyperparameter $\lambda_{\text{AE}} \geq 1$.

A Smoothness Heuristic on the Bias Term. We add one purely **heuristic** ingredient, which does not follow from the bound (25). At equal bias norm, a smooth offset is far less damaging than Gibbs-like oscillations — the characteristic artefact of spectral truncation — yet the L^2 norm does not distinguish them. We therefore also penalise the finite differences of the bias error. Writing $\mathbf{e}(\boldsymbol{\mu}) = \mathbf{U}(\boldsymbol{\mu}) - \mathbf{U}^*(\boldsymbol{\mu}) \in \mathbb{R}^{N_x \times N_t}$ and $\mathbf{D}_t, \mathbf{D}_x, \mathbf{D}_y$ for the first-order difference operators along the temporal axis and along the two axes of the spatial grid,

$$(30) \quad \mathcal{L}_{\text{bias}} = \frac{1}{1 + \lambda_{\Delta t} + \lambda_{\Delta x}} \mathbb{E}_{\boldsymbol{\mu}} \left[\|\mathbf{e}(\boldsymbol{\mu})\|_F + \lambda_{\Delta t} \|\mathbf{D}_t \mathbf{e}(\boldsymbol{\mu})\|_F + \lambda_{\Delta x} (\|\mathbf{D}_x \mathbf{e}(\boldsymbol{\mu})\|_F + \|\mathbf{D}_y \mathbf{e}(\boldsymbol{\mu})\|_F) \right],$$

the normalisation keeping $\mathcal{L}_{\text{bias}}$ on the scale of the variance term irrespective of the weights. Setting $\lambda_{\Delta t} = \lambda_{\Delta x} = 0$ recovers the plain bias of (25); with positive weights the objective is no longer an upper bound on the reconstruction error, the guarantee being traded for a preference over the **shape** of the residual. The final proxy objective is therefore:

$$(31) \quad \begin{aligned} \mathcal{L}^{\text{proxy}} = & \frac{1}{N_{\text{opt}} (1 + \lambda_{\Delta t} + \lambda_{\Delta x})} \sum_{i=1}^{N_{\text{opt}}} \left(\|\mathbf{e}(\boldsymbol{\mu}_i)\|_F + \lambda_{\Delta t} \|\mathbf{D}_t \mathbf{e}(\boldsymbol{\mu}_i)\|_F \right. \\ & \left. + \lambda_{\Delta x} (\|\mathbf{D}_x \mathbf{e}(\boldsymbol{\mu}_i)\|_F + \|\mathbf{D}_y \mathbf{e}(\boldsymbol{\mu}_i)\|_F) \right) \\ & + \frac{\|\mathbf{A}^{-1} \mathbf{F}^H\|_F}{\lambda_{\text{AE}}} \cdot \sqrt{\frac{1}{N_{\text{opt}}} \sum_{j>d_z} \sigma_j^2}. \end{aligned}$$

Minimizing (31) by gradient descent jointly optimizes the contour frequencies and the regularization parameters, trading bias against variance in a principled and fully differentiable manner. Both terms are evaluated on the same sample of N_{opt} training instances, drawn at a single rotation angle so that the sample spans the (k, C) plane rather than 36 views of a few simulations; the whole procedure runs in double precision and touches no network. The remaining settings — optimiser, step count, the values of $\lambda_{\Delta t}$, $\lambda_{\Delta x}$, λ_{AE} , and the box constraints $\sigma_k \geq -0.05$, $\omega_k \geq 0$, $\alpha_t, \lambda \in [10^{-6}, 1]$ under which (31) is minimised — are reported in Section 3.5, where the optimised contours are used.

2.5 Baselines

The approach described so far combines two ingredients: a non-linear autoencoder for spatial compression, and a Laplace transform for temporal compression. We therefore compare against two references, each removing exactly one of them. The first is **linear** in the spatial compression step but keeps the Laplace domain: a per-frequency POD basis

with a non-intrusive surrogate on the projection coefficients (Section 2.5.1). The second is **non-linear** but Laplace-free, and keeps the full temporal grid: a standard DL-ROM (Section 2.5.2). Read together, they attribute the observed performance to one ingredient or the other rather than to their combination. Both reuse the surrogate of Section 2.3 and the two-phase training protocol unchanged.

2.5.1 Linear Baseline: POD in the Laplace Domain

As a linear reference, we construct a Proper Orthogonal Decomposition (POD) basis for each Laplace frequency independently. For each s_k , the training-set spatial fields are stacked into a data matrix $\mathbf{U}^{(k)} \in \mathbb{R}^{N_x \times N_{\text{train}}}$ (one per physical component), and a truncated basis of rank k_{pod} is extracted. Because $N_x = N^2 = 16,384$ and $N_{\text{train}} = 5,184$, a direct SVD of $\mathbf{U}^{(k)}$ would be memory-intensive; we instead use a two-pass randomized SVD [13] that reduces the cost to two passes over the data and a small k_{pod} -dimensional eigenvalue problem. The real and imaginary parts of $\hat{U}(s_k)$ are treated as separate real-valued fields, each with its own basis $\mathbf{V}^{(k,r)}, \mathbf{V}^{(k,i)} \in \mathbb{R}^{N_x \times k_{\text{pod}}}$ fitted on the training set:

$$(32) \quad \Re(\hat{U}(s_k)) \approx \mathbf{V}^{(k,r)} \mathbf{c}^{(k,r)}, \quad \Im(\hat{U}(s_k)) \approx \mathbf{V}^{(k,i)} \mathbf{c}^{(k,i)},$$

where $\mathbf{c}^{(k,r)}, \mathbf{c}^{(k,i)} \in \mathbb{R}^{k_{\text{pod}}}$ are the projection coefficients. This is the classical Galerkin-POD representation applied frequency-by-frequency in the Laplace domain [14].

Surrogate for the POD coefficients. Given the compact coefficient vectors, the surrogate task reduces to predicting $2K k_{\text{pod}}$ scalar targets from $\boldsymbol{\mu}$. This is the classical non-intrusive POD-NN strategy [15], here applied frequency-by-frequency in the Laplace domain. The surrogate MLP g_θ of Section 2.3 predicts the normalized coefficients for all frequencies jointly:

$$(33) \quad \mathbf{c}^{(k,r)}, \mathbf{c}^{(k,i)} = g_\theta(\boldsymbol{\mu}, k), \quad k = 0, \dots, K-1.$$

Training minimizes a combined latent-coefficient MSE and physical reconstruction loss (after Tikhonov inversion):

$$(34) \quad \begin{aligned} \mathcal{L}_{\text{POD-surr}} = & \alpha_{\text{lat}} \sum_{k=0}^{K-1} \left(\left\| g_\theta^{(k,r)}(\boldsymbol{\mu}) - \bar{\mathbf{c}}^{(k,r)} \right\|_2^2 + \left\| g_\theta^{(k,i)}(\boldsymbol{\mu}) - \bar{\mathbf{c}}^{(k,i)} \right\|_2^2 \right) \\ & + \alpha_{\text{spat}} \left\| U_{\text{rec}} - U \right\|_2^2, \end{aligned}$$

where $\bar{\mathbf{c}}^{(k,\cdot)}$ denotes the normalized coefficients and U_{rec} is the field reconstructed by back-projecting the predicted coefficients through the fixed bases and applying the inverse Laplace transform. The spatial bases $\mathbf{V}^{(k,\cdot)}$ remain **fixed** throughout surrogate training.

The time series $U(t)$ is reconstructed by back-projecting the predicted coefficients, assembling the complex frequency field, and applying the Tikhonov inversion (3). This pipeline is fully linear in the spatial compression step, making it a clean reference that quantifies the gain of non-linear autoencoders. The irreducible truncation error of the linear basis — controlled by the discarded singular values of $\mathbf{U}^{(k)}$ — sets a hard floor that non-linear AEs are designed to overcome.

2.5.2 Nonlinear Baseline: DL-ROM without Laplace

The POD baseline removes the non-linearity; our second reference removes the Laplace transform instead. It is a standard DL-ROM [6]: the convolutional encoder maps each physical frame to a latent code and the decoder reconstructs it, with no spectral compression anywhere in the pipeline:

$$(35) \quad U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t)$$

This is exactly the LLAE pipeline of Section 2.2.2 with the latent round trip $z(t) \rightarrow \hat{z}(s_k) \rightarrow \tilde{z}(t)$ removed: the same convolutional backbone, the same FiLM conditioning on the normalised time t/T , the same L_2 ridge on the latent codes. The loss retains the first and third terms of (12),

$$(36) \quad \mathcal{L}_{\text{DL-ROM}} = \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|U_{\text{rec}}(t_n) - U(t_n)\|_2^2 + \beta \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|z(t_n)\|_2^2,$$

the ridge now applying to $z(t)$ directly rather than to $\hat{z}(s_k)$. The latent consistency term has no counterpart here: with no round trip, there is no invertibility to enforce. The comparison is therefore a controlled ablation of the Laplace block rather than a comparison between two independently tuned models. We deliberately use the DL-ROM of [6] rather than the POD-DL-ROM variant [7], whose POD pre-reduction is an efficiency device that leaves the learned representation unchanged; Section 4.5 returns to this point.

Surrogate cost. The surrogate of Section 2.3 carries over unchanged except in **what** it must predict. Where the Laplace pipelines regress the K Laplace-domain latents w_k , the DL-ROM surrogate must regress the latent trajectory at every time step, $\mu \mapsto z(t_n)$ for $n = 0, \dots, N_t - 1$: the per-frequency heads become per-time-step heads, conditioned on $t_n/(N_t - 1)$ in place of f_k , and their number grows from K to N_t . With a shared trunk of N_{trunk} parameters and heads of N_{head} parameters each — N_{head} depending only on d_z and the head width, not on their number —

$$(37) \quad N_{\text{surr}} = N_{\text{trunk}} + K N_{\text{head}}, \quad K = N_t \text{ (DL-ROM)}, \quad K \ll N_t \text{ (SLAE, LLAE)},$$

so the DL-ROM surrogate grows linearly with the temporal resolution, while the Laplace surrogates are independent of it. With the architecture of Appendix F and $d_z = 64$, this amounts to 23.5M parameters at $N_t = 150$ against 3.5M for LLAE at $K = 16$ — a factor of 6.8 — the two becoming equal in size at $N_t \approx 18$.

Temporal-resolution sweep. Since N_t is the sole driver of this cost, we sweep it by uniform subsampling of the time grid with a stride κ , so that $N_t \mapsto \lceil N_t/\kappa \rceil$ and $\Delta t \mapsto \kappa \Delta t$ while the horizon T is held fixed. This traces the DL-ROM cost/accuracy curve while the Laplace pipelines remain at $K = 16$, and it supplies a parameter-matched operating point: at $\kappa = 10$ the DL-ROM surrogate has essentially the size of LLAE at $K = 16$, both compressing time by roughly a factor of ten — uniformly on one side, spectrally on the other. The sweep concerns the DL-ROM alone; the Laplace pipelines are always trained on the full $N_t = 150$ grid, since their temporal compression is already carried by K .

2.6 Architecture Summary

Table 1 provides a synthetic overview of all nine pipelines studied. The two top rows describe the **autoencoder reconstruction** (Phase 1, input $U(t)$); the seven remaining rows compare the **surrogate inference** pipelines (Phase 2, input μ), grouped by the domain in which the Laplace transform is applied.

3 Results

3.1 Problem Setting

We consider the emulation of the methane mass fraction Y in a two-dimensional parametric transient reacting flow. The transport equation is formulated as a convection-diffusion-reaction problem over a spatial domain $\Omega \subset \mathbb{R}^2$ and a time interval $t \in [0, T]$:

$$(38) \quad \frac{\partial(\rho Y)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y) - \nabla \cdot (\mu_{\text{eff}} \nabla Y) = \dot{\omega}_{\text{eff}}(Y; \mu)$$

where ρ is the fluid density, $\mathbf{U}$ is the velocity field, μ_{eff} is the effective diffusivity, and $\dot{\omega}_{\text{eff}}$ is the effective reaction rate. A detailed formulation of the complete system of governing equations is provided in Appendix E. The system is governed by a vector of three physical parameters $\mu = [k, \phi, C]^\top \in \mathbb{R}^3$, specifically representing:

- k : Diffusion parameter,
- ϕ : Injection angle,
- C : Injection rate ($\text{kg}/\text{m}^3/\text{s}$).

The temporal domain is discretized into $N_t = 150$ time steps with uniform step size $\Delta t = 1$ s (total duration $T = 150$ s). The quantity of interest in the methodology notation is $U(t) \in \mathbb{R}^{N_x}$, the discretized spatial field of Y at time step t ; the full space–time field is a matrix in $\mathbb{R}^{N_x \times N_t}$. The number of Laplace frequency samples K is a model hyperparameter ($K = 16$ for both SLAE and LLAE; see Table 14).

Dataset construction. The parameter sweep is carried out over (k, C) only: 225 high-fidelity simulations are computed on the native 200×200 solver grid. The third parameter, the injection angle ϕ , is then realised geometrically. The governing operator is isotropic and the injection is centred, so a rotation by ϕ about the domain centre maps a solution onto the solution of the problem with the injection rotated by ϕ , and each simulation is replicated at 36 uniformly spaced angles $\phi \in \{0^\circ, 10^\circ, \dots, 350^\circ\}$. The rotation is applied at native resolution and the result centre-cropped to 128×128 ; since the corners of the retained window lie at a radius of 90.5 cells, inside the disc of radius 100 inscribed in the solver domain, the crop retains only the region untouched by the rotation fill and no interpolation is involved in the downscaling. This yields $N_{\text{sim}} = 225 \times 36 = 8,100$ parameter instances on a uniform grid of $N_x = 128 \times 128 = 16,384$ nodes.

Partition. A fixed random permutation (seed 42) reserves 20% of the instances, 1,620, as a held-out test set; the remaining 6,480 are split 80/20 into 5,184 training and 1,296 validation instances. Every quantity reported in this paper is measured on the 1,620 test instances. The validation set alone drives early stopping and checkpoint selection (Appendix F); the test set is never used for any training or selection decision, and the POD bases of Section 2.5.1 are fitted on the training partition only.

Model	Pipeline	$\mathcal{L}$ applied in
<i>Autoencoder reconstruction (Phase 1, input: $U(t)$)</i>		
SLAE		phys. domain
LLAE		latent space
<i>Surrogate inference (Phase 2, input: μ)</i>		
POD		phys. domain
SLAE-Direct		phys. domain
SLAE-SVD		phys. domain
SLAE-Tucker		phys. domain
LLAE-Direct		latent space
LLAE-SVD		latent space
LLAE-Tucker		latent space

Table 1: Architecture summary of the nine pipelines studied. **Top block** (Phase 1): autoencoder reconstruction, input $U(t)$. **Bottom block** (Phase 2): surrogate inference pipelines mapping $\mu \rightarrow U(t)$, grouped by the domain in which $\mathcal{L}$ is applied physical domain (SLAE family) or latent space (LLAE family). Colour code: ■ MLP surrogate; ■ Laplace $\mathcal{L}$ / inverse $\mathcal{L}^{-1}$; ■ SVD back-projection; ■ Tucker back-projection; ■ convolutional encoder E (wide→narrow) / decoder D (narrow→wide); • latent-space representation on the arrow.

This replication is a solver shortcut and not data augmentation: each of the 8,100 instances is an exact solution of the governing system at its own point (k, ϕ, C) , which the solver would have returned had it been run there, and ϕ enters the design of experiments on the same footing as k and C .

The performance of the models is evaluated quantitatively using the relative L^2 error over the held-out test set, defined as:

$$(39) \quad \epsilon_{\text{rel}} = \frac{\|Y_{\text{true}} - Y_{\text{pred}}\|_2}{\|Y_{\text{true}}\|_2}$$

We first characterize the autoencoder architectures in isolation, then evaluate the end-to-end surrogate models.

3.2 Baseline Results

The two references of Section 2.5 remove one ingredient each — POD in the Laplace domain drops the non-linearity, the DL-ROM drops the Laplace transform — and both are characterised here in the same two stages as the Laplace pipelines themselves. The first stage isolates the **representation**: the reconstruction error obtained when the true field is passed through the compression and back, with no surrogate involved, which sets the floor of the pipeline. The second stage is the deployed **surrogate**, predicting from μ alone. Reporting both separates what the representation cannot represent from what the regression fails to predict, and the difference between them — the Gap — is the quantity that decides whether a better floor is worth having.

3.2.1 Linear Baseline: POD in the Laplace Domain

The three hyperparameters of the linear baseline (Section 2.5.1) are swept independently around a base configuration ($K = 16$, $k_{\text{pod}} = 64$, $\gamma = 0.01$): $K \in \{8, 16, 32\}$, $k_{\text{pod}} \in \{32, 64, 128\}$ and $\gamma \in \{0, 0.001, 0.01\}$. Each configuration is evaluated at both stages on the 1620 test simulations — as a pure **reconstruction**, in which the true fields are projected onto the training bases with no surrogate involved, and **end-to-end**, in which a network predicts the coefficients from μ . Table 2 reports both.

k_{pod}	K	γ	Rec med	Rec p90	Surr med	Surr p90	Gap (pp)	Lat (ms)
64	32	0.01	7.57	15.08	8.75	17.44	+1.17	5.6 ± 0.0
64	16	0.01	8.09	14.67	8.91	16.34	+0.82	3.5 ± 0.2
64	16	0.001	8.38	14.29	8.94	15.36	+0.56	3.5 ± 0.1
64	16	0.0	8.42	14.22	8.96	15.54	+0.54	3.5 ± 0.1
128	16	0.01	7.04	9.79	11.29	21.74	+4.26	3.7 ± 0.1
32	16	0.01	11.42	22.99	11.69	23.44	+0.27	3.4 ± 0.2
64	8	0.01	12.01	17.39	12.41	18.40	+0.41	2.9 ± 0.2

Table 2: POD baseline on the test set (1620 simulations), sorted by surrogate median error. All errors are relative L^2 errors in %. **Rec**: pure POD reconstruction error (linear floor, no surrogate). **Surr**: end-to-end surrogate error. **Gap**: difference surrogate – reconstruction (positive = surrogate adds error on top of the linear floor). **Lat**: GPU inference latency at batch size 1 (50 timed runs).

Linear reconstruction floor. The floor is governed by the singular-value decay of the Laplace-domain field matrices $U^{(k)}$: the linear basis is accurate at low frequencies, where the spatial structure is smooth and low-rank, and degrades at higher ones, where the fields become localized. Increasing k_{pod} beyond 64 yields only a marginal improvement, indicating that the residual is an intrinsic limit of the linear subspace rather than a truncation artefact. The damping is nearly neutral on the physical field (8.42, 8.38 and 8.09% for $\gamma = 0, 0.001$ and 0.01) because two error sources cancel: a purely imaginary contour concentrates the energy and lowers the POD truncation error at every frequency, whereas a larger γ attenuates the inversion error that propagates those residuals back to the time domain. Appendix B reports the complete sweeps and the per-frequency breakdown that exposes this trade-off.

Surrogate overhead. The regression stage adds little above the floor, between +0.27 and +1.17 pp, with one exception: at $k_{\text{pod}} = 128$, $K = 16$, the best floor of the sweep (7.04%) yields its worst surrogate (11.29%, +4.26 pp). With $2Kk_{\text{pod}} = 4,096$ scalar targets, the higher-dimensional regression problem overwhelms the benefit of the larger basis — a trade-off between representation quality and learnability that recurs at every stage of this study. The best POD surrogate is therefore $k_{\text{pod}} = 64$, $K = 16$, $\gamma = 0.01$, at 8.91% median error and 2.9–5.6 ms per sample at batch size 1, its cost dominated by the back-projection through $V^{(k)}$ and the Tikhonov solve rather than by the network forward pass. This is the floor that the non-linear autoencoders of Section 3.3 are designed to push below; per-sample distributions are provided in Appendix A.

3.2.2 Nonlinear Baseline: DL-ROM without Laplace

The second reference removes the Laplace transform instead of the non-linearity (Section 2.5.2). It is trained on the full temporal grid ($N_t = 150$, $\Delta t = 1$ s) with the same convolutional backbone, the same latent dimension $d_z = 64$, the same two-phase protocol and the same data partition as the Laplace pipelines, the only structural difference being that its surrogate carries one head per time step instead of one per Laplace frequency. Table 3 reports the two stages.

Stage	Median (%)	Mean (%)	p90 (%)	Gap (pp)
Autoencoder (frame-wise reconstruction)	2.49	2.71	3.87	—
End-to-end surrogate	3.80	4.15	6.12	+1.31

Table 3: DL-ROM baseline at full temporal resolution ($N_t = 150$, $d_z = 64$), relative L^2 error on the physical field over the test set ($n = 1620$). **Autoencoder**: each of the 150 frames encoded and decoded, no surrogate and no Laplace round-trip. **Surrogate**: prediction from μ alone through the 150 per-time-step heads. **Gap**: surrogate – autoencoder.

Autoencoder stage. The frame-wise autoencoder reconstructs the field to 2.49% median error (2.71% mean, 3.87% p90). This is the lowest reconstruction error of the entire study, and necessarily so: sharing the backbone, the latent dimension and the training protocol with LLAE, it differs from it by the absence of the latent Laplace round-trip alone. The comparison with LLAE at the matched configuration ($d_z = 64$, $K = 16$, $\gamma = 0.01$, 3.20% in Table 4) therefore prices that round-trip exactly: 0.71 pp, the cost of representing each latent trajectory by 16 complex coefficients and recovering it by

a truncated inversion. It is the cleanest measurement of the truncation penalty we have, since nothing else differs between the two.

Surrogate stage. The end-to-end error distribution over the 1620 held-out simulations (Figure 23, Appendix A) is unimodal and tightly concentrated: median 3.80%, mean 4.15%, p90 6.12%, with a thin upper tail reaching 10%. The regression stage thus adds 1.31 pp to the reconstruction floor — more than the +0.82 pp of the best POD surrogate, which is expected of a target of 150 latent frames against $2 \times 16 \times 64$ POD coefficients, and the price at which the DL-ROM’s superior floor is delivered. Removing the Laplace transform therefore costs nothing in accuracy — on the contrary, at full temporal resolution the DL-ROM is the most accurate emulator of this study, 57% below the best POD surrogate (8.91%, Section 3.2.1) and marginally below the best Laplace surrogate reported in Section 3.4.1. This is the expected outcome of a controlled ablation: the DL-ROM predicts the latent trajectory at every one of the 150 time steps and discards no spectral content, so it faces a strictly easier approximation problem than a surrogate constrained to $K = 16$ coefficients. Its accuracy comes at a price on the cost side — 23.5M surrogate parameters against 3.5M for LLAE at $K = 16$, per equation (37) — and it delivers the trajectory only on the grid it was trained on. The DL-ROM therefore sets a demanding accuracy reference rather than a straw man, and the comparison that matters is one of accuracy **against** cost at a given output resolution; the temporal-resolution sweep of Section 2.5.2 and the full cost comparison are deferred to Sections 4.1–4.2.

3.3 Autoencoder Performance

Table 4 reports the relative L^2 reconstruction error on the held-out test set (1620 simulations) for all trained AE checkpoints, evaluated on the physical field $U(t)$.

Table 4 summarises the reconstruction floor across all 14 checkpoints. The two families present a stark contrast. LLAE achieves a best median error of 2.92% ($d_z = 128$, $K = 16$), a factor of 2.4 below the POD reconstruction at matching configuration (7.04% at $k_{\text{pod}} = 128$, $K = 16$) and of 2.5–3.5 against the other POD configurations. SLAE yields errors in the range 7.78%–12.26%, broadly comparable to POD: the **vs. POD** column of Table 4 hovers around zero, positive as often as negative. Taken as a pure compression stage, SLAE does not offer a meaningful gain over the linear reference. This may appear disappointing, but the picture changes substantially at the surrogate stage (Section 3.4).

Two compounding factors explain this divergence (Sections 2.2.1–2.2.2). First, the input spaces differ: SLAE compresses high-dimensional complex frequency frames $\hat{U}(s_k)$, whereas LLAE encodes smoother physical frames — an intrinsically easier task. Second, the training objectives are asymmetric: LLAE’s loss (12) includes a direct physical-time reconstruction term that provides dense temporal supervision, while SLAE deliberately omits it during Phase 1 (Section 2.2.1) to preserve its computational advantage.

Figures 3 and 4 report ablation studies for each family, varying one hyperparameter at a time while holding the others fixed.

Within LLAE (Figure 3), increasing the latent dimension from $d_z = 32$ to $d_z = 128$ reduces the median error from 3.43% to 2.92%, indicating that the latent bottleneck has not yet saturated at $d_z = 64$. The number of Laplace frequencies K has little impact: errors at $K = 8, 16$, and 32 are 3.35%, 3.20%, and 3.19% respectively, suggesting that 8–16 frequencies already capture the dominant transient dynamics. The damping

Model	d_z	K	γ	Median (%)	Mean (%)	p90 (%)	vs. POD (pp)
LLAE	128	16	0.01	2.92	3.15	4.38	-4.12
	64	32	0.01	3.19	3.53	5.11	-4.38
	64	16	0.01	3.20	3.47	5.04	-4.89
	64	16	0.0	3.26	3.61	5.24	-5.16
	64	16	0.001	3.32	3.65	5.16	-5.06
	64	8	0.01	3.35	3.66	5.26	-8.66
	32	16	0.01	3.43	3.85	5.60	-7.99
SLAE	64	16	0.0	7.78	8.11	9.91	-0.64
	64	16	0.001	7.82	8.27	10.36	-0.56
	128	16	0.01	7.98	8.40	10.45	+0.94
	64	32	0.01	8.34	9.27	13.43	+0.77
	64	16	0.01	8.92	9.53	12.55	+0.83
	32	16	0.01	10.76	11.14	14.76	-0.66
	64	8	0.01	12.26	12.74	15.40	+0.25

Table 4: Relative L^2 reconstruction error on the test set (1620 simulations). The last column gives the difference in median error relative to the linear POD baseline at matching $(K, k_{\text{pod}} = d_z, \gamma)$: negative values indicate improvement over POD, positive values indicate that POD outperforms the AE. Best per family in bold.

parameter γ is similarly insensitive in the range $[0, 0.01]$, consistent with the smooth energy envelope of the exponential decay at these timescales.

Within SLAE (Figure 4), errors increase sharply at low K ($K = 8$: 12.26%) and low d_z ($d_z = 32$: 10.76%), pointing to a harder compression problem where both the spectral and spatial bottlenecks are binding simultaneously. Increasing K to 32 does not improve reconstruction (8.34% vs. 7.78% at $K = 16$), likely because higher Laplace frequencies carry less spatial energy and expose the spatial compression bottleneck. The SLAE error distribution also shows a heavier right tail (higher p90) than LLAE, reflecting occasional failure cases where spatial compression degrades on outlier samples.

For SLAE, Figure 5 shows the reconstruction error broken down per Laplace frequency k . The error is lowest at low frequencies ($k = 0-3$, where most signal energy is concentrated) and rises monotonically toward higher frequencies, confirming that the spatial compression bottleneck is more severe for the fine-scale, high-frequency content.

Comparing with the POD per-frequency breakdown (Figure 27), SLAE achieves roughly half the per-frequency error of POD at each k : the convolutional encoder captures non-linear structure that the linear projection cannot. Yet this per-frequency advantage does not translate into a better overall spatial reconstruction (Table 4). The reason is Tikhonov regularisation: the inverse transform (3)–(4) damps all frequencies uniformly, so the benefit of encoding each frequency more accurately is partially cancelled before the spatial field is recovered. The global spatial error is therefore a **regularisation-weighted** aggregate of the per-frequency errors, not a direct reflection of per-frequency compression quality.

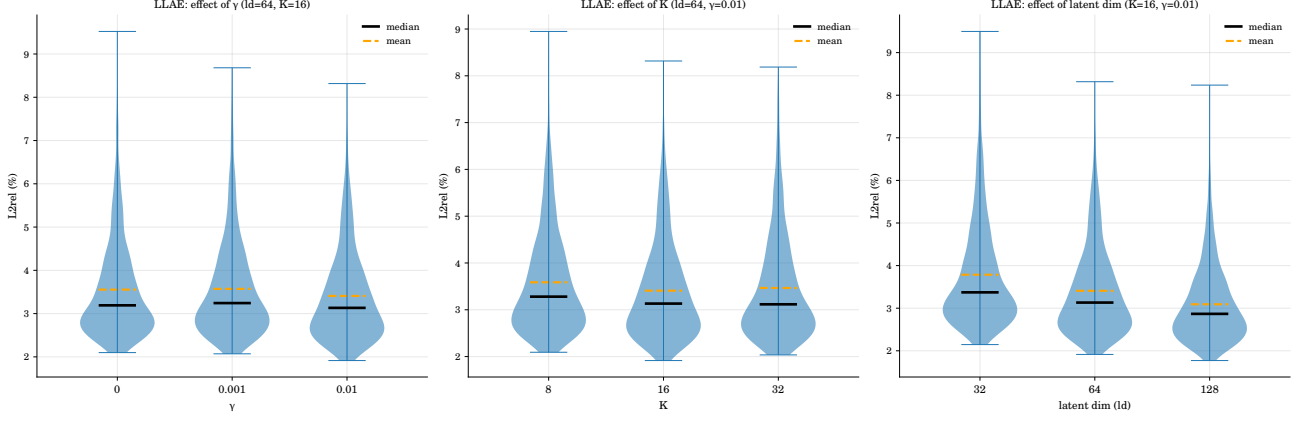


Figure 3: LLAE ablation study. From left to right: effect of latent dimension d_z ($K = 16$, $\gamma = 0.01$), number of Laplace frequencies K ($d_z = 64$, $\gamma = 0.01$), and damping parameter γ ($d_z = 64$, $K = 16$). Violin plots show the full error distribution; black line is the median, dashed orange line is the mean.

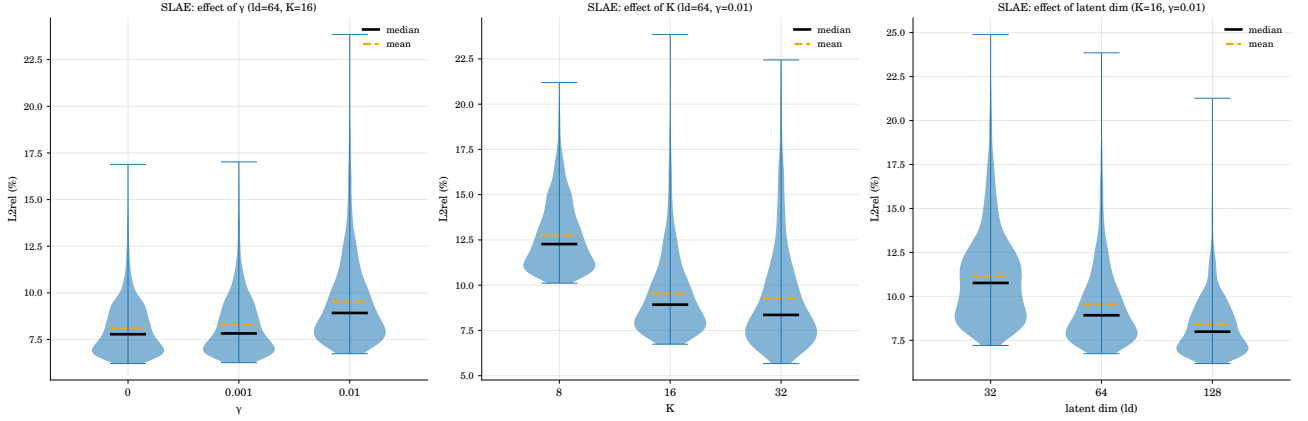


Figure 4: SLAE ablation study. Same layout as Figure 3 (fixed parameters: $K = 16$, $\gamma = 0.01$ / $d_z = 64$, $\gamma = 0.01$ / $d_z = 64$, $K = 16$). Note the larger y-axis range compared to LLAE.

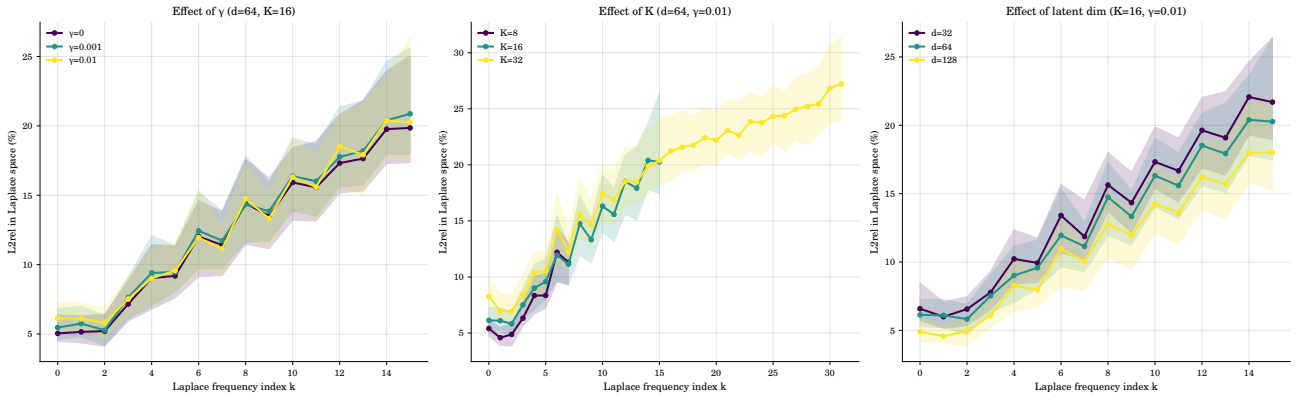


Figure 5: SLAE — per-frequency reconstruction error in the Laplace domain (median $\pm$ IQR, 200 test samples). Left: effect of γ ; centre: effect of K ; right: effect of d_z .

Inference cost. Table 5 and Figure 6 report the per-sample cost of a full reconstruction along three axes: arithmetic work (FLOPs, counted on the executed graph), wall-clock latency (batch size 1, 50 timed CUDA Event runs after 10 warm-up runs), and peak GPU memory. We lead with FLOPs because they measure the cost of the **architecture**, independently of how well a given implementation exploits the hardware; latency is reported alongside because it is what a user experiences, and the two do not agree here.

The FLOPs follow the architectural prediction of Sections 2.2.1–2.2.2 exactly. SLAE decodes K frequency frames, so its arithmetic scales linearly with K : 15.5, 31.1 and 62.2 GFLOPs at $K = 8, 16$ and 32 . LLAE decodes all $N_t = 150$ time frames regardless, so its cost is flat at 54.2 GFLOPs across a fourfold variation of K , and varies by less than 3% between $d_z = 32$ and $d_z = 128$. At $K = 16$ SLAE therefore performs $1.74\times$ fewer operations than LLAE, and $3.5\times$ fewer at $K = 8$; the ordering reverses only above $K \approx 28$, where decoding the frequency frames becomes more expensive than decoding the time frames.

Latency follows the same ordering, though with a smaller spread: 22.7 ± 1.1 ms for SLAE at $K = 16$ against 27.0 ± 1.7 ms for LLAE, a factor of 1.2, rising to 1.5 at $K = 8$ (18.4 ms). That the wall-clock gain ($1.2\times$) falls short of the arithmetic gain ($1.74\times$) is expected: reconstruction from a physical field also requires the forward Laplace transform (1) and its regularised inverse, a fixed cost the arithmetic ratio does not see and which the surrogate does not pay at all (Section 3.4.1). SLAE’s latency is also markedly more reproducible, its run-to-run spread being 1.0–1.4 ms against 1.7–2.1 for LLAE, the frequency frames being decoded as a single batch where the time frames are not.

Peak memory is the one axis on which the two families are comparable at this stage (518 against 556 MB at $K = 16$), and it is also the one that separates them most clearly by regime: SLAE’s allocation is set by K , growing from 404 to 794 MB between $K = 8$ and $K = 32$, whereas LLAE’s is set by N_t and stays within 3% of 556 MB throughout. The reconstruction stage holds the entire physical field in memory to transform it, which is what brings SLAE up to LLAE’s level here; the surrogate, which never sees a physical time frame, retains a $2.6\times$ advantage (Section 3.4.1).

3.4 Surrogate Prediction Performance

3.4.1 Direct surrogate performance

Table 6 reports the relative L^2 error on the held-out test set (1620 simulations) for all 14 end-to-end surrogate checkpoints, evaluated on the physical field $U(t)$. All three model families (POD, SLAE, LLAE) use the same surrogate MLP architecture (Appendix F), so the comparison isolates the effect of the spatial compression strategy rather than differences in the predictor capacity. Two additional columns facilitate interpretation: **Gap** is the difference between the surrogate median and the standalone AE reconstruction median from Table 4 — a positive value indicates overhead introduced by the surrogate step, while a negative value means that joint fine-tuning of the decoder actually **improves** upon the frozen-AE baseline; **vs POD** is the difference relative to the matching POD surrogate from Table 2 at the same (K, d_z, γ) , and is negative throughout, confirming that all AE surrogates outperform the linear baseline.

Overall accuracy. LLAE achieves 4.13–4.63% median L^2 error across all configurations, compared to 5.69–11.61% for SLAE. The best LLAE surrogate ($d_z = 128, K = 16, \gamma = 0.01$, 4.13%) represents a **54%** reduction in median error relative to the best POD

Model	K	d_z	GFLOPs	Latency (ms)	Peak GPU (MB)
SLAE	8	64	15.5	18.4 $\pm$ 1.3	404
SLAE	16	32	31.0	22.7 $\pm$ 1.1	510
SLAE	16	64	31.1	22.7 $\pm$ 1.1	518
SLAE	16	128	31.2	22.7 $\pm$ 1.0	534
SLAE	32	64	62.2	27.9 $\pm$ 1.4	794
LLAE	8	64	54.2	26.8 $\pm$ 2.1	556
LLAE	16	32	53.7	26.9 $\pm$ 1.8	550
LLAE	16	64	54.2	27.0 $\pm$ 1.7	556
LLAE	16	128	55.2	27.2 $\pm$ 2.0	568
LLAE	32	64	54.2	27.1 $\pm$ 1.9	556

Table 5: Reconstruction cost at $\gamma = 0.01$. GFLOPs are counted on the executed graph for one sample. Latency: batch size 1, 50 timed CUDA Event runs after 10 warm-up runs, median $\pm$ standard deviation. Peak GPU memory is the maximum allocated during one reconstruction. Best per column in bold. SLAE’s arithmetic and memory scale with K ; LLAE’s are flat, the decoder running over the N_t time frames whatever the number of Laplace coefficients.

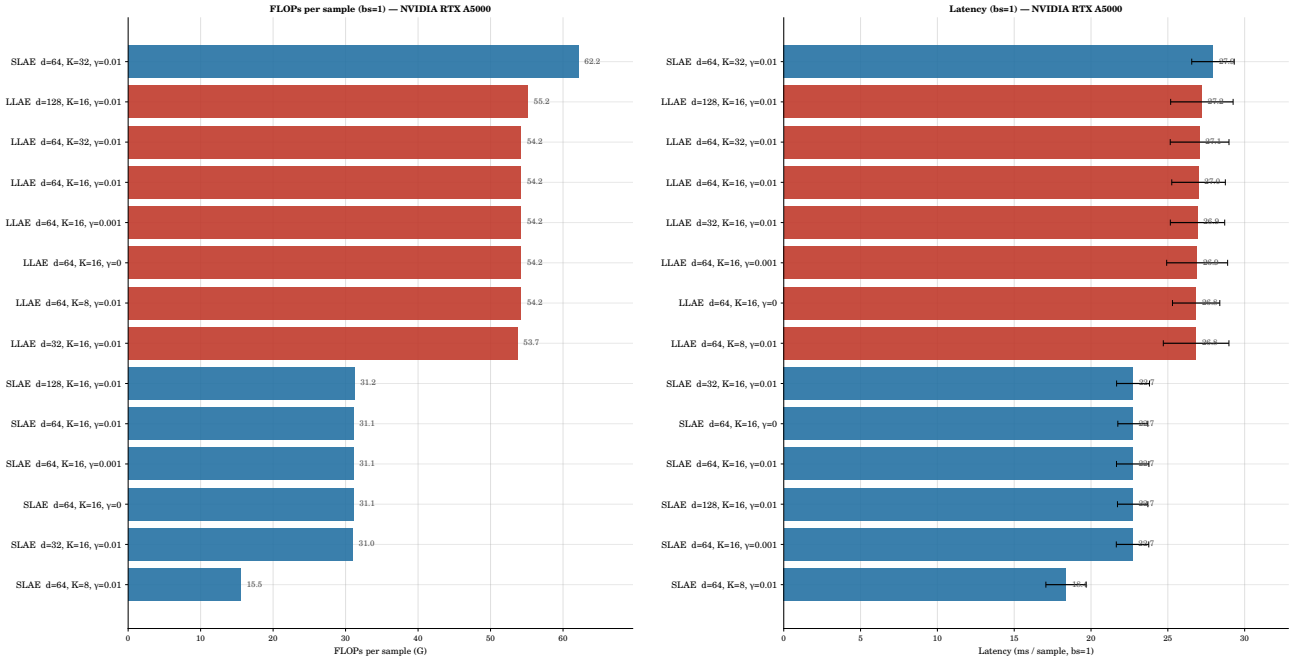


Figure 6: Reconstruction cost for all 14 AE checkpoints (batch size 1). Left: arithmetic work per sample, which separates the two families cleanly — SLAE grows with K , LLAE is flat. Right: wall-clock latency, which does not, the SLAE pipeline evaluating its Laplace transform pair in double precision on the host. LLAE in red, SLAE in blue.

Model	d_z	K	γ	Median (%)	Mean (%)	p90 (%)	Gap (pp)	vs POD (pp)
LLAE	128	16	0.01	4.13	4.48	6.82	+1.21	−7.16
	64	32	0.01	4.63	4.97	7.45	+1.44	−4.12
	64	16	0.01	4.18	4.52	6.84	+0.98	−4.73
	64	16	0.0	4.25	4.60	7.09	+0.99	−4.71
	64	16	0.001	4.21	4.53	6.91	+0.89	−4.73
	64	8	0.01	4.18	4.62	7.11	+0.83	−8.23
	32	16	0.01	4.31	4.75	7.27	+0.88	−7.38
SLAE	64	16	0.0	7.38	7.52	8.80	−0.40	−1.58
	64	16	0.001	7.42	7.56	8.89	−0.40	−1.52
	128	16	0.01	7.49	7.66	9.16	−0.49	−3.80
	64	32	0.01	5.69	6.06	7.97	−2.65	−3.06
	64	16	0.01	7.59	7.86	9.49	−1.33	−1.32
	32	16	0.01	7.62	7.84	9.49	−3.14	−4.07
	64	8	0.01	11.61	11.87	13.83	−0.65	−0.80

Table 6: Direct surrogate results on the test set (1620 simulations), same row order as Table 4. **Gap**: surrogate median – AE reconstruction median; negative values indicate that end-to-end fine-tuning improves upon the standalone AE. **vs POD**: surrogate median – matching POD surrogate median (Table 2); all values are negative. Best per family in bold.

surrogate (8.91%), while the best SLAE surrogate ($d_z = 64$, $K = 32$, $\gamma = 0.01$, 5.69%) achieves a **36%** reduction.

End-to-end fine-tuning effect. The Gap column reveals a qualitative asymmetry between the two families. For LLAE, the surrogate step introduces a small positive overhead of +0.83 to +1.44 pp above the AE reconstruction floor: the surrogate approximation is not perfectly lossless, but the cost is modest given that the AE floor itself already saturates below 3.5%.

For SLAE, by contrast, the Gap is **negative** across all configurations, ranging from -0.40 to -3.14 pp — meaning that the end-to-end surrogate pipeline **outperforms** the standalone SLAE autoencoder. The mechanism is the following. During Phase 1 (AE training), the SLAE loss $\mathcal{L}_{\text{SLAE}}$ (equation (10), Section 2.2.1) is defined purely in the Laplace domain: the decoder is never directly supervised on physical-time reconstruction quality, as computing a physical-time loss would require N_t -frame decoder passes and negate SLAE’s speed advantage. During Phase 2 (surrogate training), the combined loss $\mathcal{L}_{\text{total}}$ (Section 2.3) introduces a direct physical-time reconstruction term for the first time, and the decoder is jointly fine-tuned through it. This spatial supervision signal corrects residual errors that the Laplace-domain loss alone was unable to resolve. The effect is largest when the AE floor is highest: at $d_z = 32$ (-3.14 pp) and $K = 32$ (-2.65 pp), the spatial loss during surrogate training compensates for the harder compression task, lowering the error below the standalone AE baseline.

Hyperparameter sensitivity. LLAE surrogates remain highly robust: varying K from 8 to 32, d_z from 32 to 128, or γ from 0 to 0.01 all yield errors in a tight 4.13–4.63% band, consistent with the insensitivity already observed at the AE level (Section 3.3). SLAE is more sensitive: $K = 8$ collapses to 11.61% (the spectral truncation is too severe for the surrogate to compensate), while $K = 32$ is the best SLAE configuration precisely because the decoder, given access to 32 Laplace frequencies, can recover finer spatial structure during fine-tuning. Ablation violin plots are provided in Appendix C.

Where the error lives in the design. The parameter design is factorial — 15 values of k crossed with 15 of C and 36 injection angles ϕ , the angle being a numerical rotation of the same base solution (Section 3.1) — so an aggregate median mixes three directions of very unequal difficulty. Table 7 decomposes the variance of the per-instance error with the correlation ratio η^2 , the fraction of $\text{Var}(\varepsilon)$ explained by each factor.

The angle explains almost nothing: $\eta_\phi^2 = 0.02$ – 0.03 for every checkpoint of both families, against 0.13–0.26 for k and 0.53–0.74 for C . The error is therefore essentially flat over the 36 rotations and is carried by the two physical parameters, C foremost. Figure 7 shows the corresponding maps: the upper decile is not scattered but confined to the low- C , low- k corner of the design, where reaction and diffusion are slowest and the front takes longest to establish. What Section 3.4 describes as occasional failure cases is thus a identifiable physical regime rather than dispersion.

Two cautions on reading this. A flat dependence on ϕ does **not** establish that the aggregate is free of the optimism the rotation augmentation induces: the partition is random over the 8100 augmented instances, so the training set almost certainly contains the same base (k, C) at other angles, and η^2 cannot see that. It measures how error varies **within** the design, not what a group-wise partition would give. Second, η^2 is

a variance decomposition of a strongly right-skewed quantity and is best read as an ordering of the three factors, not as an exact budget.

Checkpoint	η_ϕ^2	η_k^2	η_C^2
LLAE $d_z = 128, K = 16$	0.027	0.264	0.529
LLAE $d_z = 64, K = 16$	0.028	0.258	0.572
LLAE $d_z = 64, K = 8$	0.028	0.242	0.568
LLAE learnable, $K = 8$	0.025	0.185	0.690
LLAE learnable, $K = 16$	0.022	0.136	0.737
LLAE learnable, $K = 32$	0.021	0.127	0.755

Table 7: Correlation ratio η^2 of the per-instance relative L^2 error against each design factor, on the test set ($n = 1620$). k and C are binned into eight quantiles; ϕ is used at its 36 levels. Values sum to less than one because the factors do not exhaust the variance. The injection angle explains two to three percent throughout.

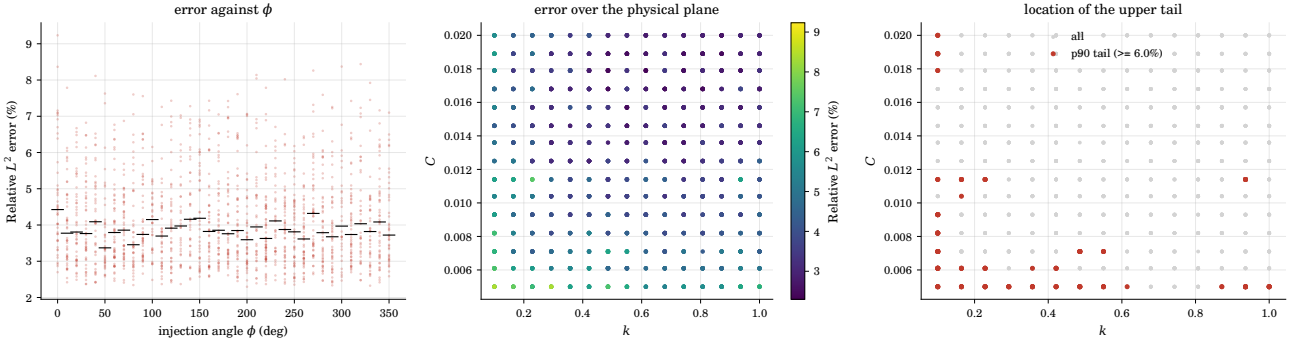


Figure 7: Error over the design of experiments for the best LLAE surrogate. Left: error against the injection angle ϕ , with the median per angle in black — flat across the 36 rotations. Centre: error over the physical plane (k, C), colour encoding the relative L^2 error. Right: location of the upper decile, which occupies the low- C band rather than being dispersed over the design.

Inference cost. The surrogate is the deployed object, and it is also where the two cost axes finally agree. Predicting the Laplace coefficients from μ removes the forward transform entirely — the surrogate emits $\hat{U}(s_k)$ directly and pays only the inversion — so the implementation asymmetry that clouded the reconstruction comparison (Section 3.3) disappears. Measured at batch size 1, SLAE at $K = 16$ requires 30.3 GFLOPs and 15.6 ± 0.3 ms per sample against 48.2 GFLOPs and 22.0 ± 4.2 ms for LLAE: 1.59 $\times$ less arithmetic, 1.41 $\times$ less wall-clock time and 2.6 $\times$ less peak memory (235 against 622 MB). The surrogate MLP itself is negligible in this budget — LLAE’s FLOPs are identical at $K = 8, 16$ and 32 (48.2 GFLOPs), confirming that the decoder, not the predictor, sets the cost. At $K = 8$ SLAE drops to 15.2 GFLOPs and 12.0 ± 0.1 ms, and at $K = 32$ it rises to 60.7 GFLOPs and 23.2 ± 0.7 ms, crossing LLAE at both ends of the sweep. The measurements also differ in dispersion by an order of magnitude — 0.1–0.7 ms for SLAE against 2.8–4.2 for LLAE — the K frequency frames being decoded in one batch where the 150 time frames are not. Figure 8 gives the full breakdown across the 14 checkpoints.

The POD surrogate is instructive as a lower bound: at $K = 16, k_{\text{pod}} = 64$ it consumes 0.2 GFLOPs, some 150 $\times$ less than SLAE, yet runs at 10.1 ms — only 1.5 $\times$ faster — on

212 MB, comparable to SLAE’s 235. Almost all of its time and memory go to the Tikhonov inversion and the back-projection through $V^{(k)}$, not to the predictor. The inversion is thus a fixed toll that every Laplace-domain pipeline pays, and it bounds how much any reduction in network cost can buy at this problem size.

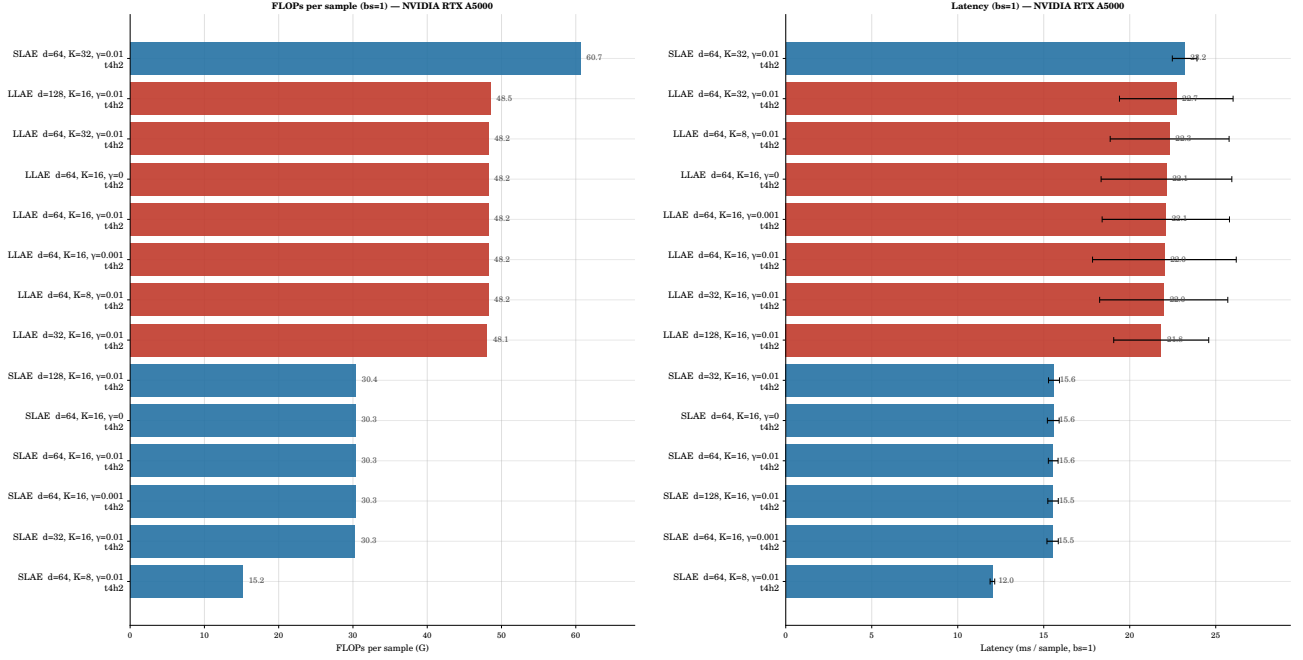


Figure 8: End-to-end surrogate cost for the 14 checkpoints (batch size 1, 50 timed CUDA Event runs). Left: arithmetic work per sample. Right: wall-clock latency. Unlike the reconstruction stage (Figure 6), the two axes agree here: the surrogate applies no forward transform, so SLAE’s fewer operations translate into proportionally lower latency.

Per-checkpoint error histograms for all 14 surrogate checkpoints are provided in Appendix A (Figure 22).

3.4.2 SVD surrogates: single-mode latent compression

The direct surrogate of Section 3.4.1 predicts the full d_z -dimensional latent at each frequency. To evaluate whether compressing that representation before learning the dynamics can instead improve the predictor’s accuracy, we assess a hybrid approach inspired by Rank Reduction Autoencoders (RRAE) [25]: projecting the latent trajectories onto a truncated SVD basis before applying the surrogate model. We evaluate 26 AE+SVD surrogate checkpoints across the LLAE and SLAE families. In these models the encoder is frozen, whereas the decoder and the SVD basis V are trained end-to-end alongside the surrogate; the latent dimension is reduced to k_{svd} (either 16 or 32) via POD (SVD) before an MLP predicts the reduced coefficients from the physical parameters.

Table 8 reports the relative L^2 error on the held-out test set evaluated on the physical field $U(t)$.

Does compressing the latent space help? The central question is whether predicting on a smaller, orthogonalised latent space simplifies the MLP’s task enough to offset the information lost to SVD truncation. It does not. Measured **at matching autoencoder configuration**, the SVD bottleneck adds a small penalty over the direct surrogate for both families: fixing $d_z = 64$, $\gamma = 0.01$, the best LLAE-SVD ($K = 16$, $k_{\text{svd}} = 32$, 4.24%) sits

Model	d_z	K	γ	k_{svd}	Median (%)	Mean (%)	p90 (%)
LLAE	64	16	0.01	32	4.24	4.61	6.75
	64	32	0.01	32	4.29	4.69	7.02
	64	16	0.001	32	4.44	4.96	7.41
	64	16	0.0	32	4.50	4.97	7.29
	64	8	0.01	32	4.60	5.04	7.47
	128	16	0.01	32	4.77	5.30	7.82
	32	16	0.01	16	5.55	6.29	9.39
	64	32	0.01	16	5.58	6.22	9.35
	128	16	0.01	16	5.83	6.51	9.48
	64	16	0.01	16	5.84	6.58	9.79
	64	16	0.001	16	5.90	6.72	9.92
	64	16	0.0	16	5.94	6.76	10.18
	64	8	0.01	16	6.54	7.37	11.05
	64	32	0.01	32	6.03	6.58	9.02
SLAE	64	32	0.01	16	6.96	7.80	11.24
	64	16	0.0	32	7.50	7.73	9.26
	64	16	0.001	32	7.62	7.88	9.52
	64	16	0.01	32	7.90	8.32	10.54
	32	16	0.01	16	7.95	8.38	10.71
	64	16	0.001	16	8.01	8.43	10.80
	128	16	0.01	32	8.12	8.57	10.98
	64	16	0.0	16	8.13	8.61	11.06
	64	16	0.01	16	8.71	9.40	12.68
	128	16	0.01	16	8.89	9.71	13.43
	64	8	0.01	32	11.64	11.93	13.96
	64	8	0.01	16	11.97	12.38	14.88

Table 8: AE+SVD surrogate results on the test set ($n = 1620$), evaluating the physical field reconstruction error using LLAE and SLAE autoencoders coupled with an intermediate SVD compression ($k_{\text{svd}} \in \{16, 32\}$). Best median per family in bold.

+0.06 pp above its direct counterpart (4.18%), and the best SLAE-SVD ($K = 32$, $k_{\text{svd}} = 32$, 6.03%) +0.34 pp above its own (5.69%). The MLP navigates the full d_z -dimensional latent space perfectly well, so the SVD projection merely discards spatial detail the direct surrogate had already learned to predict — compressing the latent before the surrogate degrades the reconstruction rather than helping it.

Impact of k_{svd} . To quantify the benefit of retaining a larger number of SVD modes, we compare pairs of configurations differing only in k_{svd} (16 vs. 32). Figure 9 highlights that setting $k_{\text{svd}} = 32$ systematically improves the surrogate’s accuracy across all models (all points lie below the $y = x$ diagonal). The gain is significantly more pronounced for LLAE models (between +1.06 and +1.94 pp reduction in median error) than for SLAE models (+0.33 to +0.93 pp), indicating that LLAE latent spaces contain more useful fine-grained information captured by the additional 16 SVD modes, whereas SLAE may inherently discard spatial details at the AE stage, limiting the potential gains of a larger k_{svd} .

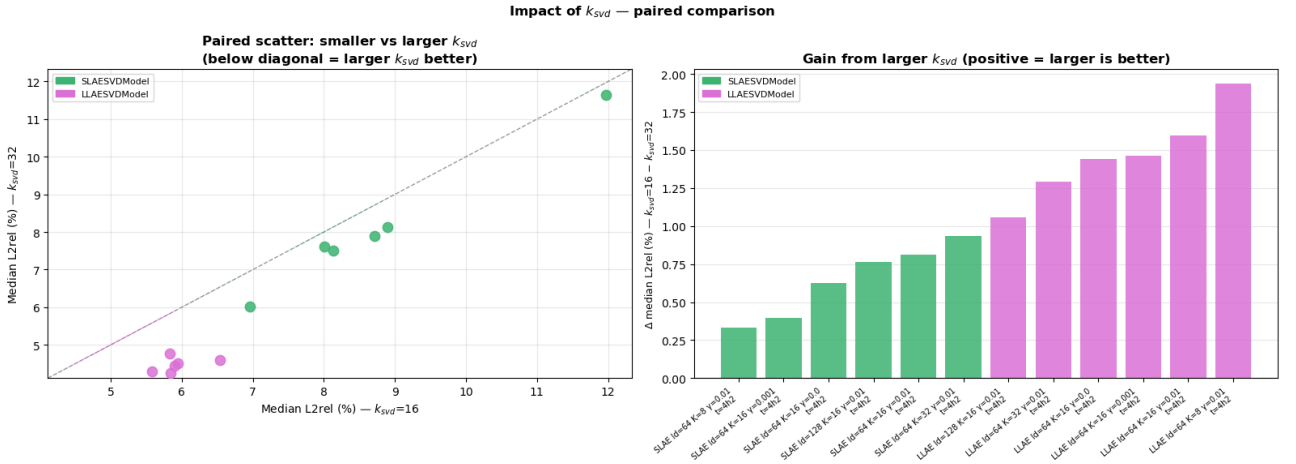


Figure 9: Impact of k_{svd} on SVD surrogate performance. Left: Scatter of paired median errors showing $k_{\text{svd}} = 32$ consistently outperforms $k_{\text{svd}} = 16$. Right: Absolute error reduction when shifting from 16 to 32 modes, sorted by magnitude.

Hyperparameter sensitivity. Sensitivity to the AE hyperparameters K , d_z , and γ mirrors the direct case (Section 3.4.1): SLAE again degrades sharply at $K = 8$ (11.6–12.0%), while LLAE stays within a tight band. The lever specific to this pipeline is k_{svd} , whose effect is quantified above: retaining 32 rather than 16 modes is uniformly beneficial, and more so for LLAE than for SLAE.

Inference latency. Introducing the SVD decoding step (matrix multiplication) between the MLP prediction and the spatial decoder adds a slight computational overhead. Measured at batch size 1, LLAE-SVD requires approximately 22 ms per sample, while SLAE-SVD requires about 9 ms per sample. The SLAE-SVD model maintains its pronounced speed advantage ($\approx 2.4\times$) over LLAE-SVD, reflecting its lighter decoder architecture. The trade-off between latency and median error for AE+SVD surrogates is presented in Figure 10.

3.4.3 Tucker surrogates: joint-mode latent compression

Section 3.4.2 compressed the latent representation along a single mode. We now evaluate the Tucker variant of Section 2.3, which factorises the frequency and latent modes **jointly**:

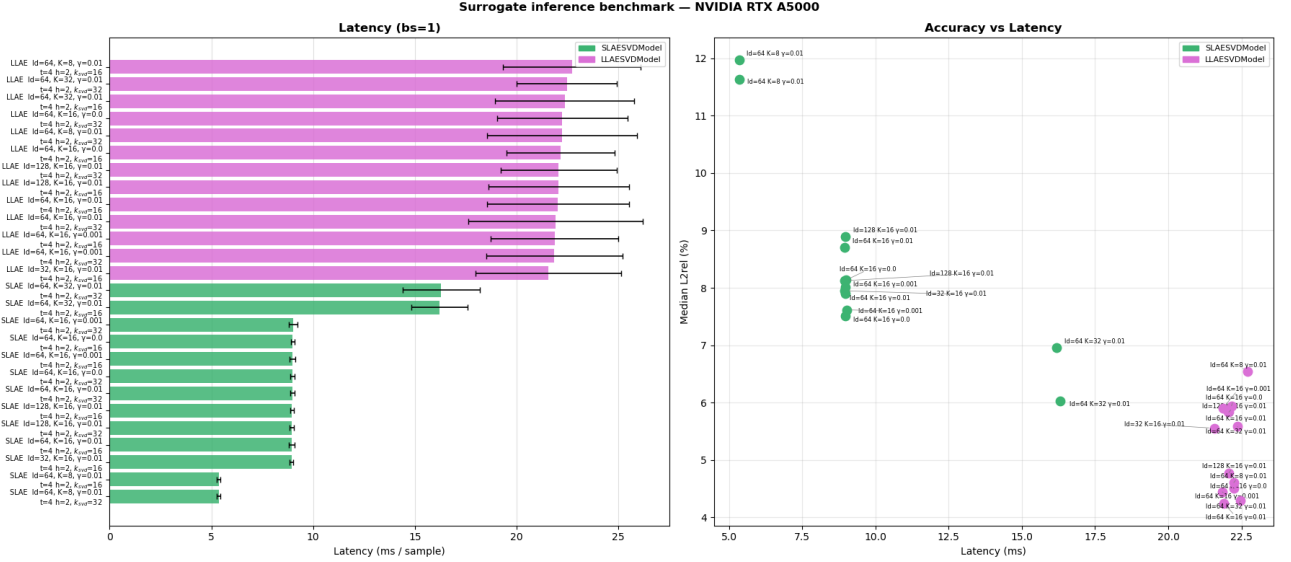


Figure 10: AE+SVD surrogate inference benchmark (batch size 1). Left: per-sample latency across checkpoints. Right: accuracy–latency scatter representing median L^2 error vs. latency.

the surrogate predicts a compact core matrix $\hat{\mathcal{G}} \in \mathbb{C}^{r_s \times r_z}$ (real-valued for SLAE) instead of K independent latents of dimension d_z , back-projecting through the frozen HOOI factors $U^{(s)}, U^{(z)}$ before decoding. We sweep the Tucker ranks over a grid $r_s \in \{4, 8\}$ (frequency mode, $K = 16 \rightarrow r_s$) $\times r_z \in \{8, 16, 32\}$ (latent mode, $d_z = 64 \rightarrow r_z$), for both families, yielding 12 checkpoints. All share the same Phase 1 autoencoder ($d_z = 64$, $K = 16$, $\gamma = 0.01$); only the offline factors and the surrogate differ. The compression ratio $\rho = r_s r_z / (K d_z)$ measures the reduction of the surrogate target dimension relative to the direct pipeline (which predicts $K d_z = 1024$ scalars per sample).

Beyond the physical-field error $\varepsilon_{\text{field}}$ (relative L^2 on $U(t)$, full pipeline), we report two decoder-free diagnostics computed in the latent space, which split the difficulty between the offline compression and the surrogate MLP:

$$(40) \quad \varepsilon_{\text{trunc}} = \frac{\|\hat{Z} - \Pi \hat{Z}\|}{\|\hat{Z}\|}, \quad \varepsilon_{\text{reg}} = \frac{\|\hat{\mathcal{G}}_{\text{pred}} - \hat{\mathcal{G}}_{\text{true}}\|}{\|\hat{\mathcal{G}}_{\text{true}}\|},$$

where $\Pi = U^{(s)} U^{(s)\top} (\cdot) U^{(z)} U^{(z)\top}$ is the Tucker projector. $\varepsilon_{\text{trunc}}$ measures the energy discarded by the rank truncation (factors only, independent of the surrogate), while ε_{reg} measures how well the MLP predicts the true core (surrogate only, independent of the truncation). Table 9 reports all three on the held-out test set.

Overall accuracy. As with SVD, the relevant question is what the extra compression costs relative to the direct and single-mode pipelines **at matching autoencoder configuration**. Fixing the shared AE ($d_z = 64$, $K = 16$, $\gamma = 0.01$), the median error grows only gently along the compression axis: $4.18\% \rightarrow 4.24\% \rightarrow 4.33\%$ for LLAE (direct $\rightarrow$ SVD with $k_{\text{svd}} = 32 \rightarrow$ Tucker with $r_s = 8$, $r_z = 32$), and $7.59\% \rightarrow 7.90\% \rightarrow 7.92\%$ for SLAE. Tucker thus adds at most $+0.15$ pp over the direct surrogate and $+0.09$ pp over SVD — while shrinking the surrogate target to $r_s r_z = 256$ scalars, a $4\times$ reduction over the direct target ($K d_z = 1024$) and a $2\times$ reduction over the SVD target ($K k_{\text{svd}} = 512$). Factorising both modes at once is what buys this: exploiting cross-frequency correlations lets Tucker match the single-mode SVD accuracy at half the target dimension.

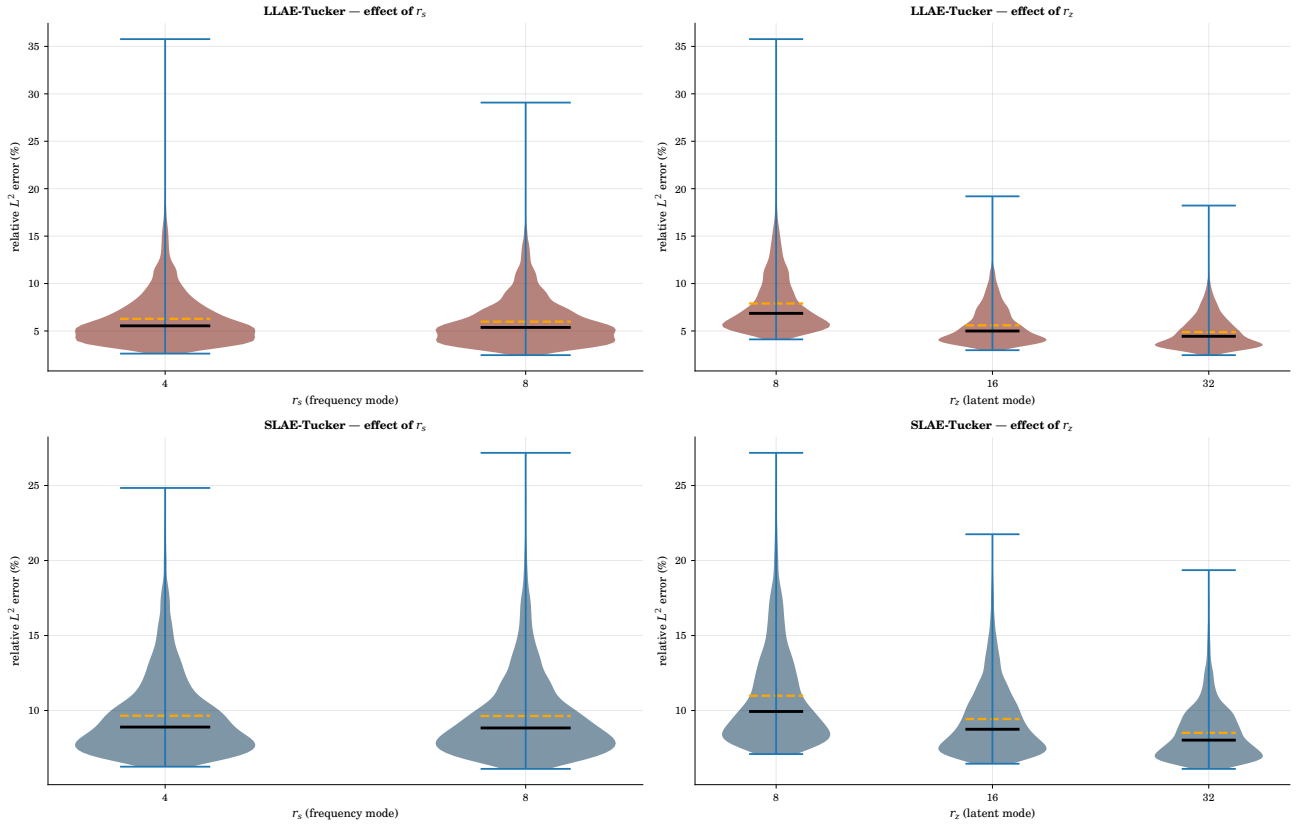
Model	r_s	r_z	ρ	Median (%)	Mean (%)	p90 (%)	$\varepsilon_{\text{trunc}}$ (%)	ε_{reg} (%)	Lat (ms)
LLAE	8	32	0.250	4.33	4.75	7.16	36.98	7.25	18.2
	4	32	0.125	4.57	5.04	7.61	40.34	5.67	18.0
	8	16	0.125	4.87	5.43	8.11	61.86	4.38	18.3
	4	16	0.062	5.16	5.77	8.70	61.59	3.83	18.0
	8	8	0.062	6.85	7.78	11.75	81.21	4.99	18.3
	4	8	0.031	6.87	8.02	12.73	81.03	3.80	18.0
SLAE	8	32	0.250	7.92	8.35	10.55	42.40	6.76	7.4
	4	32	0.125	8.15	8.68	11.14	50.82	6.58	7.1
	4	16	0.062	8.68	9.36	12.79	69.59	5.18	7.1
	8	16	0.125	8.85	9.51	12.89	67.59	5.83	7.5
	4	8	0.031	9.93	10.92	15.61	80.63	5.70	7.1
	8	8	0.062	9.96	11.05	16.00	80.05	7.16	7.5

Table 9: Tucker surrogate results on the test set ($n = 1620$), sorted by median error within each family. $\rho = r_s r_z / (K d_z)$ is the compression ratio of the surrogate target relative to the direct pipeline. $\varepsilon_{\text{trunc}}$: median latent-energy discarded by the rank truncation (factors only). ε_{reg} : median relative error of the predicted core (surrogate only). **Lat**: GPU inference latency at batch size 1. Best median per family in bold.

Rank sensitivity: r_z dominates r_s . The rank grid of Table 9 is strongly anisotropic, and this is the most informative outcome of the sweep. Increasing the latent-mode rank r_z from 8 to 32 nearly halves the LLAE error ($6.85\% \rightarrow 4.33\%$ at $r_s = 8$), and the SLAE error falls in step ($9.96\% \rightarrow 7.92\%$). Doubling the frequency-mode rank r_s from 4 to 8, by contrast, barely moves either ($4.57\% \rightarrow 4.33\%$ for LLAE, $8.15\% \rightarrow 7.92\%$ for SLAE at $r_z = 32$) — an effect an order of magnitude smaller. The rank sweeps of Figure 11 confirm the pattern distributionally: the r_z violins separate cleanly while the r_s violins overlap almost completely.

The interpretation reaches beyond this ablation. A rank-4 basis over the frequency mode already captures almost all of the joint variation of the $K = 16$ Laplace coefficients across the training set: the spectral representation is redundant enough that discarding three quarters of its dimensions costs nothing measurable. The $d_z = 64$ latent channels behave in the opposite way — they carry richer, less redundant information than a rank-8 basis truncates too aggressively. A tensor factorisation, fitted blind to the physics on training statistics alone, thus rediscovers the premise of this work: the Laplace representation is intrinsically compact along the frequency axis, and $K = 16$ already sits close to its useful floor. That this evidence arrives from a direction independent of the argument which motivated the transform is what makes it worth reporting.

In practice $r_s = 4$ is the parameter-efficient choice: it halves the surrogate size (1.35 M vs. 1.9 M parameters for $r_s = 8$) at a negligible accuracy cost. One caveat bounds the claim: the frequency axis is sampled at two points only ($r_s \in \{4, 8\}$), so the sweep establishes that the error is insensitive to r_s over that range, not that it has saturated. The uncompressed limit $r_s = K$ is structurally the single-mode compression of Section 3.4.2, although the two differ in the nature of the basis — real and trained there, complex and frozen here.



Truncation vs regression. The two decoder-free diagnostics of Table 9, shown in Figure 12, locate the accuracy bottleneck. The regression error ε_{reg} stays small throughout (3.8–7.3%): the MLP predicts the compact core reliably, and it grows only mildly as the core enlarges ($r_z = 32$ is harder to fit than $r_z = 8$). The truncation error $\varepsilon_{\text{trunc}}$, by contrast, is large and is what the field error tracks: it falls from $\sim 81\%$ at $r_z = 8$ to $\sim 37\%$ at $r_z = 32$ (LLAE), mirroring the field-error drop. Note that $\varepsilon_{\text{trunc}}$ is a **latent-space energy**, not a field error — discarding 80% of the latent norm costs only $\sim 7\%$ on $U(t)$, because the removed Tucker directions carry little decodable information. Its **variation** with the ranks, not its absolute value, is what governs accuracy. The conclusion is that the Tucker compression, not the surrogate, is the binding constraint: the MLP is never the limiting factor, so any accuracy gain must come from a richer latent basis (larger r_z) rather than a larger predictor.

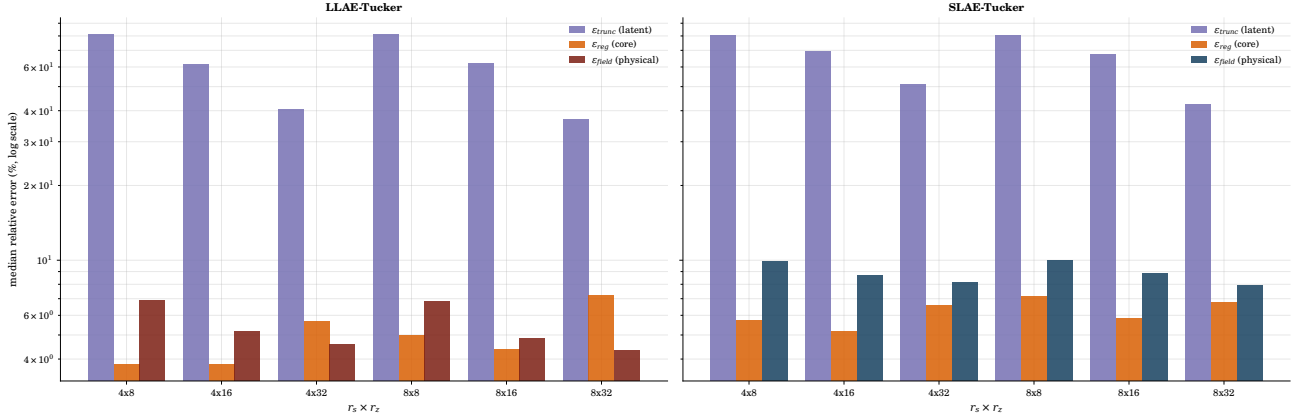


Figure 12: Error decomposition for LLAE-Tucker (left) and SLAE-Tucker (right): median latent truncation error $\varepsilon_{\text{trunc}}$, core regression error ε_{reg} , and physical-field error $\varepsilon_{\text{field}}$ for each (r_s, r_z) pair (log scale). The field error follows the truncation error, not the regression error.

Compression trade-off. Figure 13 plots the field error against the compression ratio ρ and against the surrogate size. Along the Pareto front, LLAE-Tucker degrades gracefully as the target is compressed: $r_s = 4$, $r_z = 32$ retains 4.57% median error at $\rho = 0.125$ (an $8\times$ smaller target than the direct surrogate), and $r_s = 4$, $r_z = 16$ still reaches 5.16% at $\rho = 0.062$ ($16\times$). Tucker therefore offers a tunable accuracy/compactness dial that the direct and SVD surrogates lack: when the surrogate output must be kept small — for storage, for a lightweight predictor, or for downstream many-query use — Tucker sustains near-direct accuracy at a fraction of the target dimension. It does not, however, **beat** the direct surrogate on raw accuracy; as with SVD (Section 3.4.2), reducing the latent space before the surrogate discards information the direct predictor is capable of learning.

Decoder co-adaptation. A subtle consequence of the end-to-end fine-tuning (Section 3.4.1) is worth flagging. Because the decoder is jointly updated during surrogate training ($\text{lr}_{\text{decoder}} = 5 \times 10^{-5}$), it can co-adapt to the distribution of **predicted** cores rather than the true ones. We probe this by decoding the exact projected core in place of the surrogate’s prediction: if the decoder had not co-adapted, the exact core would yield a lower error and serve as a valid oracle lower bound. For LLAE-Tucker the substitution is neutral (+0.07 pp), so the oracle bound holds. For SLAE-Tucker, however, decoding the exact core **raises** the error sharply (+11.1 pp on the probe subsample at $r_s = 8$,

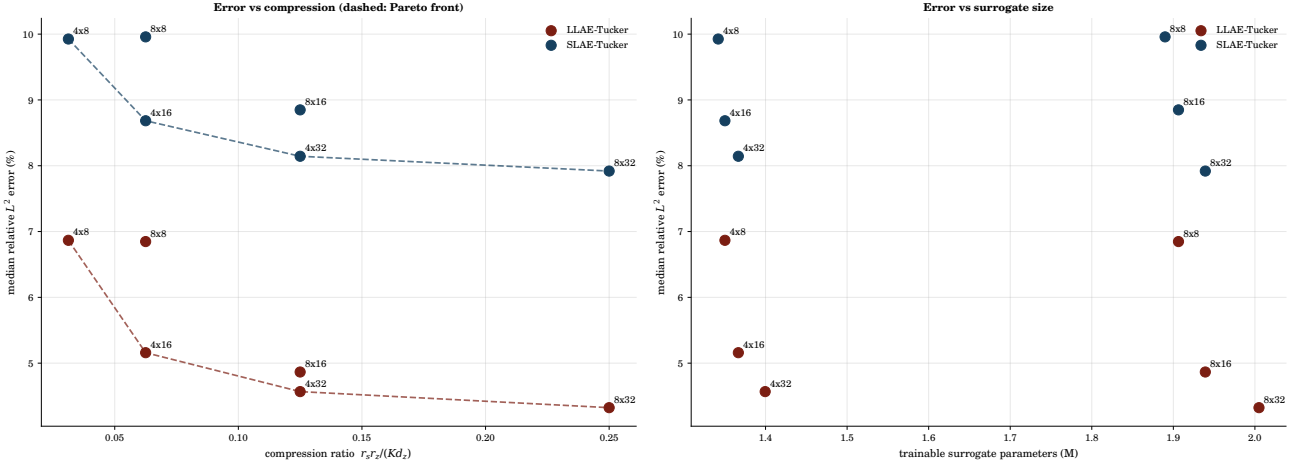


Figure 13: Left: median field error vs. compression ratio $\rho = r_s r_z / (K d_z)$ (dashed line = Pareto front per family; points sharing a ratio differ in (r_s, r_z)). Right: median field error vs. trainable surrogate parameters.

$r_z = 16$): the SLAE decoder has specialised to the predicted-core distribution, exactly the mechanism that produced the negative surrogate Gap for SLAE in Section 3.4.1. A physical-space oracle is therefore not a valid lower bound for SLAE-Tucker, which is why the truncation and regression diagnostics above are computed in the latent space, where they remain decoder-independent.

Inference cost. The Tucker back-projection $(U^{(s)} \hat{\mathcal{G}} U^{(z)})^\top$, two small matrix products) is negligible in the inference budget, and the measurements make this unusually clear: across the twelve checkpoints, the FLOP count is **constant** within each family — 30.3 GFLOPs for every SLAE-Tucker and 48.2 for every LLAE-Tucker — although the ranks vary by a factor of eight in r_z and two in r_s . Compressing the surrogate target changes what the predictor must learn, not what the decoder must compute. Latency follows suit (Figure 14): ~ 15 ms for SLAE-Tucker and ~ 24 ms for LLAE-Tucker at batch size 1, essentially independent of the ranks and within 3% of the direct pipelines of Section 3.4.1. SLAE-Tucker thus keeps the $1.6\times$ arithmetic and $1.6\times$ wall-clock advantage of its lighter decoder, at the cost of the higher error floor discussed above — and the rank reduction, whose entire purpose was to lower cost, buys none.

Per-checkpoint error histograms for all 12 Tucker surrogates are provided in Appendix A (Figure 24).

3.5 Optimal Laplace contour choice

The results reported so far all rest on a prescribed integration contour: ω_k set to the K leading Fourier modes of the time grid, uniform abscissa $\sigma_k = \gamma$, and (α_t, λ) set a priori and never tuned. The comparison of this section is therefore between two **triplets** (s_k, α_t, λ) , one conventional and one optimised, and not between two placements of the poles at equal regularisation (Section 2.4). We assess the two selection strategies of Section 2.4, which differ in when and against what criterion the triplet is determined: the **learnable** contour of LLAE, where $(\sigma_k, \omega_k, \alpha_t, \lambda)$ are treated as parameters of the model and optimised jointly with the autoencoder, and the **offline** contour of SLAE, determined before any training by minimising the bias–amplification proxy (31). Six configurations are trained, one per (family, K) pair with $K \in \{8, 16, 32\}$, all at $d_z = 64$; architecture,

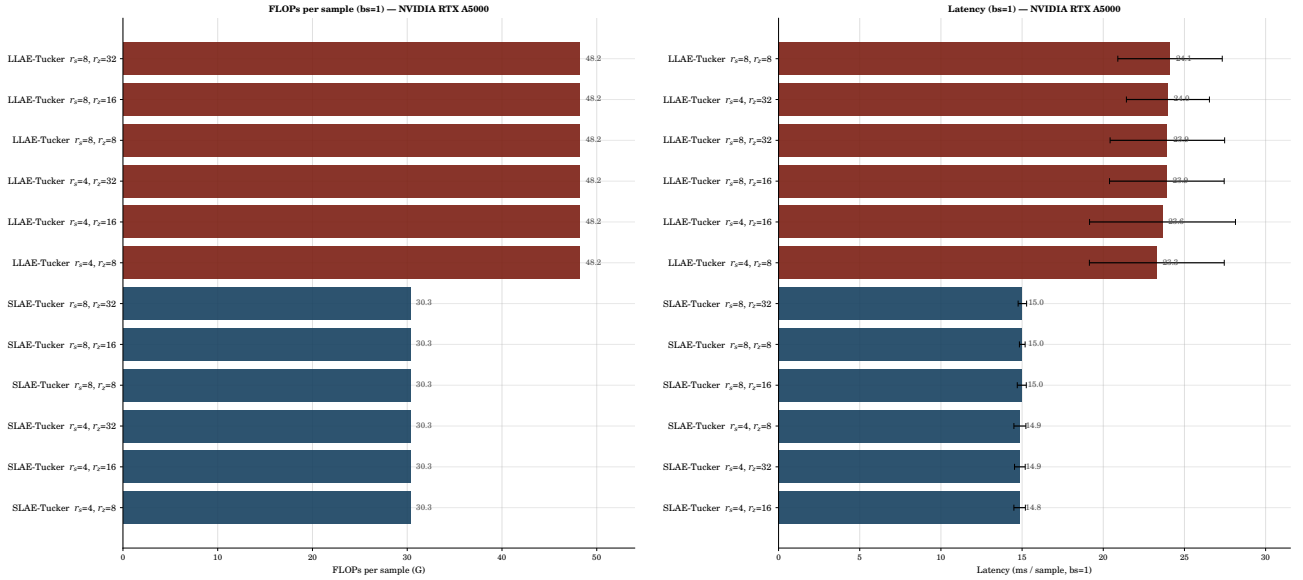


Figure 14: Tucker surrogate inference benchmark (50 CUDA Event timed runs, batch size 1). Left: per-checkpoint latency. Right: accuracy–latency scatter. Latency is set by the decoder family, not the Tucker ranks.

optimiser, schedule and data partition are held at the values of Sections 3.3–3.4.1, so that each configuration differs from its prescribed counterpart by the triplet alone. The offline contours are obtained by 500 AdamW steps on (31) (learning rate 5×10^{-3} on plateau decay, weight decay 10^{-4} , gradient clipping at 1.0, double precision), with $\lambda_{\text{AE}} = 1.25$, $\lambda_{\Delta t} = \lambda_{\Delta x} = 0.5$, rank $d_z = 64$ for the SVD residual, and both terms evaluated over the same $N_{\text{opt}} = 100$ training instances sampled at a single rotation angle. The poles are initialised on the prescribed Bromwich contour at $\gamma = 0$ and (α_t, λ) at 10^{-3} ; all four are optimised jointly under the box constraints $\sigma_k \geq -0.05$, $\omega_k \geq 0$ and $\alpha_t, \lambda \in [10^{-6}, 1]$. The resulting triple (s_k, α_t, λ) is then held fixed throughout both phases — it defines the Laplace-domain dataset, the inversion inside the surrogate, and the inversion at inference alike.

Geometry of the optimised contours. Figure 15 displays the resulting sampling points in the complex plane alongside their prescribed counterparts. Both strategies displace the contour in the same qualitative direction and by a substantial margin: departing from the imaginary axis, the abscissae acquire a positive real part ($\sigma_k \approx 0.05$ – 0.06 for LLAE at $K = 16$, ≈ 0.035 for SLAE), so that the optimised contour is a genuine Bromwich contour displaced into the right half-plane rather than a purely oscillatory one, while the imaginary parts are concurrently dilated, widening the spectral window sampled by the transform.

The two strategies nonetheless select markedly different regularisation regimes. The learnable contour drives the ridge term down to $\lambda \approx 1.2 \times 10^{-5}$, leaving the inversion nearly unregularised and relying on the contour geometry alone for conditioning; the offline optimum retains a far heavier ridge ($\lambda \approx 4.3 \times 10^{-3}$ at $K = 16$, rising to 8.9×10^{-2} at $K = 32$). This asymmetry expresses the two objectives directly: the offline criterion assigns an explicit cost to the amplification factor $\|A^{-1}F^H\|$ multiplying the **predicted**-coefficient error and trades bias for conditioning accordingly, whereas the end-to-end objective, which passes through the same inversion, sees that amplification act on the far smaller autoencoder error and is therefore almost insensitive to λ .

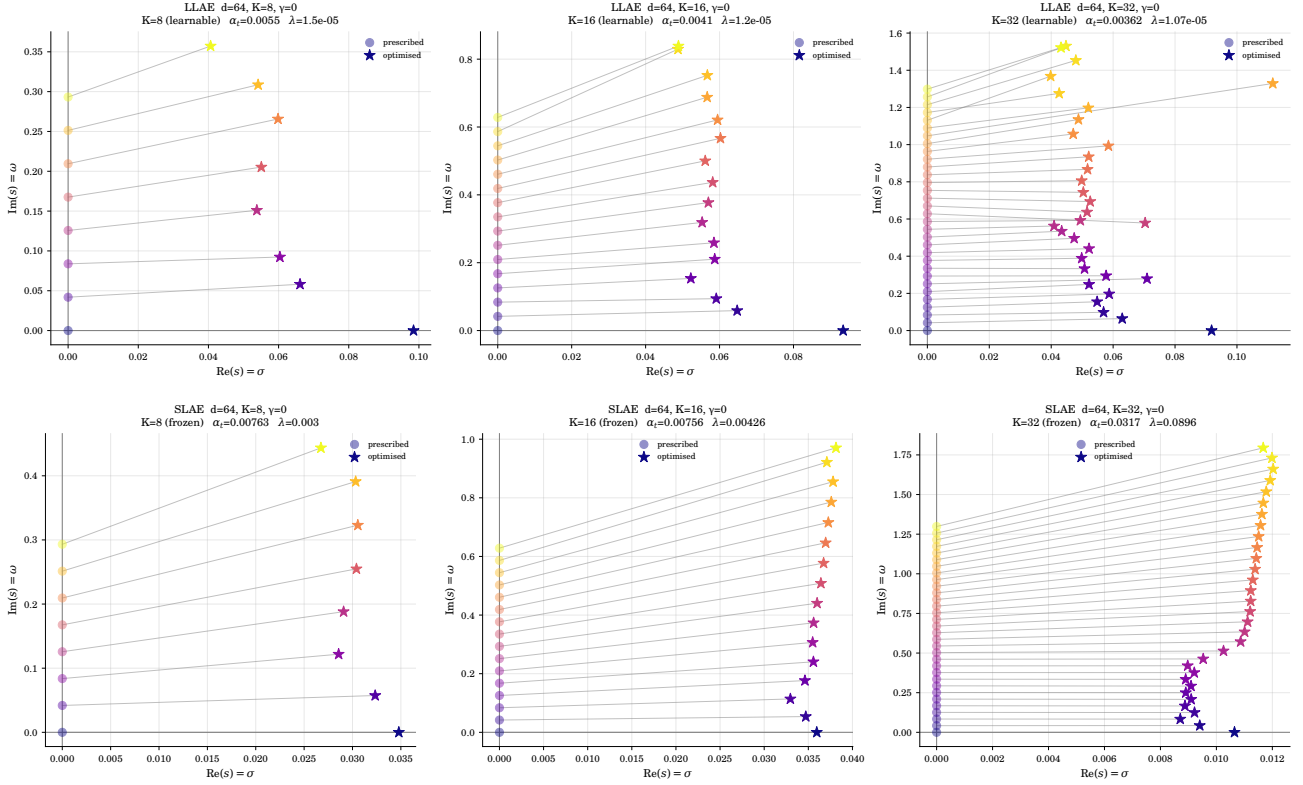


Figure 15: Optimised Laplace contours in the complex plane ($d_z = 64$, $K \in \{8, 16, 32\}$). Circles: prescribed sampling points on the truncated Bromwich contour; stars: optimised points, joined to their prescribed position by a grey segment; colour encodes the frequency index k . Top row: LLAE learnable contour, optimised jointly with the autoencoder. Bottom row: SLAE offline contour, obtained by minimising the proxy (31). The associated (α_t, λ) are reported in each panel title.

Autoencoder reconstruction. Table 10 reports both stages for the six configurations, together with the variation induced with respect to the corresponding prescribed contour (all comparisons are made at $\gamma = 0.01$; since γ was shown to be inconsequential over $[0, 0.01]$ in Section 3.3, this choice does not affect the conclusions). At the reconstruction stage the two strategies behave in opposite ways: the learnable contour lowers the error floor at every K , by 0.46 to 0.58 pp, whereas the offline contour **degrades** it — moderately at $K = 8$ and $K = 32$ (+0.66 and +0.19 pp), pronouncedly at $K = 16$ ($8.92 \rightarrow 16.05\%$). The reconstruction stage is however not the deliverable, and this divergence is the expected consequence of evaluating each criterion away from the regime for which it was calibrated. Appendix D details the reconstruction-level behaviour.

Origin of the reconstruction penalty. Both the standalone autoencoder and the end-to-end surrogate feed **approximate** Laplace coefficients into the **same** inversion, so the decomposition (25) applies verbatim to both, with only the magnitude of the coefficient error changing:

$$(41) \quad \mathcal{E}(\epsilon) \lesssim \underbrace{\|A^{-1}L_{\text{reg}}\mathbf{u}\|}_{B(\text{contour})} + \underbrace{\|A^{-1}F^H\|}_{a(\text{contour})} \cdot \epsilon, \quad \epsilon = \begin{cases} \epsilon_{\text{AE}} & (\text{AE stage}), \\ \epsilon_{\text{surr}} & (\text{surrogate stage}), \end{cases}$$

with $\epsilon_{\text{AE}} \ll \epsilon_{\text{surr}}$, the surrogate having to regress the coefficients from μ in addition to compressing the field. Since B and a depend on the contour alone, comparing two contours amounts to comparing two affine functions of ϵ , the optimised one being preferable above the crossover $\epsilon^* = (B_{\text{opt}} - B_{\text{fix}})/(a_{\text{fix}} - a_{\text{opt}})$ at which the additional bias is amortised by the reduced amplification. The proxy (31) is calibrated at $\epsilon = \epsilon_{\text{surr}}$, and accordingly selects a contour that is deliberately more biased and better conditioned; the sign pattern of Table 10 is consistent with a crossover situated between the two stages. The procedure thus selects a representation that is **less amenable to compression** but **better conditioned under inversion**, the latter property being the one that governs the accuracy of the complete emulator.

Two observations complete this reading. First, the penalty is not governed by the ridge term alone: $K = 32$ carries by far the heaviest regularisation ($\lambda \approx 8.9 \times 10^{-2}$) yet incurs the smallest penalty (+0.19 pp), whereas $K = 16$ incurs +7.13 pp with $\lambda \approx 4.3 \times 10^{-3}$; both B and a are set by the joint geometry of (s_k, α_t, λ) . Second, the penalty is largely **recoverable**, the bias operator $A^{-1}L_{\text{reg}}$ being linear, fixed and known, so that a decoder optimised **through** the inversion can pre-compensate it. The SLAE Phase-1 objective, formulated entirely in the Laplace domain, never involves the inversion (Section 2.2.1) and precludes such compensation; Phase 2 introduces the physical-time term and the bias is absorbed. This amplifies the mechanism already responsible for SLAE’s negative Gap under a prescribed contour (Section 3.4.1), which deepens from -1.33 to -8.59 pp at $K = 16$, and explains the absence of any comparable effect for LLAE, whose Phase-1 objective (12) already contains a physical-time term.

End-to-end surrogate. The surrogate columns of Table 10 invert the ordering, and Figure 16 shows the corresponding error distributions. For SLAE, the offline contour reduces the end-to-end error at every K : by 1.85 pp at $K = 8$ ($11.61 \rightarrow 9.76\%$), 0.65 pp at $K = 32$ ($5.69 \rightarrow 5.04\%$) and 0.13 pp at $K = 16$ ($7.59 \rightarrow 7.46\%$), notwithstanding the +7.13 pp reconstruction penalty incurred at that same K . The improvement is largest where the prescribed contour was least adequate, at $K = 8$, for which the spectral

Family	Contour	K	AE med. (%)	Δ AE (pp)	Surr. med. (%)	Δ Surr. (pp)	Gap (pp)
LLAE	learnable	8	2.89	−0.46	4.54	+0.36	+1.65
	learnable	16	2.68	−0.52	4.63	+0.45	+1.95
	learnable	32	2.61	−0.58	4.58	−0.05	+1.97
SLAE	offline	8	12.92	+0.66	9.76	−1.85	−3.16
	offline	16	16.05	+7.13	7.46	−0.13	−8.59
	offline	32	8.53	+0.19	5.04	−0.65	−3.49

Table 10: Optimised-contour configurations ($d_z = 64$), test set ($n = 1620$). **AE med.:** standalone reconstruction error; **Surr. med.:** end-to-end surrogate error. The Δ columns give the variation with respect to the corresponding prescribed-contour configuration of Tables 4–6 at identical (d_z, K) and $\gamma = 0.01$; negative values indicate an improvement brought by the optimised contour. **Gap:** surrogate median – AE median. Best per family in bold.

truncation is most severe and the placement of the sampling points correspondingly most consequential. The Gap column exhibits the mechanism directly: negative throughout and reaching -8.59 pp at $K = 16$, it indicates that the entire reconstruction-stage penalty is recovered once the surrogate is trained. At $K = 32$, SLAE attains 5.04%, the most accurate SLAE result of this study and a 43% reduction relative to the best POD surrogate (8.91%), while retaining the reduced decoding cost characteristic of the family.

The learnable contour serves here as a control. Its benefit does **not** propagate to the surrogate stage: despite a distinctly lower reconstruction floor, the end-to-end error remains essentially constant (4.54–4.63%, against 4.13–4.63% under the prescribed contour) and is marginally higher at $K = 8$ and $K = 16$ (+0.36 and +0.45 pp). Displacing the sampling points outward sharpens the reconstruction while rendering the map $\mu \mapsto \hat{z}(s_k)$ harder to regress: the contour is free to buy fidelity at the price of a target that is more difficult to learn, nothing in a reconstruction objective opposing that exchange.

This is not overfitting in the usual sense: the learnable contour improves the reconstruction error on the **held-out** set (Appendix D), so neither the autoencoder nor the contour is fitting noise in the training data. What the contour overfits is the **task**: optimised against reconstruction alone, it specialises the latent representation to that criterion, with no pressure to keep it predictable from μ . The widened spectral window it selects populates the latent trajectories with faster-varying components, so that the parameter-to-coefficient map to be regressed becomes correspondingly less regular — a plausible mechanism, though one we do not attempt to establish here, as it would require characterising the regularity of $\mu \mapsto \hat{z}(s_k)$ under each contour independently of the surrogate trained on it.

Inference cost. Contour optimisation modifies only the numerical values of (s_k, α_t, λ) , leaving every tensor dimension unchanged, and is therefore free at inference. The measurements confirm this to the digit: the optimised-contour surrogates consume exactly the arithmetic of their prescribed-contour counterparts — 15.2, 30.3 and 60.7 GFLOPs for SLAE at $K = 8, 16, 32$, and 48.2 GFLOPs for LLAE irrespective of K — and evaluate in 11.9, 15.5 and 22.9 ms respectively for SLAE against 23.4–24.3 ms for LLAE (batch size 1, 50 timed CUDA Event runs on the idle benchmarking GPU). At $K = 16$ SLAE

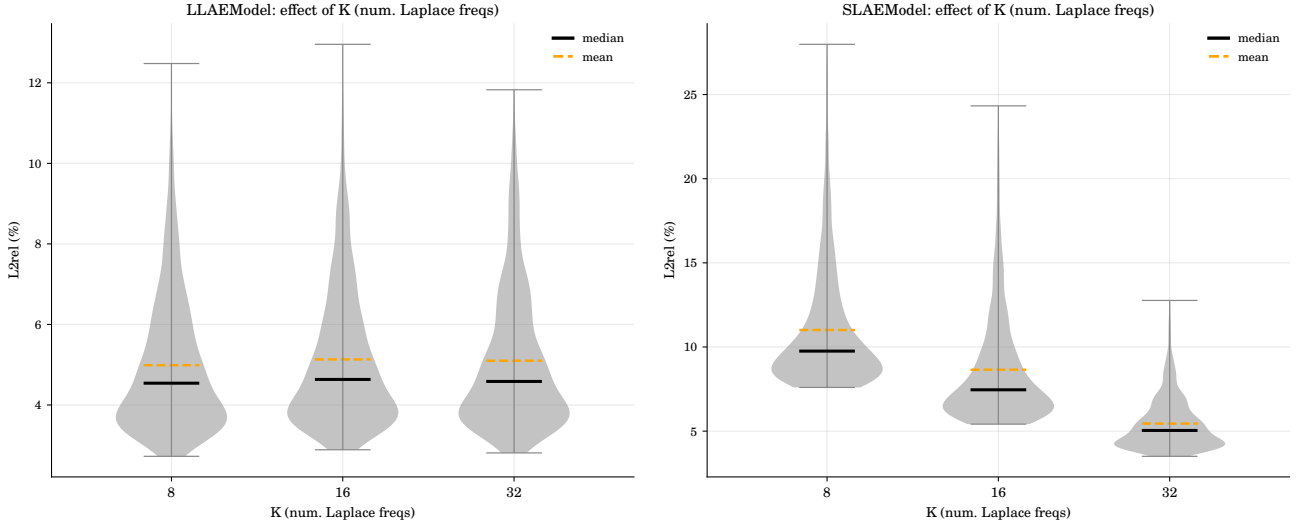


Figure 16: End-to-end error distributions under the optimised contour, as a function of the number of Laplace frequencies K ($d_z = 64$, test set, $n = 1620$). Left: LLAE, learnable contour. Right: SLAE, offline contour. Black line: median; dashed orange line: mean. The two families exhibit opposite dependences on K : the LLAE distributions are indistinguishable across a fourfold variation of K , whereas the SLAE distributions shift downward and contract markedly as K increases, the upper tail being reduced from $\approx 28\%$ at $K = 8$ to $\approx 13\%$ at $K = 32$.

retains a $1.6\times$ arithmetic and $1.5\times$ wall-clock advantage, on $2.6\times$ less memory. Contour optimisation thus displaces the SLAE accuracy–cost Pareto front strictly downward: the accuracy improves at every K while the cost, along all three axes, is unchanged.

Assessment of the two strategies. Taken together, the two procedures constitute a controlled comparison of **objectives** rather than of optimisers, and each delivers precisely what its objective encodes. The learnable contour is optimised against the reconstruction of exact coefficients; it yields the most accurate Laplace autoencoder of the study (2.61%, second only to the Laplace-free DL-ROM reconstruction of Table 3) and leaves the end-to-end error unchanged. The offline proxy is the only one of the two criteria evaluated at the magnitude of coefficient error the surrogate actually incurs, and it is the only one to improve the deployed pipeline — at every K , by up to 1.85 pp. Since both strategies displace the sampling points comparably (Figure 15), the discriminating ingredient is not the displacement itself but the regime in which (31) weighs it: a contour that sharpens the representation and a contour that keeps it learnable are not the same contour, and only the second governs what the emulator ultimately delivers.

This asymmetry is what makes the offline procedure the more consequential of the two, and two properties compound its practical value. It requires no network training whatsoever — 500 gradient steps on a purely linear-algebraic functional evaluated over 100 instances, against a complete two-phase training for every candidate contour were the selection to be made empirically — and it is free at inference, since it alters no tensor dimension. Its effect on the accuracy–cost trade-off is the principal result of this section: SLAE at $K = 32$ attains 5.04%, within 0.9 pp of the most accurate configuration of the entire study (LLAE at 4.13%) where a prescribed contour left a gap of 1.6 pp, while evaluating roughly $1.5\times$ faster; at $K = 16$ the advantage is $1.6\times$ in arithmetic and $1.5\times$ in wall-clock time, for 7.46%. The regularized inversion — poles and regularisation weights alike, fixed by convention in every Laplace-domain reduced-order model we are aware

of — thus proves to be a design object of first order, and one that can be optimised at negligible cost against the error that actually matters.

4 Comparison and Discussion

4.1 Global synthesis

The three references of this study isolate one ingredient each. POD in the Laplace domain (Section 3.2.1) keeps the spectral compression and removes the non-linearity; the DL-ROM (Section 3.2.2) keeps the non-linearity and removes the spectral compression; SLAE and LLAE keep both. Table 11 places them side by side at a common operating point — $d_z = 64$, $K = 16$ where applicable, one full 150-step trajectory per query — with accuracy and the four cost axes measured on the same hardware in a single benchmarking run.

Model	Contour	Targets	N_{surr}	Lat. (ms)	FLOPs (G)	Mem. (MB)	Median (%)
POD ($k_{\text{pod}} = 64$)	prescribed	16	—	10.1	0.2	212	8.91
DL-ROM	—	150	23.50 M	31.4	48.3	927	3.80
LLAE	prescribed	16	3.48 M	23.3	48.2	622	4.18
	learnable	16	3.48 M	24.2	48.2	621	4.63
SLAE	prescribed	16	3.22 M	15.7	30.3	235	7.59
	offline	16	3.22 M	15.7	30.3	235	7.46

Table 11: Synthesis at the common operating point ($d_z = 64$, $K = 16$, $\gamma = 0.01$ for the prescribed contours), test set ($n = 1620$), one 150-step trajectory per query. **Targets**: number of coefficient blocks the surrogate must predict (K Laplace frequencies, or N_t time steps for the DL-ROM). N_{surr} : surrogate parameters, equation (37). Latency, FLOPs and peak memory are measured at batch size 1 on an idle GPU, in a single benchmarking run shared by all rows and therefore mutually comparable. Best per column in bold, excluding accuracy. Median errors are those of Tables 2, 6 and 10.

Two orderings emerge, and they are not the same. On accuracy, the DL-ROM leads (3.80%), LLAE follows within half a point (4.18%), SLAE trails by a further three points (7.46–7.59%), and POD is last (8.91%). The non-linear autoencoder is what buys the bulk of the accuracy: every deep model improves on POD, by 15 to 57% at this operating point and by up to 43% for SLAE once its contour is optimised at $K = 32$ (Section 3.5). Temporal compression, by contrast, buys nothing in accuracy — it costs a little, 0.38 pp for LLAE and around 3.7 pp for SLAE, the price of representing a 150-step trajectory by 16 complex coefficients and inverting a truncated spectrum.

On cost the ordering reverses, and it does so unevenly across axes. Among the deep models SLAE is the cheapest on all four — 30.3 GFLOPs, 15.7 ms, 235 MB, 3.22 M parameters — and the DL-ROM the most expensive on all four. LLAE occupies an intermediate position that the two axes describe differently: it matches the DL-ROM’s arithmetic to 0.2% (48.2 against 48.3 GFLOPs, the shared decoder being essentially the whole computation) yet runs 1.35× faster (23.3 against 31.4 ms) on 1.5× less memory, because its surrogate carries 16 frequency heads instead of 150 temporal ones. The linear POD is cheaper than everything by a wide margin on arithmetic — 0.2 GFLOPs,

some $150\times$ below SLAE — but only $1.6\times$ faster in wall-clock time and no lighter in memory, the Tikhonov inversion being a floor that no reduction in network cost can go below. Figure 17 shows the corresponding Pareto view. No model dominates: the frontier at 150-step output is occupied by the DL-ROM (most accurate, most expensive), LLAE (near-equal accuracy at a seventh of the parameters and $1.35\times$ faster) and SLAE (twice as fast for three points of error), with POD dominated on accuracy by all three. Section 4.2 explains why the latency axis behaves differently from the parameter axis, and why this ranking is not invariant: it is stated at a fixed output resolution, and changing that resolution changes it.

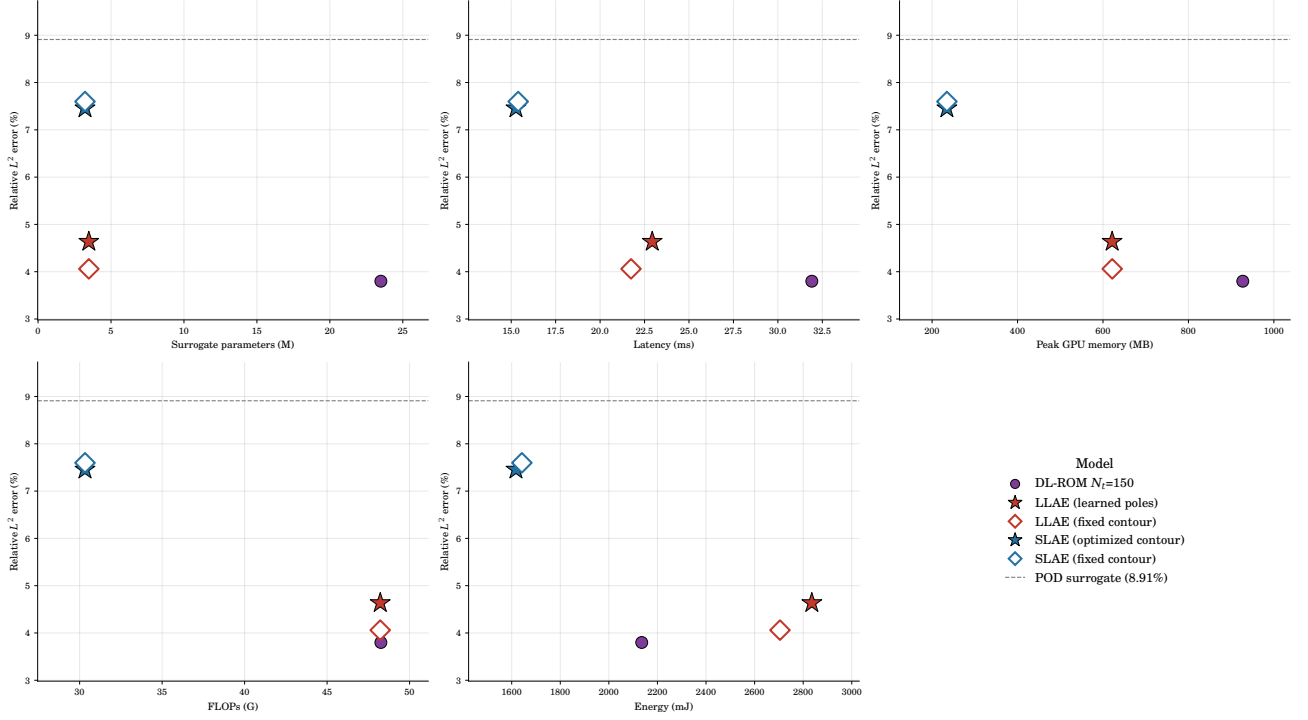


Figure 17: Accuracy against five inference-cost axes at fixed 150-step output (batch size 1): surrogate parameters, latency, peak GPU memory, FLOPs and energy. Bottom-left is best. Stars: optimised contours (LLAE learnable, SLAE offline); open diamonds: prescribed contours; green disc: DL-ROM at $N_t = 150$. The dashed line marks the best POD surrogate (8.91%).

4.2 Accuracy, cost, and output resolution

Where temporal compression reaches. Equation (37) predicts that the Laplace surrogates are $6.8\times$ lighter than the DL-ROM, and Table 11 confirms it exactly. It does **not** predict a $6.8\times$ speed-up, and none is observed: LLAE runs $1.35\times$ faster than the DL-ROM it replaces, and performs exactly as much arithmetic: normalised to the full DL-ROM, its parameter count falls to 0.15 while its FLOPs stay at 1.00.

The explanation is that the surrogate is not what the pipeline spends its time on — the decoder is, as already observed when comparing the direct, SVD and Tucker variants (Section 4.3). Both LLAE and the DL-ROM invoke the decoder once per output time step: 150 decoder passes each, on the same convolutional backbone. Compressing the time axis **in latent space** shrinks the quantity the MLP must regress, from 150 latent frames to 16 complex ones, but the inverse transform expands it back to 150 frames before decoding, so the dominant term of the forward pass is untouched. The

measurements bear this out with precision: LLAE and DL-ROM require 48.2 and 48.3 GFLOPs respectively, an agreement to 0.2% that identifies the shared decoder as essentially the entire computation. The residual differences on the remaining axes follow from the surrogate, and they favour LLAE: $1.5\times$ less peak memory (622 against 927 MB, the 20M parameters it does not allocate) and $1.35\times$ less wall-clock time (23.3 against 31.4 ms), the DL-ROM evaluating 150 per-time-step heads where LLAE evaluates 16 per-frequency ones. That the two differ in latency while agreeing in arithmetic is the clearest illustration of why we report both: at equal FLOPs, the model whose predictor is split into ten times fewer kernels is the faster one, and no operation count can express that. Energy tracks latency closely, at 3.01 and 3.01 J, and adds nothing the latency does not already say — the board draws near-constant power over these millisecond kernels, so energy is latency in other units.

SLAE is the configuration in which temporal compression does reach the decoder. Applying the transform in the physical domain leaves $K = 16$ complex frequency frames to decode instead of 150 temporal ones, so the whole forward pass, and not merely the surrogate, is $O(K)$: 30.3 GFLOPs against 48.3, a $1.6\times$ reduction in arithmetic that this time **does** convert into wall-clock time — $2.0\times$ lower latency (15.7 against 31.4 ms), $3.9\times$ lower peak memory (235 against 927 MB) and $1.7\times$ lower energy than the DL-ROM at identical 150-step output. The three points of accuracy this costs are the counterpart — the decoder never sees a physical time frame, and the trajectory is reconstructed by inverting a 16-point spectrum — and the offline contour of Section 3.5 recovers part of it, down to 5.04% at $K = 32$ while retaining a $1.6\times$ speed advantage.

The general principle is worth stating separately from the numbers, since it governs where temporal compression pays. A Laplace-domain pipeline is cheap on the axes that the compressed representation actually reaches. Compression in latent space (LLAE) reaches the surrogate only: it delivers the parameter and memory savings and leaves the arithmetic unchanged. Compression in the physical domain (SLAE) reaches the decoder as well, and converts into latency, FLOPs and energy. Reporting parameter counts alone — the natural reading of equation (37) — would have credited LLAE with a $6.8\times$ gain that no user would ever measure, an instance of the efficiency-misnomer effect documented by Dehghani et al. [4]; this is why Table 11 and Figure 17 report several axes jointly.

The cost of a fixed output grid. All the comparisons above are stated at a fixed output resolution of 150 steps, and that qualification is not cosmetic. The DL-ROM has exactly one lever for reducing its temporal cost: producing fewer time steps. Sweeping the stride κ over the training grid (Section 2.5.2) traces that lever, one trained model per point.

The sweep is informative in two ways. First, the DL-ROM’s per-step accuracy is flat: 3.73 to 3.97% across a tenfold reduction of N_t , with no monotone trend. Subsampling the time grid does not make the individual frames harder to predict — if anything the shorter latent trajectory is slightly easier — so the $9\times$ cost reduction from $\kappa = 1$ to $\kappa = 10$ is bought purely by delivering fewer steps. Second, the parameter-matched point falls at $\kappa = 10$: at 3.06M parameters the DL-ROM surrogate is the size of LLAE at $K = 16$ (3.48M), both compressing the time axis roughly tenfold — uniformly on one side, spectrally on the other. The difference is what comes out: 15 time steps for one, the complete 150-step trajectory for the other, at 4.18% median error. Recovering the missing 135 steps from the DL-ROM requires interpolating between samples 10 time units apart, or retraining at a finer stride.

Model	κ	Output steps	N_{surr}	Lat. (ms)	FLOPs (G)	Median (%)
DL-ROM	1	150	23.50 M	32.9	48.3	3.80
DL-ROM	2	75	12.15 M	16.6	24.1	3.97
DL-ROM	3	50	8.36 M	11.3	16.1	3.84
DL-ROM	5	30	5.33 M	7.0	9.7	3.86
DL-ROM	10	15	3.06 M	3.9	4.8	3.73
LLAE ($K = 16$)	—	any	3.48 M	Fig. 18	4.8–48.2	4.18
SLAE ($K = 16$)	—	any	3.22 M	Fig. 18	30.2–30.3	7.59

Table 12: Temporal-resolution sweep of the DL-ROM against the two Laplace pipelines. Each DL-ROM row is a separately trained model; the two Laplace rows are a **single** checkpoint each, queried at output resolutions $N_t \in \{15, 30, 50, 75, 150\}$, the FLOPs range spanning that interval and the corresponding latency profile being given in Figure 18. Errors are measured on each model’s own output grid — the DL-ROM at stride κ against the ground truth subsampled at the same stride — with no interpolation, so they are comparable in accuracy per delivered step but not in the number of steps delivered. The two Laplace medians are those measured at the 150-step output; the **cost** of querying a single Laplace checkpoint at coarser grids is measured (Figure 18), its **accuracy** at those grids is not, and we return to this in Section 4.6.

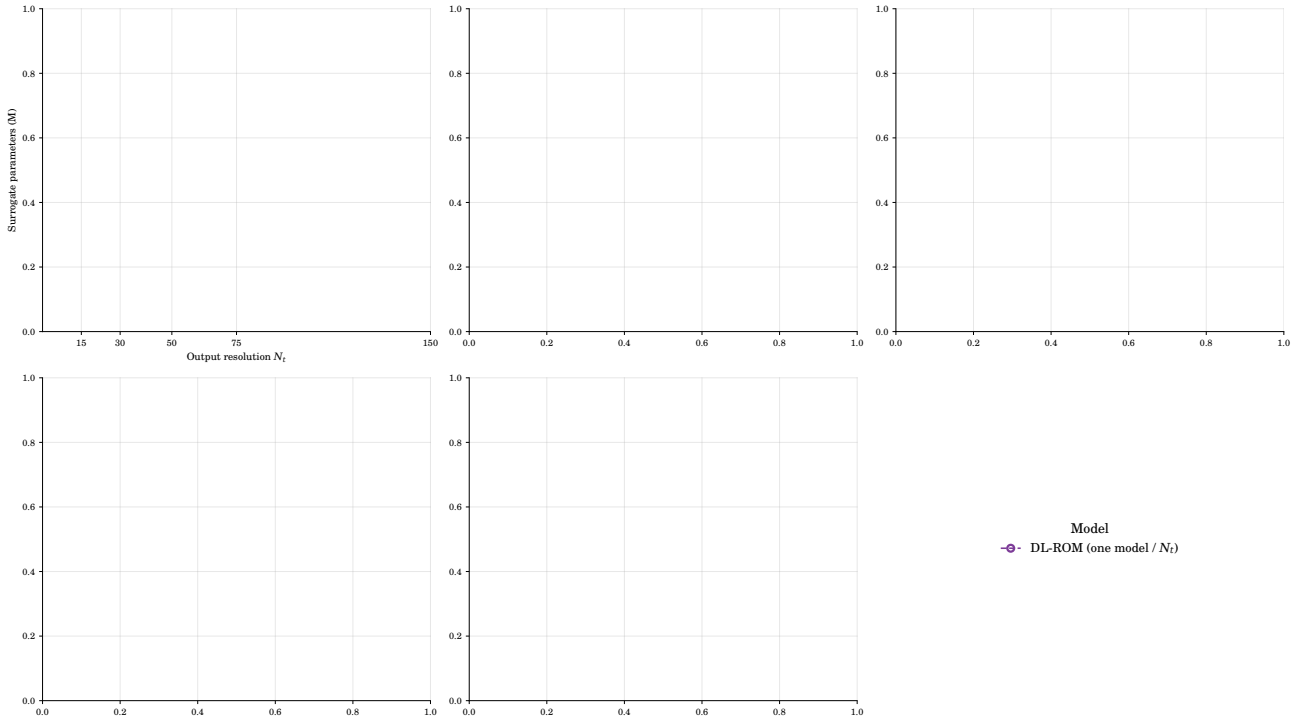


Figure 18: Inference cost against requested output resolution N_t at fixed horizon T (batch size 1). Solid lines: a **single** Laplace checkpoint per curve, queried at each N_t . Dashed line with open circles: the DL-ROM, one separately trained model per point, each evaluable only at its own N_t . SLAE is flat on parameters, FLOPs, latency and memory; LLAE tracks the DL-ROM on FLOPs (both decode N_t frames) at a constant surrogate size.

One model, any Δt . That asymmetry is structural rather than incidental, and it is the second consequence of compressing time spectrally. The Laplace coefficients $\hat{z}(s_k)$ predicted by the surrogate carry no reference to any time grid; the trajectory is materialised by the Tikhonov solve (3)–(4), whose matrix $\mathbf{F}_{k,n} = \Delta t w_n e^{-s_k t_n}$ is assembled from whatever nodes t_n are requested. A single trained model therefore answers at any output resolution — any Δt at fixed horizon T — for the cost of reassembling a small $K \times N_t$ matrix, whereas the per-time-step heads of the DL-ROM surrogate (equation (37)) are tied to the grid they were trained on.¹ Figure 18 traces the resulting cost profiles. It reports cost; accuracy across the same sweep is measured below, and it does not follow the intuition that a coarser grid asks strictly less of the same spectrum. LLAE keeps a constant surrogate (3.48M parameters at every N_t) but decodes N_t frames, so its arithmetic follows the DL-ROM curve exactly, 4.8 to 48.2 GFLOPs; SLAE decodes $K = 16$ frames whatever is asked of it and is flat on every axis, 30.2–30.3 GFLOPs from $N_t = 15$ to $N_t = 150$. Below $N_t \approx 100$ the DL-ROM and LLAE are cheaper in FLOPs than SLAE, above it SLAE wins and its margin grows without bound: refining the output grid is free for SLAE, and the crossover is a property of the decoder, not of the transform.

This is where temporal Laplace compression is decisive rather than merely economical. The five DL-ROM rows of Table 12 are five trainings, five checkpoints and five fixed answers to a question — at what temporal resolution is the trajectory needed? — that is often settled after the model is deployed, and differently for each downstream use. The two Laplace rows are one training each, answering the entire range at inference time. For a many-query setting in which a coarse scan precedes a refined analysis of a few selected parameter values, that flexibility, and not the parameter count, is the operative advantage.

4.3 Is latent compression worth it?

Sections 3.4.1–3.4.3 evaluated three surrogate strategies — direct, single-mode SVD, and joint-mode Tucker — that differ only in how the latent prediction target is compressed. Do the compressed variants gain anything? On inference cost, the answer is no, and the reason is structural: the three pipelines share the same decoder, and it is the decoder — reconstructing the K Laplace frames (SLAE) or all N_t time frames (LLAE) — that dominates inference. SVD and Tucker change only the width of the MLP output head and insert a small linear back-projection ($V^\top$, or the two Tucker factors) before decoding; neither touches the decoder workload. Latency is therefore effectively invariant to the compression strategy: within each family the three pipelines fall in the same band (SLAE ~ 7 –9 ms, LLAE ~ 18 –22 ms at batch size 1), the SVD variant being marginally the slowest because of its per-frequency back-projection. Table 13 places accuracy, target dimension and latency side by side at the shared AE configuration.

The verdict is clear. Relative to the direct surrogate, neither SVD nor Tucker improves accuracy (each adds a fraction of a point) nor latency (the pipeline is decoder-bound). What they **do** buy is a smaller, simpler prediction target — Tucker down to 256 coefficients, a 4 $\times$ reduction over the direct $Kd_z = 1024$. That compactness only pays off when the surrogate, not the decoder, is the binding constraint: a tight training-data budget, storage or transmission of the predicted coefficients, or downstream analysis of the compact Tucker core. For the present problem, where the decoder governs both cost

¹A DL-ROM whose surrogate takes t as an input coordinate rather than through indexed heads, as in [6], is queryable off-grid; its **cost** nonetheless remains proportional to the number of requested steps, one network evaluation per step, which is the property at issue here.

Family	Strategy	Target (coeff.)	Median (%)	Latency (ms)
LLAE	Direct	1024	4.18	18.3
	SVD ($k_{\text{svd}} = 32$)	512	4.24	~22
	Tucker ($r_s=8, r_z=32$)	256	4.33	18.2
SLAE	Direct	1024	7.59	8.4
	SVD ($k_{\text{svd}} = 32$)	512	7.90	~9
	Tucker ($r_s=8, r_z=32$)	256	7.92	7.4

Table 13: Direct vs. SVD vs. Tucker at the shared autoencoder configuration ($d_z = 64$, $K = 16$, $\gamma = 0.01$). **Target (coeff.)**: number of latent coefficients the surrogate predicts per sample (Kd_z , Kk_{svd} , $r_s r_z$ respectively; complex-valued for LLAE, real for SLAE). Median relative L^2 error on the test set ($n = 1620$); latency at batch size 1. Compression shrinks the target by up to $4\times$ but leaves accuracy and latency essentially unchanged.

and error, the **direct** surrogate is the pragmatic choice; the value of SVD and Tucker is chiefly methodological, showing that most of the latent’s predictive content survives aggressive rank reduction.

4.4 Making the regularized inversion an optimisation variable

In every Laplace-domain reduced-order model we are aware of, the inversion is fixed by convention in its entirety — poles at the leading Fourier modes of the time grid, a uniform abscissa chosen small, regularisation weights set a priori — and never revisited, for want of any statement of what a given choice costs. The bound (25) supplies that statement, and it supplies it for the whole triplet at once. Everything the inversion does to the reconstructed trajectory is carried by two scalars, the regularisation bias $\|A^{-1}L_{\text{reg}}u\|$ and the amplification factor $\|A^{-1}F^H\|$ — a condition number for the inversion — which multiply respectively one and the coefficient error ϵ , and which depend on (s_k, α_t, λ) **alone**. That last property is what turns a convention into an optimisation variable: a candidate triplet can be scored without training anything, since neither scalar involves the network that will eventually produce the coefficients. It is worth noting that the poles enter these two scalars on exactly the same footing as α_t and λ — the bound draws no distinction between placing a sampling point and weighting a penalty — which is the formal counterpart of the reading announced in Section 2.4: what is being designed is the inversion, not the contour.

The procedure that follows inherits this cheapness in full. Comparing two triplets reduces to comparing two affine functions of ϵ , and selecting one reduces to minimising the proxy (31) built from the bound: 500 gradient steps on a purely linear-algebraic functional evaluated over 100 instances — seconds of computation, no network in the loop — where an empirical selection would require a complete two-phase training per candidate. It is equally free at deployment, altering no tensor dimension, so the optimised and prescribed rows of each family in Table 11 are indistinguishable on all four cost axes. Accuracy is thus obtained here without spending anything on either side of the pipeline, which is rare enough among the levers examined in this work to be worth stating plainly: latent compression (Section 4.3) bought a smaller target and no accuracy; the inversion, tuned as a whole, buys accuracy and costs nothing.

And it delivers, in the manner the bound predicts rather than in spite of it. The offline triplet improves the end-to-end error at every K , by up to 1.85 pp, **while degrading** reconstruction by as much as 7.13 pp — a sign pattern that would be paradoxical without (25) and is a direct consequence of it: the criterion is calibrated at the coefficient error the surrogate actually incurs, and at that magnitude it is worth buying conditioning with bias. The Gap column of Table 10, negative throughout and reaching -8.59 pp at $K = 16$, measures the reconstruction penalty being handed back once the surrogate is trained. The outcome is the best SLAE result of the study, 5.04% at $K = 32$ — a 43% reduction on the best POD surrogate, within 0.9 pp of the most accurate configuration we trained while evaluating $1.6\times$ faster — and 7.46% at $K = 16$ for a $2.9\times$ speed advantage. Optimising the inversion displaces the SLAE accuracy–cost front strictly downward, and it is on that family, the one whose compression reaches the decoder and converts into latency and energy (Section 4.2), that the recovery is most useful.

The learnable contour, which moves the sampling points comparably (Figure 15) but is scored on reconstruction, is the control that isolates the criterion from the displacement. It attains the most accurate Laplace autoencoder of the study, 2.61%, and leaves the deployed error flat at 4.54–4.63%. Our reading is that the representation it selects is simply harder to learn: the widened spectral window populates the latent trajectories with faster-varying components, so the map $\mu \mapsto \hat{z}(s_k)$ that the surrogate must regress becomes less regular, and the fidelity gained upstream is spent in full on the harder regression — with, plausibly, some degree of specialisation to the reconstruction task itself, an overfitting of the objective rather than of the data, since held-out reconstruction does improve (Appendix D). Whichever share belongs to each, the lesson is that an inversion must be judged on the deployed error and not on the error floor it permits, and this is exactly what the ϵ of the bound formalises.

Three reservations temper the result. The proxy is calibrated at an assumed magnitude of surrogate error, known only after the fact; we used one setting throughout and did not characterise the sensitivity of the selected triplet to it. The learnability reading of the previous paragraph is an interpretation, not a measurement: establishing it would require characterising the regularity of $\mu \mapsto \hat{z}(s_k)$ under each inversion independently of the surrogate trained on it. And, as stated in Section 2.4, the gain is credited to the triplet and not apportioned within it: our reference leaves (α_t, λ) at untuned values, so part of what we measure may be the fitting of the regularisation rather than the displacement of the poles. None of the three touches the central point — that the regularized inversion is an optimisable object, that (25) says at what price, and that acting on it is the cheapest accuracy in this study.

4.5 Positioning with respect to related work

Our pipeline reuses well-established components and combines them along a new axis. The **spatial** compressor is a convolutional encoder–decoder applied directly to the field, i.e. the deep convolutional-autoencoder ROM of Lee and Carlberg [19] and of the original DL-ROM [6]; we deliberately omit the preliminary POD reduction of POD-DL-ROM [7], which is an efficiency device (it lowers the autoencoder’s training cost) rather than an accuracy device, and is unnecessary for a convolutional network operating on a structured grid. The **linear** baseline against which we measure the nonlinear gain is a POD basis followed by a parameter-to-coefficient network, that is, the non-intrusive POD-NN of Hesthaven and Ubbiali [15], here applied frequency-by-frequency in the Laplace domain. Our optional SVD and Tucker stages (Sections 2.3.2–2.3.3) also use

linear/multilinear projections, but in a role opposite to POD-DL-ROM: they act **after** the nonlinear autoencoder, in the latent space, to shrink the surrogate target — a post-compression, not a spatial pre-reduction.

What distinguishes our approach is not the spatial compressor but the treatment of **time**. All the DL-ROMs above keep the full temporal resolution of the trajectory: the reduced dynamics are queried once per time step through a (μ, t) -conditioned network [6, 7] or propagated sequentially in time [26], so the temporal work scales with N_t . We instead compress the time axis itself with the Laplace transform, predicting $K \ll N_t$ spectral coefficients in a single shot and recovering the full trajectory by a cheap linear inversion. This exploits the same spectral intuition as Laplace-domain linear ROMs [29, 10] and the Laplace Neural Operator [2], but transposes it into the **nonlinear, parameter-to-field** DL-ROM setting, where — to our knowledge — temporal Laplace compression has not previously been used. The Laplace stage is therefore orthogonal to the spatial-compression choices above and could in principle be layered on top of any of them, including POD-DL-ROM.

Relation to neural operators. The learnable-contour variant of LLAE (Section 2.4) deserves a closer comparison with operator-learning methods, which it resembles along two axes. First, it applies its spectral transform **inside a learned low-dimensional latent space** rather than on the physical field, exactly the design principle of the Latent Neural Operator [30]: encode to a compact latent, act there, decode back. Second, and more specifically, learning the complex poles $s_k = \sigma_k + i\omega_k$ end-to-end is a latent-space counterpart of the learnable pole–residue representation at the heart of the Laplace Neural Operator [2]; in both cases the temporal (or trajectory) dynamics are captured by a small set of trainable poles in the complex plane rather than by a fixed spectral grid or by explicit time-stepping. The surrogate itself reinforces the analogy: its shared trunk conditioned on a sinusoidal encoding of the frequency s_k (Figure 2) maps $(\mu, s_k) \mapsto \hat{z}(s_k)$, so the Laplace frequency plays the role of the output coordinate much as the trunk coordinate does in a DeepONet [21]. The essential difference is one of **setting**, not of mechanism: genuine neural operators are field-to-field maps between function spaces, discretization-invariant in their input, whereas our input is a finite parameter vector μ and our spatial decoder is a fixed-resolution convolutional network. LLAE with a learnable contour is thus best understood as a **parameter-conditioned, latent Laplace operator** — importing the learnable-pole and latent-operator ideas from [2, 30] into the DL-ROM regime, rather than as a neural operator in the strict field-to-field sense.

4.6 Discussion and perspectives

What the ablations attribute to what. The two references were chosen to remove one ingredient each, and read together they place the credit unambiguously. Removing the non-linearity costs a great deal: POD in the Laplace domain sits at 8.91% against 3.80–4.63% for every deep model, and no amount of additional POD modes closes the gap — $k_{\text{pod}} = 128$ improves the reconstruction floor to 7.04% and **degrades** the deployed error to 11.29%, the regression problem growing faster than the basis improves. Removing the Laplace transform, by contrast, costs nothing in accuracy and in fact gains a little: the DL-ROM is the most accurate emulator of the study. This is the honest reading of our own experiment, and it is the expected one — a surrogate that predicts 150 latent frames faces a strictly easier approximation problem than one constrained to 16 complex coefficients, and any spectral truncation can only remove information the time-domain

model retains.

Temporal compression must therefore be justified on cost and on flexibility, not on accuracy, and this is where the study’s substantive findings lie. The first is that **where** the transform is applied decides which cost axes benefit, and that the parameter count — the quantity equation (37) predicts and the one a paper is tempted to report — is the axis least representative of what a user measures. LLAE is $6.8\times$ lighter than the DL-ROM in surrogate parameters and $1.5\times$ lighter in memory, yet performs exactly as much arithmetic — the two agree to 0.2% in FLOPs — because both decode N_t frames through the same backbone. Its $1.35\times$ latency advantage comes not from the compression but from evaluating 16 prediction heads instead of 150. SLAE, whose decoder sees only the K frequency frames, is the configuration in which the compression reaches the arithmetic, and it converts into $2.0\times$ latency and $3.9\times$ memory at identical output. A Laplace pipeline is cheap on exactly the axes its compressed representation reaches, and no further.

The second is that the regularized inversion — the poles **and** the regularisation weights (α_t, λ) that go with them, fixed by convention in every Laplace-domain ROM we are aware of — is a design object of first order that can be optimised for free, and one that must be scored on the deployed error rather than on the reconstruction floor. The two strategies we compare displace the sampling points comparably but are calibrated at different magnitudes of coefficient error, and only the one calibrated at the surrogate’s magnitude improves the emulator: the offline contour gains up to 1.85 pp end-to-end **while degrading** reconstruction by up to 7.13 pp, whereas the learnable contour produces the best Laplace autoencoder of the study (2.61%) and leaves the deployed error flat. Among all the levers examined here this is the only one that buys accuracy at no cost on either side of the pipeline.

The third is a negative result worth stating as such: compressing the latent target before the surrogate — by SVD along one mode or by Tucker along two — buys neither accuracy nor latency. It shrinks the prediction target by up to $4\times$ for a fraction of a point, which is useful only when the surrogate rather than the decoder is the binding constraint. Its value here is diagnostic: that a rank-4 basis over the frequency mode is as good as a rank-8 one says that the $K = 16$ Laplace coefficients are already close to their useful floor, evidence arriving from a tensor factorisation fitted blind to the physics and independent of the argument that motivated the transform.

Limitations. Several qualifications bound the above, and we state them in decreasing order of consequence.

Cost measurements. Every cost figure in this paper was collected on the same idle GPU (Appendix F) with a single instrumentation path — CUDA Events for latency, a counter over the executed graph for FLOPs, and the allocator’s high-water mark for memory — so the numbers are mutually comparable across sections. Two caveats nonetheless bound what should be read into them. First, wall-clock latency measures an implementation as much as an architecture, and our two reconstruction pipelines are not written with equal care: the SLAE path evaluates its Laplace transform pair in double precision on the host, which the LLAE path avoids by performing its transform inside the model, and this alone accounts for the sign disagreement between the FLOP and latency columns of Table 5. Single precision would remove the gap but makes the Tikhonov inversion diverge at the ridge values of the prescribed contours, so the comparison is left as measured rather than repaired by a change that would compromise the numerics. The surrogate

stage, which applies no forward transform, is free of this asymmetry, and it is the stage the conclusions rest on. Second, energy is reported for completeness but carries little independent information at this scale: the board draws near-constant power over kernels of a few milliseconds, so the energy column is proportional to latency to within a few percent and should not be read as a separate axis.

Structure of the design. As set out in Section 3.1, the parameter design is full factorial — 225 points in (k, C) crossed with 36 injection angles — and partitioned uniformly over the product grid. This is a statement about the shape of the design, not about the validity of the instances, each of which is an exact solution at its own parameter value. Its consequence is that the test error measures interpolation at held-out points of a dense design rather than extrapolation to a (k, C) pair outside it, and that the aggregate is weighted towards the ϕ direction, which the factorial resolves 36 ways against 225 for the (k, C) plane. A per-direction error breakdown, or a second partition holding out whole (k, C) pairs, would separate the two; both are cheap and neither was done here.

Statistical resolution. Every number in this paper comes from a single training run per configuration. We report medians over 1,620 test instances, which makes the per-configuration error well resolved, but we have no estimate of run-to-run variability under reseeding. Conclusions resting on differences of a few tenths of a point — notably that SVD and Tucker are marginally worse than the direct surrogate (+0.06 and +0.15 pp for LLAE) — are consequently not resolved by our evidence, and should be read as “no measurable gain” rather than as a measured penalty. The conclusions resting on differences of a point or more, which are all the ones stated above, are unaffected.

Confounds in the SLAE/LLAE comparison. The two families do not share a Phase-1 objective: LLAE is supervised on the physical-time reconstruction, SLAE only in the Laplace domain, by design, since a physical-time loss would require N_t decoder passes and negate SLAE’s advantage. Their batch sizes differ for the same reason. The comparison is therefore between two complete design points, not between two placements of the transform holding everything else fixed, and the gap we measure conflates the two. That the SLAE Gap is negative in every configuration — the surrogate phase, which finally introduces a physical-time term, systematically improving on the standalone autoencoder, by up to 8.59 pp under an optimised contour — indicates that a substantial share of SLAE’s deficit is objective mismatch rather than architectural limitation. The control we did not run is the informative one: an SLAE trained in Phase 1 with a physical-time term on a subsampled set of frames, which would recover most of the supervision at a fraction of the cost.

Choice of DL-ROM formulation. Our baseline gives the DL-ROM surrogate one head per time step, which is what makes it 23.5M parameters against 3.48M for LLAE. A DL-ROM whose surrogate takes t as an input coordinate, as in [6], would be far smaller in parameters while paying one network evaluation per requested step. The $6.8\times$ parameter advantage is thus specific to the head-indexed formulation and should not be read as a general property of temporal compression; the FLOPs, latency and memory comparisons, which are decoder-dominated, are insensitive to this choice and are the ones to rely on.

Unmeasured resolution behaviour. The cost of querying a single Laplace checkpoint across output resolutions is measured; its accuracy across them is not (Section 4.2). The flexibility claim is therefore one of availability at known cost.

Scope. All results concern one two-dimensional convection–diffusion–reaction problem with $p = 3$ parameters and a single output field, on a structured grid, over a fixed

horizon. The proxy (31) is moreover calibrated at an assumed magnitude of surrogate error through λ_{AE} , held at one value throughout; the sensitivity of the selected triplet to that setting is not characterised, any more than is the share of the gain attributable to the poles rather than to (α_t, λ) , which our untuned reference does not separate.

Perspectives. The most direct continuations follow from the limitations. Re-measuring all timings uniformly and repartitioning at the level of the underlying simulations are prerequisites to any quantitative claim beyond the orderings established here. Beyond that, three directions seem to us the most promising.

The first is to transport the offline criterion to the family that currently lacks it. LLAE has learnable poles but scores them on reconstruction, which we show to be the wrong objective; the proxy (31) is the right one but is currently applied only to SLAE. A learnable contour trained **through** a surrogate-error term — or simply initialised at the offline optimum — would test whether the contour’s benefit is intrinsic to the criterion, as our reading holds, or specific to the SLAE pipeline. Relatedly, the interpretation we offer for the learnable contour’s failure to transfer, namely that a widened spectral window makes $\mu \mapsto \hat{z}(s_k)$ less regular, is stated as a hypothesis; measuring that regularity independently of the surrogate would settle it.

The second is to exploit the resolution-freedom the representation already possesses. Because the trajectory is materialised by a linear solve on requested nodes, non-uniform time grids — dense during the transient, sparse in the tail — come at no additional cost and are exactly what a many-query workflow wants. Whether the same coefficients support extrapolation beyond the training horizon T , where the decay axis γ of the Laplace transform has a natural interpretation that Fourier lacks, is the more ambitious version of the same question and the one that would most distinguish the transform from a truncated DFT.

The third is compositional. The Laplace stage acts on time and is orthogonal to every spatial-compression choice in the DL-ROM literature, so it could be layered onto POD-DL-ROM [7] or onto a parameter-conditioned U-Net [16] without modification, and the SLAE variant — the one whose compression reaches the decoder — is where such a combination would pay most. Scaling to three-dimensional fields, to several coupled species, and to higher-dimensional parameter spaces is the necessary test of whether the accuracy cost of spectral truncation, 0.4 points here for LLAE, remains as modest when the transient structure is richer.

5 Conclusion

We have introduced the Laplace transform as a temporal compression stage within the DL-ROM framework, predicting $K \ll N_t$ complex spectral coefficients from a parameter vector and recovering the full trajectory through a differentiable Tikhonov inversion, and we have studied two instantiations differing only in where the transform is applied: in the physical domain (SLAE) or in the learned latent space (LLAE). Against a linear POD surrogate in the same Laplace domain and a Laplace-free DL-ROM ablation, on a parametric transient combustion problem with $N_t = 150$, LLAE reaches 4.13% median relative L^2 error and SLAE 5.04% with an optimised contour, against 8.91% for POD and 3.80% for the DL-ROM.

The conclusion we draw from this is deliberately not that temporal compression improves

accuracy — it does not, and our own DL-ROM ablation is the evidence. It is that it changes what a fixed accuracy costs, and that the benefit is confined to the axes the compressed representation actually reaches: applied in latent space it shrinks the surrogate alone, leaving the arithmetic of the forward pass untouched; applied in the physical domain it reaches the decoder and converts into $2.0\times$ latency and $3.9\times$ memory at identical output. It also changes what a trained model can be asked for: because the predicted coefficients refer to no time grid, one checkpoint answers at any output resolution, where the DL-ROM required one training per grid across our sweep.

The second contribution is to treat the regularized inversion — the poles and the regularisation weights jointly, not the contour alone — as an object to be designed rather than fixed by convention. The bias–amplification bound (25) shows that everything it does to the reconstructed trajectory passes through two scalars that depend on the triplet (s_k, α_t, λ) and on nothing else, which makes a candidate triplet scorable without training any network. Acting on it improves the deployed error at every K , by up to 1.85 pp, for a few hundred gradient steps on a purely linear-algebraic functional and at no inference cost — while, instructively, degrading the reconstruction floor by as much as 7.13 pp. That sign reversal is the methodological point we would most like to carry beyond this paper: an inversion, and more generally a representation, must be judged at the magnitude of error the deployed model actually incurs, not at the floor it permits. The same criterion applied to reconstruction alone selects the best autoencoder of our study and improves nothing that is delivered.

Appendices

A Per-Checkpoint Error Distributions

POD reconstruction. Figure 19 shows the distribution of relative L^2 errors for each of the 7 POD reconstruction configurations evaluated in Section 3.2.1.

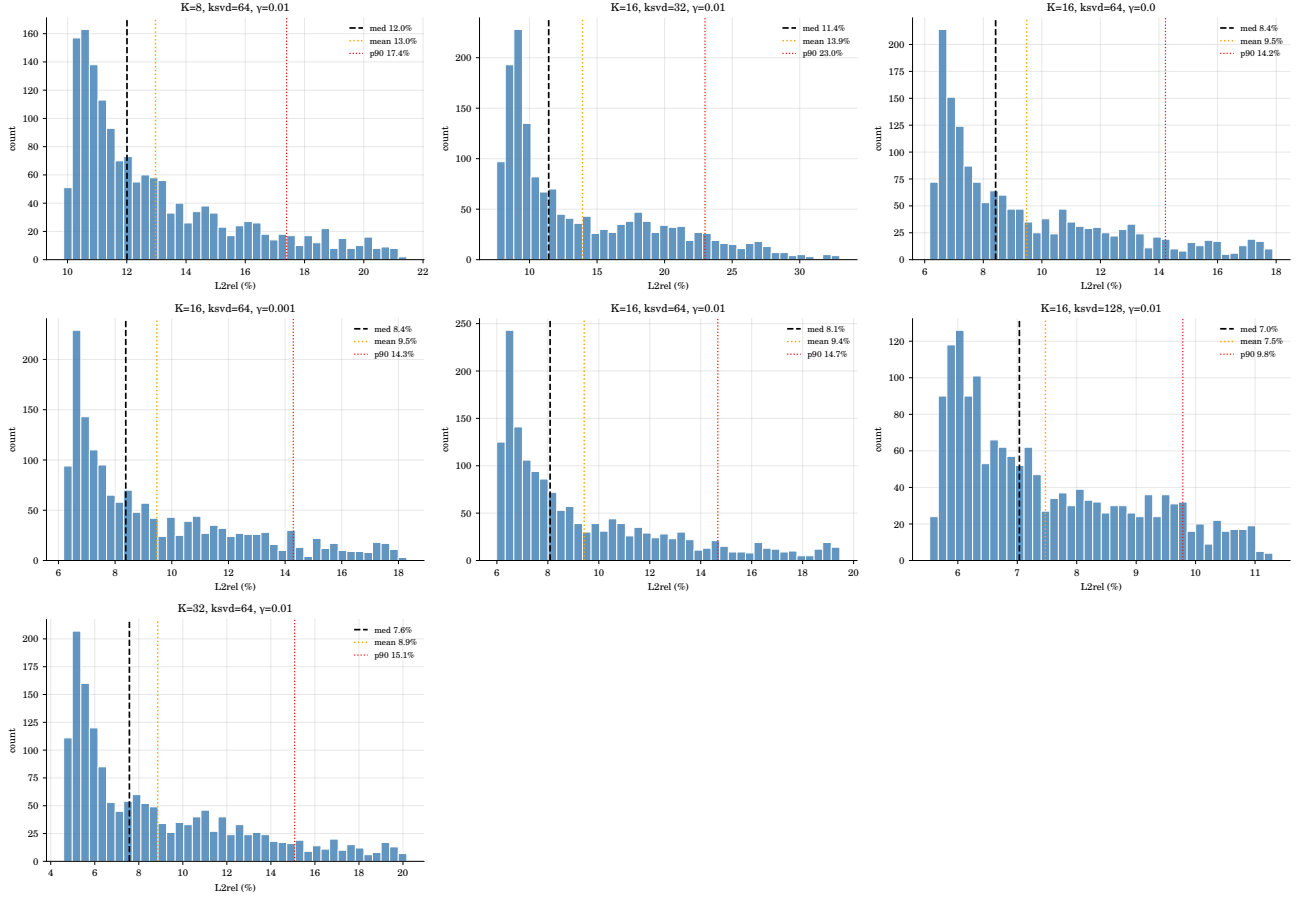


Figure 19: Per-configuration error histograms for the POD reconstruction (test set, $n = 1620$). The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

POD surrogate. Figure 20 shows the corresponding distributions for the end-to-end POD surrogate (Section 3.2.1).

AE checkpoints. Figure 21 shows the distribution of relative L^2 errors across the 1620 test simulations for each of the 14 AE checkpoints evaluated in Section 3.3. Dashed lines indicate the median (black) and mean (orange); the dotted red line marks the 90th percentile.

Surrogate checkpoints. Figure 22 shows the per-sample error distribution for each of the 14 direct surrogate checkpoints evaluated in Section 3.4.1.

DL-ROM baseline. Figure 23 shows the corresponding distribution for the Laplace-free DL-ROM baseline of Section 3.2.2.

Tucker surrogate checkpoints. Figure 24 shows the per-sample error distribution for each of the 12 Tucker surrogate checkpoints evaluated in Section 3.4.3.

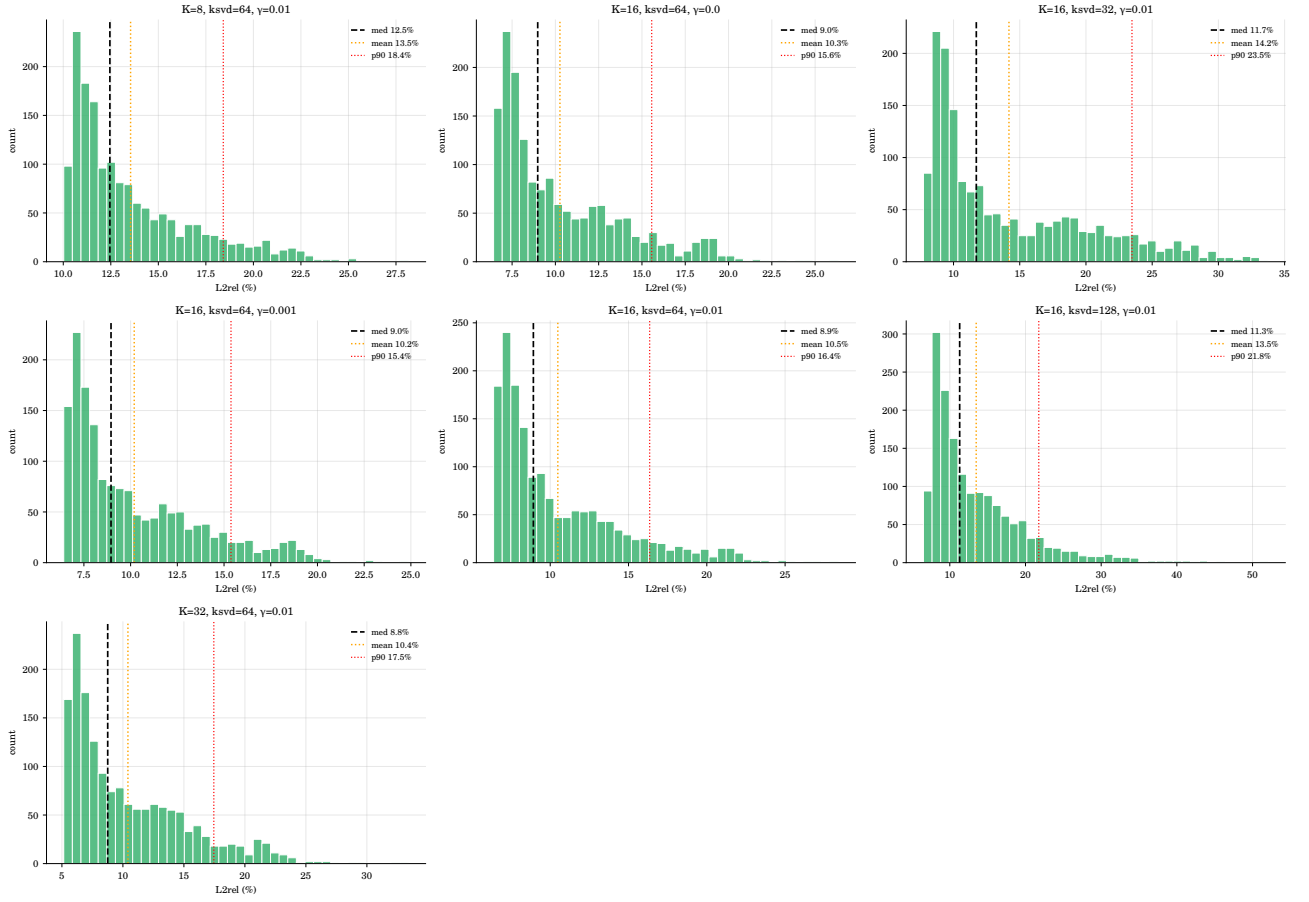


Figure 20: Per-configuration error histograms for the POD surrogate (test set, $n = 1620$). Same layout as Figure 19; green bars indicate surrogate predictions.

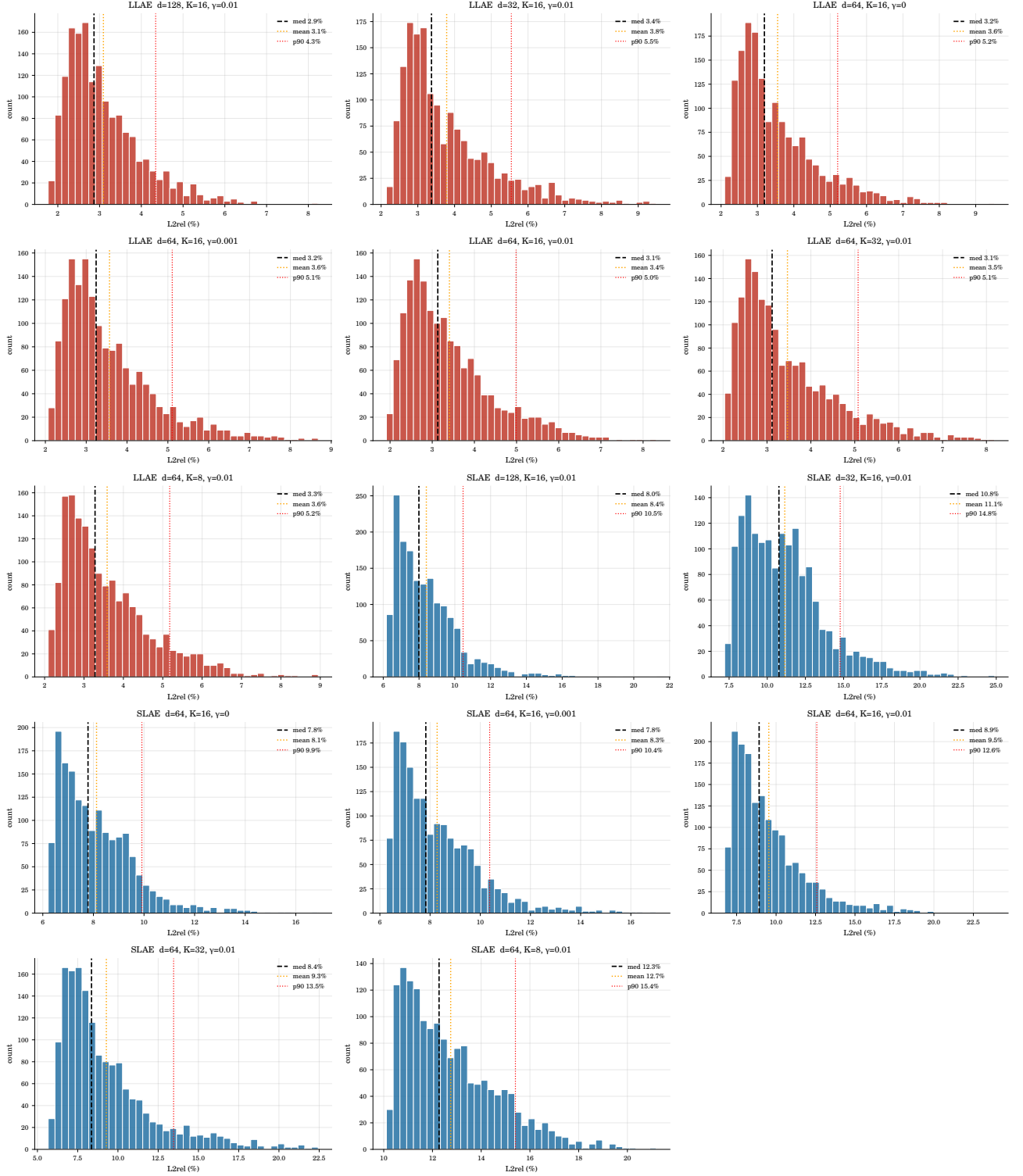


Figure 21: Per-checkpoint error histograms on the test set ($n = 1620$). Each panel corresponds to one checkpoint; SLAE in blue, LLAE in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

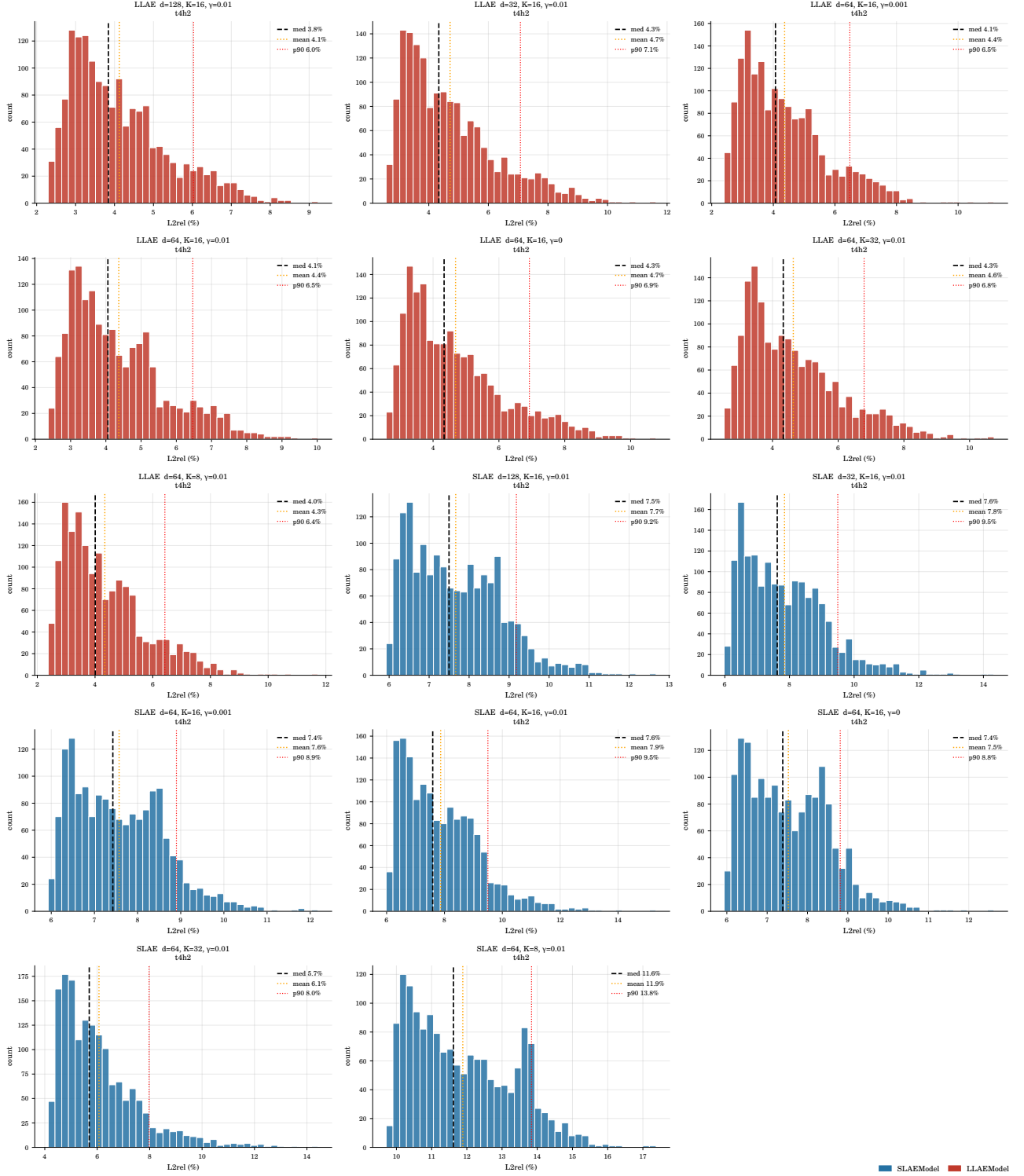


Figure 22: Per-checkpoint error histograms for all 14 direct surrogate models (test set, $n = 1620$). SLAE checkpoints in blue, LLAE checkpoints in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

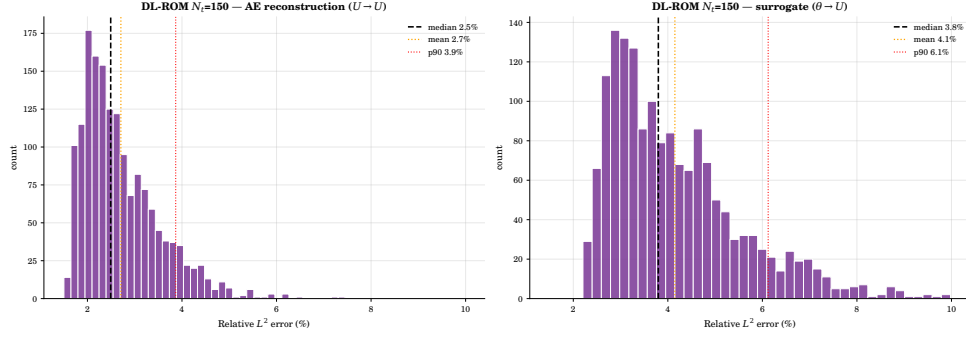


Figure 23: DL-ROM baseline at full temporal resolution ($N_t = 150$, $d_z = 64$): distribution of the relative L^2 error on the physical field $U(t)$ over the test set ($n = 1620$). Median 3.80%, mean 4.15%, p90 6.12%.

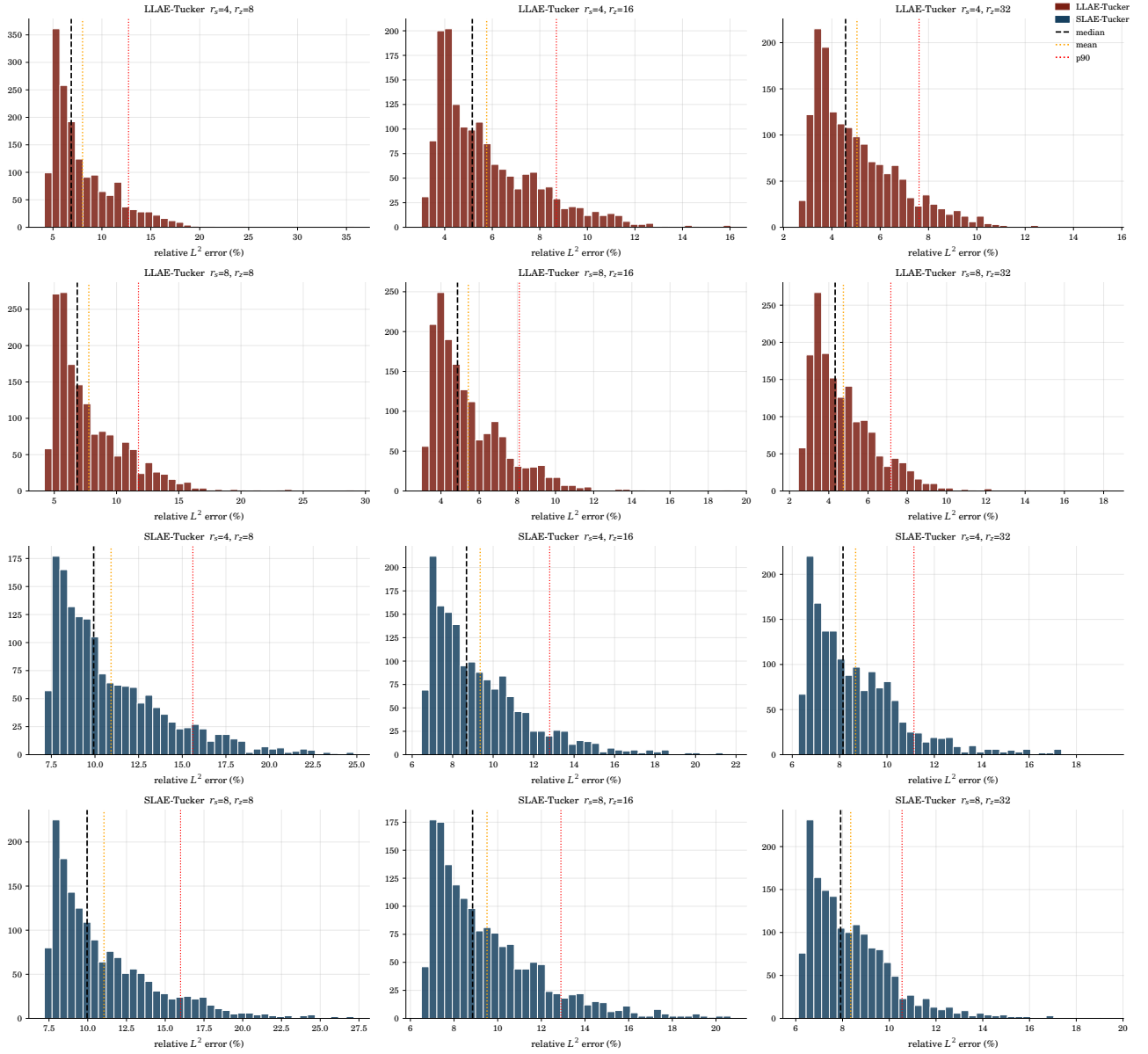


Figure 24: Per-checkpoint error histograms for all 12 Tucker surrogate models (test set, $n = 1620$). SLAE-Tucker in blue, LLAE-Tucker in red. The dashed black line is the median, dashed orange the mean, dotted red the 90th percentile.

B POD Baseline: Full Ablations

This appendix collects the complete hyperparameter sweeps behind Section 3.2.1, for both the pure POD reconstruction and the end-to-end POD surrogate.

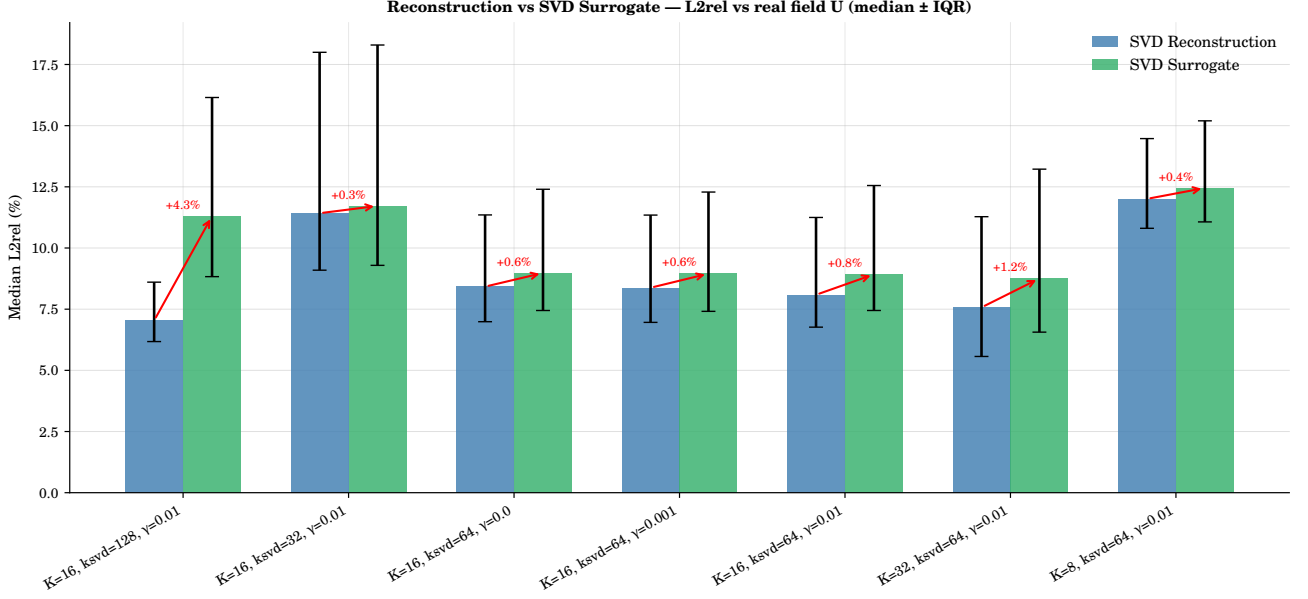


Figure 25: POD reconstruction (blue) vs. end-to-end surrogate (green): median relative L^2 error for all configurations of Table 2 (median $\pm$ IQR). Arrows indicate the surrogate overhead.

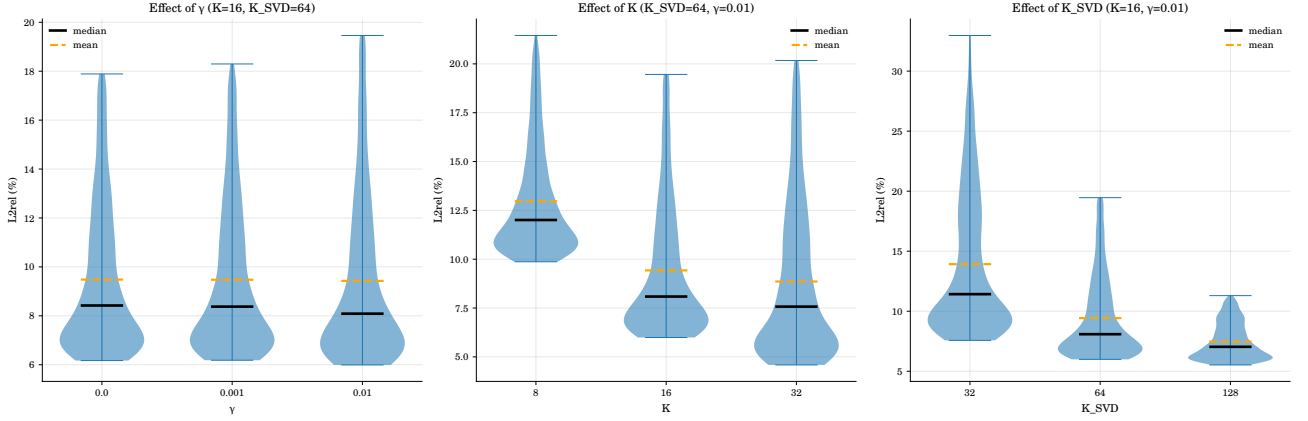


Figure 26: POD reconstruction ablation (test set, median $\pm$ IQR, violin plots; black line = median, dashed orange = mean). From left to right: effect of the damping parameter γ ($K = 16, k_{\text{pod}} = 64$), effect of the number of Laplace frequencies K ($k_{\text{pod}} = 64, \gamma = 0.01$), and effect of the number of POD modes k_{pod} ($K = 16, \gamma = 0.01$).

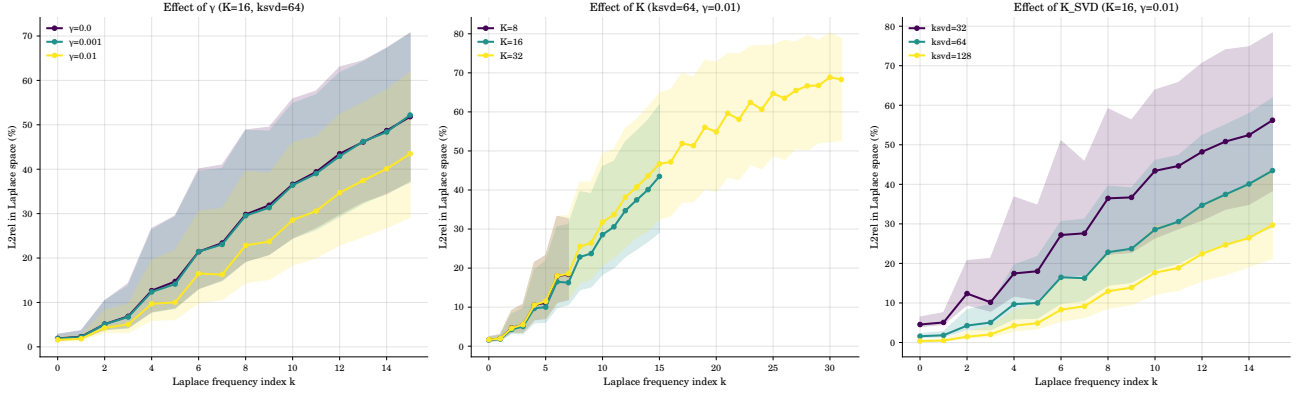


Figure 27: POD reconstruction — per-frequency Laplace-domain error (median $\pm$ IQR, test set). Left: effect of γ ; centre: effect of K ; right: effect of k_{pod} . The left panel exposes the trade-off discussed in Section 3.2.1: the Laplace-domain error is systematically **lower** for $\gamma = 0$ at every frequency index, yet the net physical-field error is nearly unchanged, because the Tikhonov inversion error moves in the opposite direction.

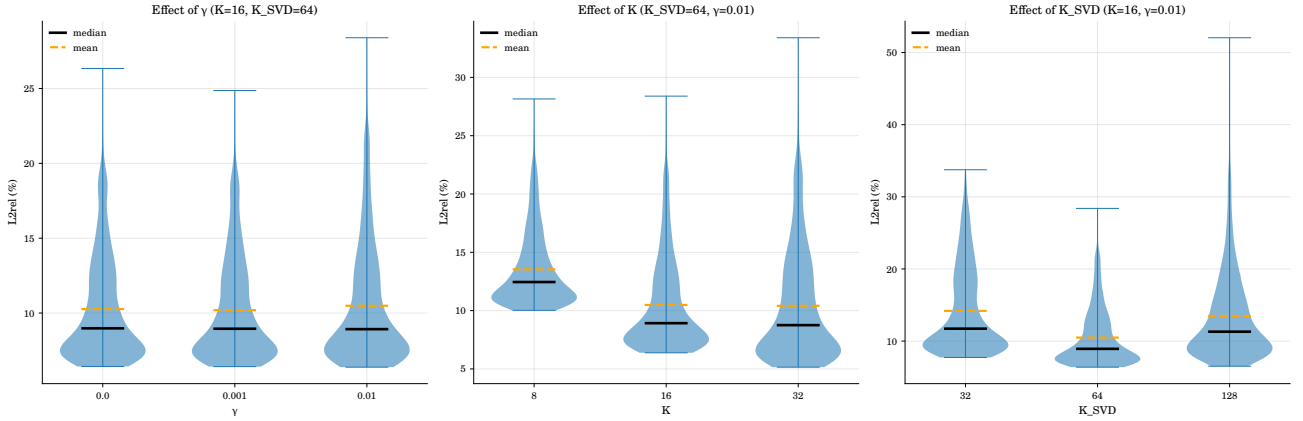


Figure 28: POD surrogate ablation (test set, median $\pm$ IQR). From left to right: effect of γ ($K = 16$, $k_{\text{pod}} = 64$), effect of K ($k_{\text{pod}} = 64$, $\gamma = 0.01$), and effect of k_{pod} ($K = 16$, $\gamma = 0.01$). Black line = median, dashed orange = mean.

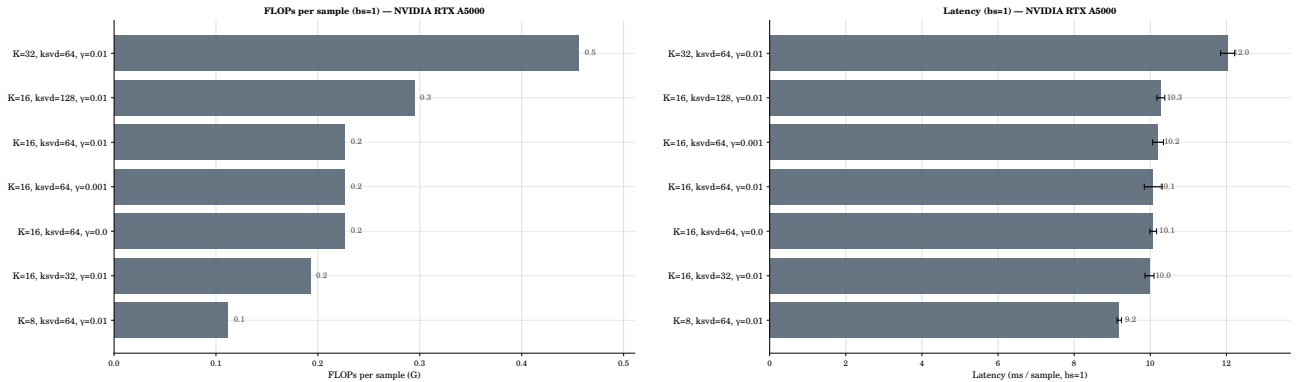


Figure 29: POD surrogate inference benchmark (50 timed CUDA Event runs, 10 warm-up runs). Left: per-sample latency at batch size 1, sorted by latency. Right: throughput (samples/s) vs. k_{pod} at batch size 32.

C Surrogate Ablation Studies

Figures 30 and 31 show the effect of each hyperparameter on the end-to-end surrogate error, using violin plots of the full test-set distribution (one hyperparameter varied at a time, others fixed at the baseline values of Table 14).

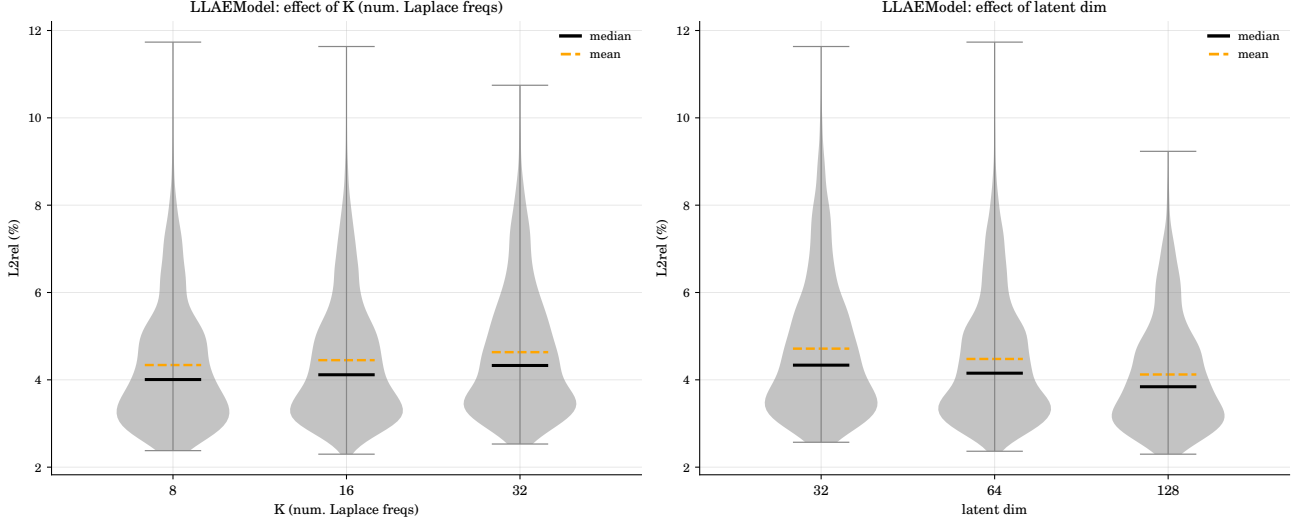


Figure 30: LLAE surrogate ablation (test set, $n = 1620$, violin plots; black line = median, dashed orange = mean). From left to right: effect of the number of Laplace frequencies K ($d_z = 64$, $\gamma = 0.01$) and effect of the latent dimension d_z ($K = 16$, $\gamma = 0.01$). The surrogate error varies by less than 0.5 pp across all LLAE configurations.

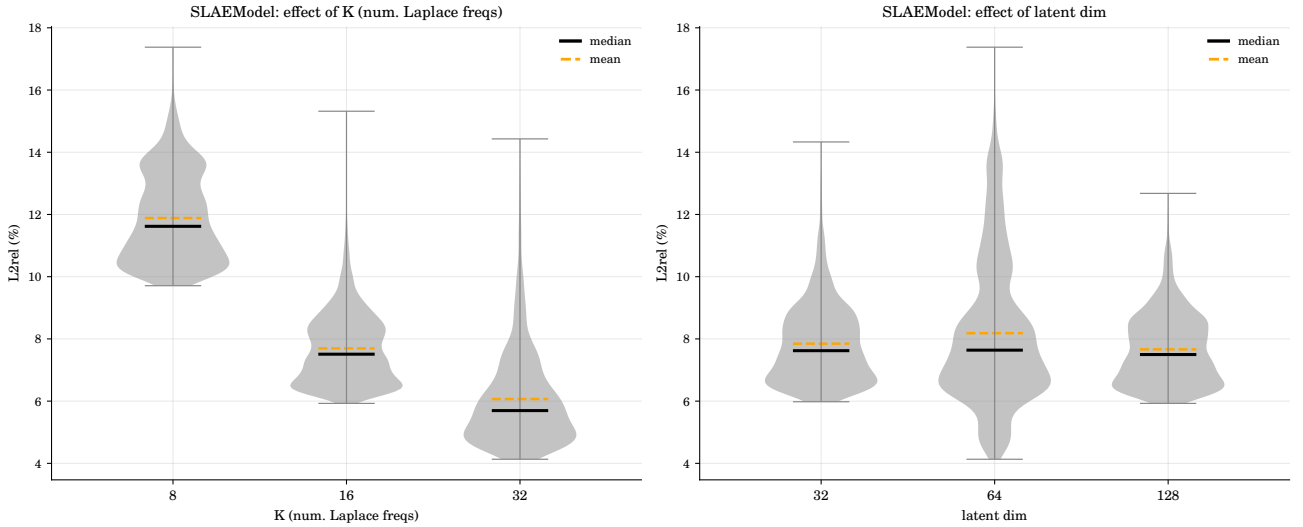


Figure 31: SLAE surrogate ablation (test set, $n = 1620$). Same layout as Figure 30. SLAE error is strongly sensitive to K : $K = 8$ degrades to 11.6% while $K = 32$ achieves the best SLAE result (5.7%). Note the larger y-axis range compared to LLAE.

D Optimised Contours at the Reconstruction Stage

Section 3.5 assesses the two contour-selection strategies at the surrogate stage, which is the deliverable. This appendix reports their behaviour at the reconstruction stage, where the two criteria diverge most visibly. Figure 32 summarises the six configurations.

The learnable contour lowers the reconstruction floor at every K . The most accurate LLAE autoencoder of the study is obtained at $K = 32$ (2.61% median error), and even $K = 8$ (2.89%) improves upon every prescribed-contour instance of Table 4, including the one at $d_z = 128$ (2.92%): at the reconstruction stage, optimising the contour is worth more than doubling the latent dimension, and at no additional inference cost since the sampling points enter only the transform. The residual sensitivity to K also all but vanishes (2.61–2.89% over a fourfold variation of K), consistent with a contour that redistributes the sampling points towards the spectral region actually carrying the signal energy. None of this transfers to the end-to-end error (Section 3.5), the Gap widening from approximately +0.9 to +1.9 pp: the representation becomes sharper to reconstruct and harder to regress from μ .

The offline contour degrades the reconstruction instead, by +0.66 and +0.19 pp at $K = 8$ and $K = 32$ and by +7.13 pp at $K = 16$. The penalty is analysed in Section 3.5: it is incurred below the crossover error ϵ^* of (41), and is subsequently absorbed by the decoder during Phase 2.

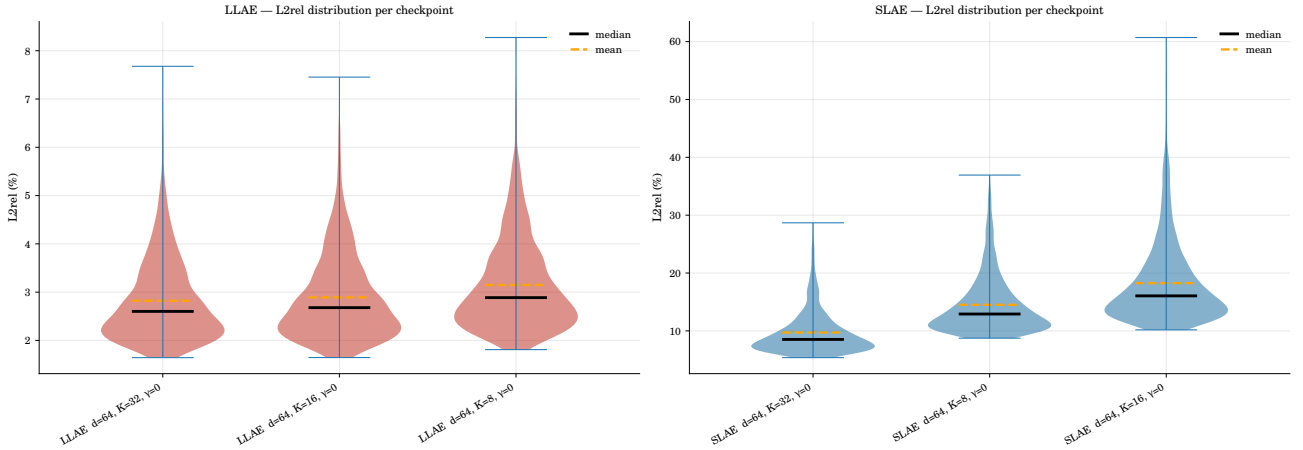


Figure 32: Standalone reconstruction error under the optimised contours: distribution of the relative L^2 error on the test set ($n = 1620$), one panel per family, configurations ordered by median. Black line: median; dashed orange line: mean. Left: LLAE, learnable contour, whose three configurations are tightly grouped between 2.61 and 2.89%. Right: SLAE, offline contour, on a y-axis an order of magnitude wider and with pronounced upper tails. The ordering is the reverse of that obtained at the surrogate stage (Figure 16), reflecting the negative Gap of Table 10.

E Governing Equations

Our neural surrogate model is designed to emulate the transient dynamics of a reacting flow system. The physical system is governed by a set of coupled partial differential equations representing the conservation of mass, momentum, energy, and chemical species.

1. Fluid Mechanics (Momentum):

$$(42) \quad \frac{\partial(\rho \mathbf{U})}{\partial t} + \nabla \cdot (\rho \mathbf{U} \mathbf{U}) - \nabla \cdot \boldsymbol{\tau} = -\nabla p + \rho \mathbf{g}$$

2. Chemical Species Transport:

$$(43) \quad \frac{\partial(\rho Y_i)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y_i) - \nabla \cdot (\mu_{\text{eff}} \nabla Y_i) = \dot{\omega}_{i,\text{eff}}$$

under the constraint $\sum Y_i = 1$.

3. Energy:

$$(44) \quad \frac{\partial(\rho h)}{\partial t} + \nabla \cdot (\rho \mathbf{U} h) + \frac{\partial(\rho K)}{\partial t} + \nabla \cdot (\rho \mathbf{U} K) - \frac{\partial p}{\partial t} - \nabla \cdot (\alpha_{\text{eff}} \nabla h) = \dot{q}_{\text{chem}}$$

where $K = \frac{1}{2} |\mathbf{U}|^2$ is the kinetic energy.

4. Combustion Model (PaSR): The effective reaction rate is modulated by the Partially Stirred Reactor model:

$$(45) \quad \dot{\omega}_{i,\text{eff}} = \kappa \dot{\omega}_i(c^*, T^*), \quad \text{with} \quad \kappa = \frac{\tau_c}{\tau_c + \tau_{\text{mix}}}$$

where τ_c and τ_{mix} are the characteristic chemical and turbulent mixing times, respectively.

5. Equation of State: The ideal gas law closes the system:

$$(46) \quad p = \rho R T \quad \text{or} \quad \rho = \psi p \quad \text{with} \quad \psi = \frac{1}{RT}$$

F Architectural Details and Setup

Hardware and software. Every model of this study was trained, and every cost figure measured, on identically configured workstations of the same pool: an Intel Xeon W-1270P CPU (8 cores, 16 threads) with 62 GB of RAM and a single NVIDIA RTX A5000 GPU (24 GB). The software stack is Python 3.13, PyTorch 2.12 (CUDA 13.0) and PyTorch Lightning 2.6. All inference-cost figures reported in this paper — latency, FLOPs, peak GPU memory and energy — were collected on one such idle machine, at batch size 1 unless stated otherwise, with 50 timed CUDA Event runs after 10 warm-up runs; the device is not named again in the individual captions.

Optimisation protocol. Both phases share the same protocol, and it is the one written in the configuration files distributed with the code; the only departures are the sweep overrides named in the tables below. Optimiser AdamW with weight decay 10^{-4} ; learning-rate schedule `ReduceLROnPlateau` (factor 0.5, patience 15, floor 10^{-6}) driven by the monitored validation quantity; mixed 16-bit precision; gradient-norm clipping at 1.0; at most 300 epochs with early stopping at patience 40. The monitored quantity is the validation loss for the autoencoders and the validation relative L^2 error for the surrogates, and it also selects the checkpoint that is subsequently evaluated. The global seed is 42 for every run.

Data. The partition is the one described in Section 3.1: 5,184 training, 1,296 validation and 1,620 test instances, drawn by a fixed permutation of seed 42 and identical across every model, phase and hyperparameter configuration of this study. No test instance enters any training or selection decision at any point. Fields are standardised pixel-wise using training-set statistics; for SLAE the Laplace frames are standardised per frequency and per pixel, and the compressed targets of the SVD and Tucker variants are standardised as well, so all losses below are evaluated in normalised space. The forward quadrature uses the trapezoidal rule on the $N_t = 150$ nodes of the $\Delta t = 1$ s grid.

Table 14 summarises the autoencoders (Phase 1) and Table 15 the surrogates (Phase 2).

Group	Hyperparameter	SLAE	LLAE
Architecture (baseline [†])	Latent dim d_z	64	64
	Frequencies K	16	16
	Contour γ	0.01	0.01
	Conditioning coord.	$k/(K-1)$	$t/(N_t-1)$
	Sinusoidal levels L	8	8
Backbone	Grid $N \times N$	128^2	128^2
	Input channels	2 (Re, Im)	1
	Encoder channels	$16 \rightarrow 32 \rightarrow 64$, then FC $\rightarrow 2d_z \rightarrow d_z$	
	Decoder channels	$128 \rightarrow 128 \rightarrow 64 \rightarrow 32$	$64 \rightarrow 64 \rightarrow 32 \rightarrow 16$
	Decoder norm	BatchNorm	GroupNorm(8)
	Decoder output heads	2 (Re, Im)	1
Regularization	Ridge β on z	10^{-2}	10^{-2}
	Latent roundtrip β_{latent}	—	0.1
	Tikhonov α_t	7×10^{-3} (eval)	7×10^{-3}
	Tikhonov λ	3×10^{-5} (eval)	3×10^{-5}
Training	Epochs (max)	300	300
	Batch size	$16^\ddagger$	16
	Learning rate	5×10^{-4}	5×10^{-4}
	Weight decay	10^{-4}	10^{-4}
	Early stopping patience	40	40
	Monitored quantity	validation reconstruction loss	

Table 14: Autoencoder hyperparameters (Phase 1), as set in `configs/model` and `configs/training/ae.yaml`. FiLM modulation is $x \mapsto x(1 + \tanh \alpha) + \delta$ at every convolutional stage, the $\tanh$ bounding the gain in $(0, 2)$; the last convolution of each decoder head is zero-initialised. [†]Baseline values held fixed when the other architectural parameters are swept: $d_z \in \{32, 64, 128\}$, $K \in \{8, 16, 32\}$, $\gamma \in \{0, 0.001, 0.01\}$. [‡]The batching unit differs between families: a batch of 16 groups 16 (simulation, frequency) pairs for SLAE, which encodes frequency frames independently, against 16 complete simulations for LLAE, which encodes the whole trajectory — the two figures are therefore not comparable in memory or in gradient noise.

Group	Hyperparameter	SLAE	LLAE
Architecture (fixed)	Trunk layers n_{trunk}	4	4
	Trunk hidden dim	512	512
	Head layers n_{head}	2	2
	Head hidden dim	256	256
	Frequency encoding levels	$L = 6$, raw sin/cos (no MLP)	
Loss	Latent weight α_{lat}	1.0	1.0
	Spatial weight α_{spat}	1.0	1.0
	Tikhonov α_t	7×10^{-3}	7×10^{-3}
	Tikhonov λ	3×10^{-5}	3×10^{-5}
Trainable parts	Encoder	frozen	
	Surrogate MLP	3×10^{-4} direct, 5×10^{-4} SVD/Tucker	
	Decoder (fine-tuned)	$lr = 5 \times 10^{-5}$	
	Poles (s_k, α_t, λ)	frozen	10^{-5} if learnable
Compression variants	SVD rank k_{svd}	$\{8, 16, 32\}$, basis frozen	
	Tucker ranks (r_s, r_z)	$\{4, 8, 16\} \times \{8, 16, 32\}$, frozen	
	HOOI iterations	5	
Training	Epochs (max)	300	300
	Batch size	16	16
	Weight decay	10^{-4}	10^{-4}
	Early stopping patience	40	40
	Monitored quantity	validation relative L^2 error	

Table 15: Surrogate hyperparameters (Phase 2), as set in `configs/training/surrogate_*.yaml`. One surrogate is trained per autoencoder checkpoint, inheriting its d_z , K and contour. The encoder is frozen throughout; the decoder is fine-tuned at a reduced learning rate, and the SVD basis and Tucker factors, computed offline, are held fixed. The DL-ROM baseline uses this same table with $N_t = 150$ per-time-step heads in place of the K frequency heads.

G Derivations for the Bias–Amplification Contour Optimization

Throughout this appendix, $\mathbf{F} \in \mathbb{C}^{K_{\text{full}} \times N_t}$ and the coefficient vectors are those assembled on the conjugate-symmetric set $\mathcal{S}$ of (2), as in Sections 2.1 and 2.4; on $\mathcal{S}$ the matrix $\mathbf{F}^H \mathbf{F}$ and the vector $\mathbf{F}^H \hat{\mathbf{u}}$ are real, so the operator $\Re(\cdot)$ below may be read as an identity and is kept only for uniformity with (4).

Proposition G.1 (Invertibility of $\mathbf{A}$). *Since $\lambda > 0$ and $\mathbf{D}_t^T \mathbf{D}_t$ is positive semi-definite, $\mathbf{L}_{\text{reg}} = \alpha_t \mathbf{D}_t^T \mathbf{D}_t + \lambda \mathbf{I}$ is positive definite ($\mathbf{L}_{\text{reg}} \succeq \lambda \mathbf{I}$). Thus $\mathbf{A} = \Re(\mathbf{F}^H \mathbf{F}) + \mathbf{L}_{\text{reg}}$ is symmetric positive definite and hence invertible. The unique reconstructed solution is $\mathbf{u}^*(\boldsymbol{\mu}) = \mathbf{A}^{-1} \Re(\mathbf{F}^H \hat{\mathbf{u}}(\boldsymbol{\mu}))$.*

Derivation of the bound (25). Starting from decomposition (23) and applying the triangle inequality:

$$\|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\| \leq \|\mathbf{u}(\boldsymbol{\mu}) - \mathbf{u}^*(\boldsymbol{\mu})\| + \|\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\|.$$

The first term equals $\|\mathbf{A}^{-1} \mathbf{L}_{\text{reg}} \mathbf{u}(\boldsymbol{\mu})\|$ by (24). For the second term:

$$\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu}) = \mathbf{A}^{-1} \Re\left(\mathbf{F}^H (\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu}))\right).$$

Applying sub-multiplicativity of the operator norm and $\|\Re(M)\| \leq \|M\|$ yields:

$$\|\mathbf{u}^*(\boldsymbol{\mu}) - \mathbf{u}_{\text{rec}}(\boldsymbol{\mu})\| \leq \|\mathbf{A}^{-1} \mathbf{F}^H\| \cdot \|\hat{\mathbf{u}}(\boldsymbol{\mu}) - \tilde{\mathbf{u}}(\boldsymbol{\mu})\|_{\mathbb{C}^{K_{\text{full}}}},$$

which gives (25). $\square$

On the norm used in practice. The derivation above uses the operator 2-norm, for which sub-multiplicativity holds. The implementation evaluates the amplification factor as the Frobenius norm of the real $2N_t \times K_{\text{full}}$ stack $[\mathbf{A}^{-1} \Re(\mathbf{F}^H); \mathbf{A}^{-1} \Im(\mathbf{F}^H)]$, which is exactly $\|\mathbf{A}^{-1} \mathbf{F}^H\|_F$. Since $\|\cdot\|_2 \leq \|\cdot\|_F$, substituting it in (25) and in the proxy (31) preserves the inequality; it is preferred because it is a smooth function of every entry, hence better behaved under gradient descent than the largest singular value.

Proposition G.2 (SVD residual as a floor for the **optimal** autoencoder). *Let $\mathcal{H}_{d_z}$ be the class of encoder–decoder pairs with bottleneck dimension d_z realizable by the chosen architecture and acting on the stacked space–frequency representation $\mathbf{a}(\boldsymbol{\mu}) \in \mathbb{R}^{2N_x K}$, and let $\epsilon_{\text{AE}}^* = \inf_{\mathcal{H}_{d_z}} \epsilon$ be the infimum of the reconstruction error over that class. If $\mathcal{H}_{d_z}$ contains all linear encoder–decoder pairs of the same bottleneck dimension, then $\epsilon_{\text{AE}}^* \leq \mathcal{E}_{\text{SVD}}$.*

Derivation. A linear autoencoder with bottleneck dimension d_z minimizing the mean squared reconstruction error recovers exactly the truncated SVD projection [1], whose error is $\mathcal{E}_{\text{SVD}}$ by definition. A nonlinear autoencoder with the same bottleneck optimizes over a class containing all such linear pairs, so its infimum is no larger. $\square$

Which class is bounded. The hypothesis class here compresses the **whole** space–frequency field of a simulation into d_z coefficients, whereas SLAE compresses each of the K frequency frames into d_z coefficients separately, and thus disposes of Kd_z in total. The proposition therefore does not bound SLAE’s attainable error; $\mathcal{E}_{\text{SVD}}$ is the floor of a strictly more constrained reduction, and is used in (31) only as a contour-dependent scalar whose variation is expected to track that of the surrogate error. The per-frequency variant $\sum_k \mathcal{E}_{\text{SVD}}(s_k)^2$, which would bound SLAE’s class proper, is the natural alternative and was not the one used here.

Scope of the statement. The inequality bounds the **optimum** over the hypothesis class, under exact global minimisation and unlimited data. It says nothing about the error of a network actually trained by stochastic gradient descent for a finite number of epochs under a particular loss, which is what Table 4 measures — and which is bounded by no linear residual: SLAE exceeds the linear reconstruction floor of Section 3.2.1 in four of its seven configurations (up to +0.94 pp). Two effects that the proposition explicitly excludes account for this. SLAE is trained on a Laplace-domain objective that is not the physical-field error being reported, so the quantity it minimises is not the quantity the comparison scores; and the attained optimum is a local one. The proposition is used in Section 2.4 only as a computable, deterministic **proxy** whose **variation** with the contour tracks that of the surrogate error, not as a guarantee on any trained model; the scaling factor λ_{AE} introduced there absorbs the residual mismatch in level.

References

- [1] Pierre Baldi and Kurt Hornik. Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks*, 2(1):53–58, 1989.
- [2] Qianying Cao, Somdatta Goswami, and George Em Karniadakis. LNO: Laplace neural operator for solving differential equations, 2023.
- [3] L. De Lathauwer, B. De Moor, and J. Vandewalle. On the best rank-1 and rank- $(r_1, r_2, \dots, r_n)$ approximation of higher-order tensors. *SIAM Journal on Matrix Analysis and Applications*, 21(4):1324–1342, 2000.
- [4] Mostafa Dehghani, Yi Tay, Anurag Arnab, Lucas Beyer, and Ashish Vaswani. The efficiency misnomer. *International Conference on Learning Representations (ICLR)*, 2022.
- [5] X. Fan, Y. Lin, B. Gong, and H. Li. Offline meteorology-pollution coupling global air pollution forecasting model with bilinear pooling, 2025.
- [6] Stefania Fresca, Luca Dedè, and Andrea Manzoni. A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized pdes. *Journal of Scientific Computing*, 87(2):61, 2021.
- [7] Stefania Fresca and Andrea Manzoni. POD-DL-ROM: Enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. *Computer Methods in Applied Mechanics and Engineering*, 388:114181, 2022.
- [8] Matteo Giacomini and Pedro Díez. Surrogates for physics-based and data-driven modelling of parametric systems, 2026. Preprint. TODO: confirm venue.
- [9] Silke Glas, Antonia Mayerhofer, and Karsten Urban. Two ways to treat time in reduced basis methods. In Peter Benner, Mario Ohlberger, Anthony Patera, Gianluigi Rozza, and Karsten Urban, editors, *Model Reduction of Parametrized Systems*, volume 17 of *MS&A*, pages 113–128. Springer, 2017.
- [10] Goeckner et al. Model order reduction in contour integral methods for parametric PDEs, 2022. Preprint. TODO: complete author list and venue.

- [11] Nicola Guglielmi, María López-Fernández, and Giancarlo Nino. Numerical inverse Laplace transform for convection-diffusion equations, 2020. TODO: confirm venue (Mathematics of Computation, vol. 89?).
- [12] Q. Guo, M. Ren, S. Wu, et al. Applications of artificial intelligence in the field of air pollution: a bibliometric analysis. *Frontiers in Public Health*, 10:933665, 2022.
- [13] N. Halko, P.-G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.
- [14] Jan S Hesthaven, Gianluigi Rozza, and Benjamin Stamm. *Certified Reduced Basis Methods for Parametrized Partial Differential Equations*. Springer, 2015.
- [15] Jan S Hesthaven and Stefano Ubbiali. Non-intrusive reduced order modeling of nonlinear problems using neural networks. *Journal of Computational Physics*, 363:55–78, 2018.
- [16] Michael Urs Lars Kastor, Jan Rottmayer, Anna Hundertmark, and Nicolas Ralph Gauger. Parameter conditioned interpretable U-Net surrogate model for data-driven predictions of convection-diffusion-reaction processes, 2026. Preprint. TODO: confirm venue.
- [17] T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.
- [18] Toni Lassila, Andrea Manzoni, Alfio Quarteroni, and Gianluigi Rozza. Model order reduction in fluid dynamics: Challenges and perspectives. In Alfio Quarteroni and Gianluigi Rozza, editors, *Reduced Order Methods for Modeling and Computational Reduction*, volume 9 of *MS&A*, pages 235–274. Springer, 2014.
- [19] Kookjin Lee and Kevin T Carlberg. Model reduction of dynamical systems on non-linear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.
- [20] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.08895.
- [21] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.
- [22] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*, 393:114778, 2022.
- [23] Swaroop Mallick and Monika Mittal. AI-based model order reduction techniques: A survey, 2025. Preprint. TODO: confirm venue.
- [24] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020.

- [25] Jad Mounayer, Sebastian Rodriguez, Chady Ghnatios, Charbel Farhat, and Francisco Chinesta. Rank reduction autoencoders, 2025. Preprint. TODO: confirm venue.
- [26] Pranshu Pant, Ruchit Doshi, Pranav Bahl, and Amir Barati Farimani. Deep learning for reduced order modelling and efficient temporal evolution of fluid simulations. *Physics of Fluids*, 33(10), 2021.
- [27] Eric Parish, Masayuki Yano, Irina Tezaur, and Traian Iliescu. Stabilization of CDR ROMs, 2025. Preprint. TODO: confirm venue.
- [28] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [29] Ricardo Reyes. The frequency reduced-basis method: Reduced order models for time-dependent problems using the Laplace transform, 2025. Preprint. TODO: confirm venue.
- [30] Tian Wang and Chuang Wang. Latent Neural Operator for solving forward and inverse PDE problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.